MAANG / FAANG CONCURRENCY INTERVIEW MASTERY SUITE (BILINGUAL)

Aptala Anlatır Gibi Java Concurrency

Explain Like I'm 5: MAANG Java Concurrency & Multithreading Masterclass

10 Bölüm • 60 Ders • Bellek & Thread Şemaları • Java 25 LTS • Virtual Threads • JMM • Lock-Free

---

⚡ JMM & Memory Model

Happens-Before, volatile, Memory Barriers, CPU Caches, False Sharing & CAS

🔒 Synchronization & Locks

HotSpot Monitors, ReentrantLock, StampedLock, Semaphore & Phaser

🧵 Project Loom & Lock-Free

Virtual Threads, Structured Concurrency, ScopedValue & Disruptor RingBuffer

Yayın Sürümü: 1.0.0 • Modern Java 25 LTS & Project Loom • 2026

Önsöz ve Başlarken / Preface & Getting Started

🇹🇷 Bu Eğitim Hakkında

Bu kitap; Meta, Amazon, Apple, Netflix, Google (MAANG) ve yüksek frekanslı finans (HFT / FinTech) teknoloji şirketlerinin **Senior, Staff ve Principal Yazılım Mühendisi** mülakatlarına hazırlanan mühendisler için özel olarak hazırlanmıştır.

Java Concurrency; yalnızca `Thread` açıp `synchronized` yazmaktan ibaret değildir. Donanımsal L1/L2/L3 CPU önbellekleri, MESI önbellek tutarlılık protokolü, Store Buffer, bellek bariyerleri (Memory Barriers), Java Bellek Modeli (JMM Happens-Before), HotSpot kilitleme optimizasyonları ve kilit-siz (Lock-Free) CAS veri yapıları gibi donanım ve çalışma zamanı dinamiklerini derinlemesine kavramayı gerektirir.

Ayrıca modern **Java 25 LTS** ile birlikte gelen Project Loom Sanal Thread'leri (Virtual Threads M:N), Taşıyıcı Thread (Carrier Thread) montajı, Pinning tehlikesi, Yapısal Eşzamanlılık (StructuredTaskScope) ve ScopedValue devrimi çoklu iş parçacığı paradigmasını kökten değiştirmiştir.

Bu 60 derslik kapsamlı müfredatta;

- İşletim sistemi çekirdek thread'leri ile JVM thread durumlarını,
- Volatile, atomiklik ve False Sharing önbellek satırı dolgusunu,
- HotSpot Monitör şişmesini (Biased, Lightweight, Heavyweight Lock),
- ReentrantLock, StampedLock, CountDownLatch, Semaphore ve Phaser primitiflerini,
- ThreadPoolExecutor büyüme dinamiklerini ve Fork/Join Work-Stealing algoritmasını,
- ConcurrentHashMap iç yapısını (Node CAS, Bin-Level Kilitleri ve TreeBin),
- CompletableFuture asenkron boru hatlarını ve zaman aşımı yönetimini,
- Sanal Thread'leri, Carrier unmounting döngüsünü ve Pinning önleme stratejilerini,
- Deadlock, Livelock, Starvation ve ABA problemlerini ve thread dump analizini,
- MAANG beyaz tahta sistem tasarımı vakalarını (O(1) LRU Cache, LMAX Disruptor, Token Bucket, HikariCP ConcurrentBag)

bir aptala anlatır gibi en yalın analojiler, mimari ASCII akış şemaları ve çalışan modern Java 25 kodlarıyla inceleyeceksiniz.

🇬🇧 About This Masterclass

This handbook is engineered for software engineers, distributed systems architects, and low-latency engineers targeting Senior, Staff, and Principal roles at Tier-1 tech giants (MAANG / FAANG) and High-Frequency Trading (HFT) firms.

Java Concurrency transcends simple locking and threads. True mastery demands an uncompromising understanding of CPU hardware caches, StoreLoad memory barriers, the Java Memory Model (JMM happens-before guarantees), HotSpot lock inflation lifecycles, and lock-free atomic CAS primitives.

Furthermore, the arrival of modern **Java 25 LTS** and Project Loom has redefined the concurrency landscape: M:N Virtual Threads, carrier thread mounting/unmounting, carrier pinning hazards, Structured Concurrency (`StructuredTaskScope`), and immutable `ScopedValue` context propagation.

Across 60 deep-dive lessons organized into 10 structured modules, and powered by **Java 25 LTS**, you will master:

- OS kernel threads, 6 JVM states, and hardware cache lines,
- Volatile visibility semantics, memory barriers, and false sharing,
- HotSpot monitor inflation, lightweight spin-locks, and ReentrantLock,
- StampedLock optimistic reads, Semaphore pooling, and dynamic Phasers,
- ThreadPoolExecutor queue dynamics and Fork/Join work-stealing algorithms,
- Modern ConcurrentHashMap internals (Node CAS, bin-locking, and TreeBins),
- Reactive CompletableFuture DAG pipelines, scatter-gather, and fallbacks,
- Project Loom Virtual Threads, Epoll continuation unmounting, and pinning remedies,
- Coffman deadlock forensics, live production thread dump analysis (jstack/jcmd),
- Legendary MAANG concurrency whiteboard challenges: O(1) LRU Cache, LMAX Disruptor RingBuffer, Token-Bucket Rate Limiter, and HikariCP ConcurrentBag.

İçindekiler / Table of Contents

🇹🇷 Bu Eğitim Hakkında 2
🇬🇧 About This Masterclass 2

BÖLÜM 1 / MODULE 1

Thread'ler, İşletim Sistemi Çekirdeği & JMM Temelleri

Threads, OS Kernels & Java Memory Model (JMM) Fundamentals

Process vs Thread farkı, JVM thread durumları, donanım önbellekleri (L1/L2/L3), False Sharing, JMM Happens-Before garantisi ve modern Java 25 thread oluşturma mekanizmaları.

Process vs Thread dichotomy, JVM thread states, CPU cache hierarchies (L1/L2/L3), False Sharing, the JMM Happens-Before relationship, and modern Java 25 thread creation.

1

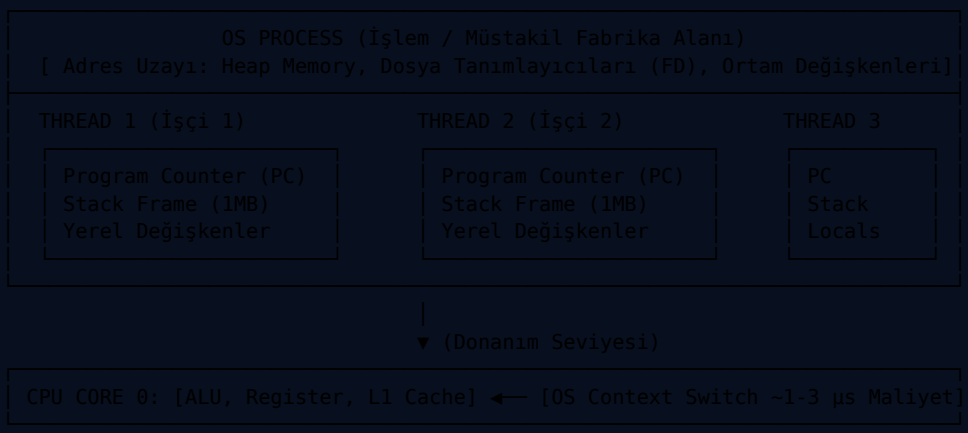
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Thread Nedir? Process vs Thread, OS Context Switching & CPU Çekirdekleri

What is a Thread? Process vs Thread, OS Context Switching & CPU Cores

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir Process (İşlem), etrafı yüksek duvarlarla çevrili koca bir fabrikadır. Kendi su tesisatı, elektrik sayacı ve tapusu (ayrı RAM adres alanı) vardır. Bir fabrikanın diğer fabrikanın odalarına izinsiz girmesi yasaktır! Thread (İş parçacığı) ise o fabrikanın içinde koşturan işçilerdir. Bütün işçiler aynı fabrikanın ortak deposunu (Heap Memory) paylaşırlar; ama her işçinin cebinde kendi özel not defteri (Thread Stack) vardır. İki işçi aynı depodaki aynı kutuya (Paylaşılan Değişken) haberleşmeden aynı anda dalarsa ortalık karışır (Race Condition)!

🇬🇧 ENGLISH (ELI5)

A Process is an entire secure factory compound with its own private land, utilities, and security gate (isolated virtual address space). Process A cannot inspect Process B's floor plan. A Thread is a worker inside that factory. All workers walk the same shared factory warehouse floor (Heap Memory), but each worker carries a personal pocket notepad (Thread Stack). If two workers update the same inventory box simultaneously without coordination, inventory records corrupt instantly (Race Condition)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG seviyesinde Process vs Thread anatomisi: 1) Bellek İzolasyonu: Process'ler MMU (Memory Management Unit) ve Page Table ile sanal adres alanlarında tamamen izole edilir. İletişim IPC (Inter-Process Communication: Sockets, Shared Memory, Pipes) gerektirir ve pahalıdır. 2) Thread Paylaşımı: Bir JVM process'indeki tüm platform thread'leri Heap, Metaspace ve dosya tanıtıcılarını (FD) paylaşır; ancak her thread kendi Stack alanına (1MB varsayılan), Program Counter (PC) register'ına ve Native Method Stack'ine sahiptir. 3) Context Switch Maliyeti: İşletim sistemi çekirdeği (Kernel) iki platform thread'i arasında geçiş yaparken CPU register'larını kaydeder, TLB (Translation Lookaside Buffer) önbelleğini temizler ve L1/L2 CPU cache'lerini bozar (Cache Pollution). Bu maliyet 1-3 mikrosaniyedir ve saniyede milyonlarca istek alan MAANG sistemlerinde verim kaybına yol açar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG-level architectural mechanics: 1) Address Space Isolation: Processes operate in distinct virtual memory boundaries governed by the hardware MMU and OS page tables. Cross-process IPC requires context-switching into ring-0 kernel space. 2) Intra-Process Thread Concurrency: Java platform threads map 1:1 to OS native threads (POSIX pthreads / Windows kernel threads). They share the JVM Heap and Metaspace, but retain private Program Counters and runtime call stacks (1MB default). 3) Context Switching Overhead: When the OS scheduler preempts Thread A for Thread B, it saves instruction pointers and registers to kernel memory, often triggering TLB invalidation and L1/L2 cache misses (Cache Pollution), incurring 1-3 microseconds of pure CPU latency per switch.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Platform Thread vs Donanım Çekirdekleri Bilgisi
void main() {
    int availableCores = Runtime.getRuntime().availableProcessors();
    long totalMemoryMb = Runtime.getRuntime().totalMemory() / (1024 * 1024);

    IO.println("CPU Donanım Çekirdek Sayısı: " + availableCores);
    IO.println("JVM Tahsis Edilen Heap Bellek: " + totalMemoryMb + " MB");

    // Geleneksel Platform Thread başlatma
    Thread osThread = new Thread(() -> {
        var current = Thread.currentThread();
        IO.println("Çalışan İşçi: " + current.getName() + " | Platform Thread mi: " + !
current.isVirtual());
    }, "MAANG-Worker-01");

    osThread.start();
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Daha fazla thread açarsak sistemimiz her zaman daha mı hızlanır?' diye sorar. Yanıt: Hayır! CPU çekirdek sayısından çok fazla platform thread açılırsa, CPU zamanını iş yapmaya değil işçiler arasında geçiş yapmaya (Context Switching thrashing) harcar ve throughput çöker!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Trap: The interviewer asks: 'Can we endlessly boost throughput simply by adding more threads?' Senior Answer: 'No! If runnable platform thread count drastically exceeds physical CPU cores, the kernel spends more clock cycles on thread context switching and cache invalidation than actual task computation, precipitating throughput collapse.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Process'ler izole adres alanlarına sahiptir; Thread'ler ise aynı Heap alanını paylaşır.

• Platform thread'leri 1:1 işletim sistemi kernel thread'lerine eşlenir (~1MB stack).

• Gereğinden fazla thread context-switching thrashing'e yol açarak sistemi kilitler.

🎯 Key Takeaways for the Interview

• Processes own isolated virtual address spaces; threads share the JVM heap.

• Platform threads map 1:1 to kernel threads, incurring ~1MB memory footprint.

• Oversubscribing threads degrades throughput via context-switch thrashing.

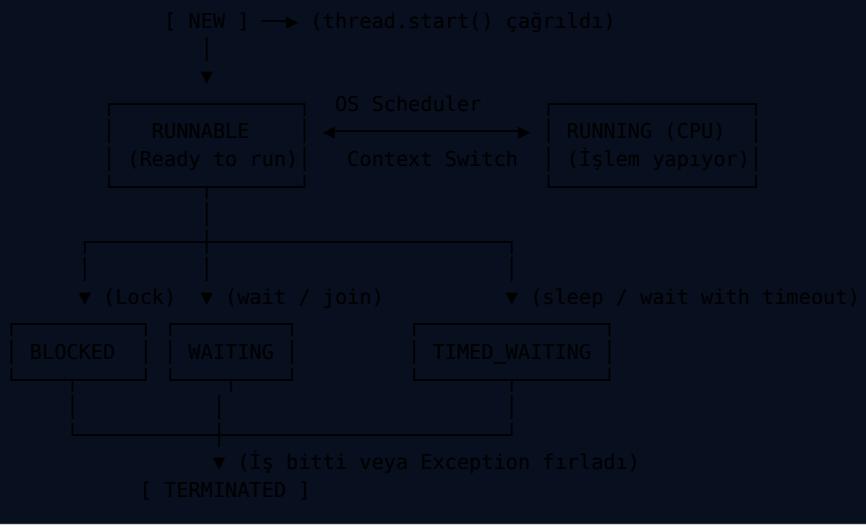
2

BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Java Thread Yaşam Döngüsü: 6 Kritik Durum (State Transitions)

Java Thread Lifecycle: The 6 Core JVM States & State Transitions

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir işçinin gün içindeki 6 hali vardır: 1) NEW: İşe yeni alındı, masasında oturuyor ama henüz 'Başla' denmedi. 2) RUNNABLE: Masasında hazır bekliyor veya harıl harıl çalışıyor. 3) BLOCKED: Tuvalet kapısının önünde bekliyor çünkü içeride biri var (Kilit kavgası). 4) WAITING: Patron 'Ben sana haber verene kadar kenarda bekle' dedi (Süresiz bekleme). 5) TIMED_WAITING: '20 dakika öğle uykusuna yat' dendi (Süreli bekleme). 6) TERMINATED: Mesaisi bitti, evine gitti!

🇬🇧 ENGLISH (ELI5)

An employee's lifecycle stages: 1) NEW: Hired and given a badge, but hasn't clocked in yet. 2) RUNNABLE: Either actively typing at the desk or standing ready for an assignment. 3) BLOCKED: Waiting outside the restroom because a colleague locked the door from inside (Lock Contention). 4) WAITING: Sitting in the lounge waiting indefinitely for a manager's signal. 5) TIMED_WAITING: Taking an exact 15-minute coffee break. 6) TERMINATED: Shift finished, clocked out forever!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Thread.State enum değerleri ve mülakat derinliği: 1) NEW: `new Thread()` yapılmış ancak `start()` çağrılmamıştır. `start()` yerine doğrudan `run()` çağrılırsa yeni thread açılmaz, çağıran thread içinde senkron çalışır! 2) RUNNABLE: JVM seviyesinde thread koşmaya hazırdır veya CPU'da çalışıyordur. JVM, işletim sisteminin 'Ready' ve 'Running' durumlarını tek bir RUNNABLE altında toplar. 3) BLOCKED: Thread bir `synchronized` bloğuna girmek için Monitor Lock (Intrinsic Lock) beklemektedir. 4) WAITING: `Object.wait()`, `Thread.join()` veya `LockSupport.park()` ile süresiz askıya alınmıştır; başka bir thread `notify()` veya `unpark()` çağırana kadar CPU tüketmez. 5) TIMED_WAITING: `Thread.sleep(ms)`, `wait(ms)` veya `parkNanos()` ile zaman sınırıyla bekler. 6) TERMINATED: `run()` metodu doğal yolla bitmiş veya yakalanmamış bir exception ile sonlanmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Thread State nuances: 1) NEW to RUNNABLE: Calling `start()` triggers OS `pthread_create()`. Invoking `run()` directly bypasses thread creation, executing synchronously on the caller's stack. 2) RUNNABLE Collapsing: The JVM intentionally does not distinguish between OS 'Ready' (in kernel runqueue) and OS 'Executing' (on CPU core); both report as RUNNABLE. 3) BLOCKED vs WAITING: A critical distinction in thread dumps! BLOCKED strictly means waiting for a synchronized monitor lock. WAITING means parked via `LockSupport.park()` or `Object.wait()`, awaiting a signal. 4) TERMINATED: Dead threads cannot be restarted. Calling `start()` on a TERMINATED thread throws `IllegalThreadStateException`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Thread Durum Geçişlerini İnceleme
void main() throws InterruptedException {
    final Object lock = new Object();

    Thread worker = new Thread(() -> {
        try {
            Thread.sleep(100); // TIMED_WAITING
            synchronized (lock) {
                lock.wait();   // WAITING
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });

    IO.println("1. Durum: " + worker.getState()); // NEW
    worker.start();
    IO.println("2. Durum: " + worker.getState()); // RUNNABLE

    Thread.sleep(50);
    IO.println("3. Durum (Sleep anı): " + worker.getState()); // TIMED_WAITING

    Thread.sleep(100);
    synchronized (lock) {
        IO.println("4. Durum (Wait anı): " + worker.getState()); // WAITING
        lock.notify();
    }

    worker.join();
    IO.println("5. Durum (Bitti): " + worker.getState()); // TERMINATED
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'ReentrantLock bekleyen bir thread hangi duruma geçer?' Yanıt: BLOCKED DEĞİL, WAITING (veya TIMED_WAITING) durumuna geçer! Çünkü ReentrantLock `LockSupport.park()` kullanır; BLOCKED durumu SADECE `synchronized` monitor lock için geçerlidir!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Interview Trap: 'What state is a thread in when blocked on a ReentrantLock?' Answer: 'WAITING (or TIMED_WAITING), NOT BLOCKED! ReentrantLock is built on AbstractQueuedSynchronizer (AQS) using LockSupport.park(). BLOCKED is strictly reserved for intrinsic synchronized monitors.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- JVM'de 6 thread durumu vardır: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- BLOCKED sadece synchronized içindir; ReentrantLock bekleyen thread WAITING olur.
- Ölmüş (TERMINATED) bir thread yeniden `start()` edilemez.

🎯 Key Takeaways for the Interview

- JVM threads traverse 6 states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- BLOCKED is exclusive to synchronized; AQS locks park threads in WAITING state.
- A TERMINATED thread cannot be restarted; doing so throws IllegalThreadStateException.

3

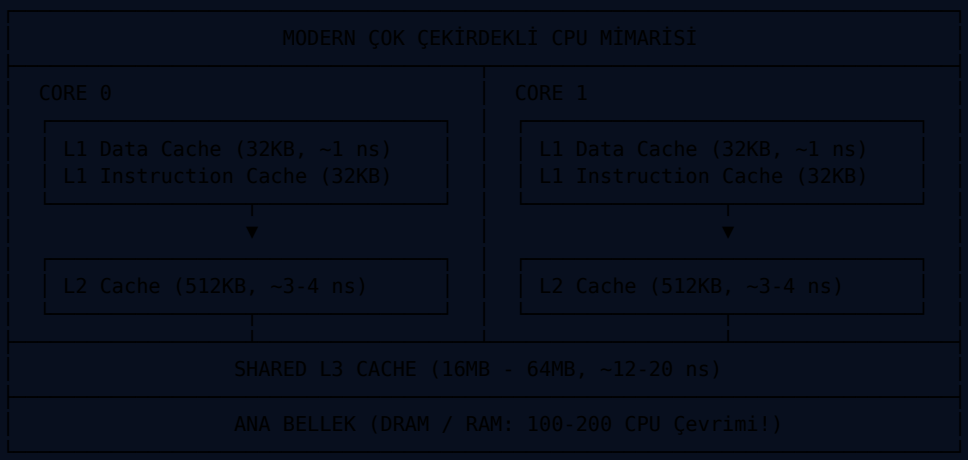
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Donanım Mimarisi: CPU Önbellekleri (L1/L2/L3), Store Buffer & False Sharing

Hardware Reality: CPU Caches (L1/L2/L3), Store Buffers & False Sharing

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

CPU o kadar hızlı koşar ki, ana belleğe (RAM) gitmek onun için 3 ay süren bir gemi yolculuğu gibidir! Bu yüzden masasının hemen üstüne minicik çekmeceler koymuştur: L1 (kol mesafesinde), L2 (odanın köşesinde), L3 (koridorda). CPU veriyi belleğe giderken tek bir bayt olarak değil, her defasında 64 baytlık paket (Cache Line) halinde getirir. Eğer iki farklı işçi, aynı 64 baytlık paketin içindeki farklı iki kutuyu güncellemeye çalışırsa, CPU'lar sürekli birbirinin çekmecesini geçersiz kılar (False Sharing). İşçiler birbirine dokunmasa bile sistem yerlerde sürünür!

🇬🇧 ENGLISH (ELI5)

CPUs execute so fast that fetching from RAM feels like waiting 3 months for cargo delivery! To survive, the core keeps micro-drawers on its desk: L1 (arm's length, 1ns), L2 (across the room, 4ns), L3 (down the hall, 15ns). Hardware never transfers a lone 4-byte integer from RAM; it fetches an entire 64-byte Cache Line. If Thread A on Core 0 modifies variable X, and Thread B on Core 1 modifies variable Y, but X and Y sit inside the same 64-byte line, cache coherency protocols (MESI) invalidate both cores' caches repeatedly (False Sharing)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MESI Protokolü ve False Sharing derinliği: 1) Cache Line: Modern x86 ve ARM64 mimarilerinde önbellek aktarım birimi 64 bayttır. 2) MESI Protokolü (Modified, Exclusive, Shared, Invalid): Bir çekirdek bir cache line'a yazdığında, donanım veri yoluna 'Invalidate' sinyali gönderir. Diğer tüm çekirdeklerin o satırdaki L1/L2 kopyaları anında çöpe gider ve RAM/L3'ten tekrar çekilmek zorunda kalır. 3) False Sharing Felaketi: Mantıksal olarak bağımsız iki `volatile` veya atomik değişken bellekte bitişik tanımlanmışsa (ör. `long a; long b;`), farklı thread'ler bunlara bağımsız yazsa bile aynı 64 baytlık satır çekirdekler arasında pinpon topu gibi gidip gelir (Bus saturation). 4) Çözüm: Java 8+ ile gelen `\@jdk.internal.vm.annotation.Contended` anotasyonu veya manuel alan doldurma (Padding: 128 bayt aralık bırakma).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Cache Coherency & False Sharing mechanics: 1) The 64-Byte Unit: Memory subsystems always populate L1/L2 caches in contiguous 64-byte aligned blocks. 2) MESI Protocol State Flips: When Core 0 writes to an address, it transitions the cache line from Shared to Modified, forcing Core 1 to mark its local copy as Invalid. 3) False Sharing Inefficiency: If Thread 1 frequently writes to `val1` and Thread 2 writes to `val2`, and both fields inhabit the same 64-byte cache line, the CPU interconnect is choked with cache-line invalidation traffic despite zero algorithmic overlap. 4) Remediation: JVM `\@Contended` annotation (requires `-XX:-RestrictContended`) pads instances with 128 dummy bytes, ensuring isolated cache lines.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: False Sharing Korumalı Cache-Line Padded Sayaç
public class CachePaddedCounter {
    // 64-baytlık satırı doldurmak için boşluk (Padding) bırakma tekniği
    public static class PaddedAtomicLong {
        // Nesne başlığı (Object Header) ~12-16 byte
        // Önceki 56 bayt doldurma
        public volatile long p1, p2, p3, p4, p5, p6, p7;

        // Asıl değerimiz (8 bayt)
        public volatile long value = 0L;

        // Sonraki 56 bayt doldurma
        public volatile long p8, p9, p10, p11, p12, p13, p14;
    }

    void main() {
        var counter = new PaddedAtomicLong();
        counter.value = 42L;
        IO.println("Cache-line ayrılmış güvenli sayaç değeri: " + counter.value);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Dizideki farklı indekslere farklı thread'ler yazarsa race condition olur mu?' diye sorar. Yanıt: 'Mantıksal race condition olmaz ama dizinin bitişik elemanları aynı 64-baytlık cache line'a düşeceği için donanımsal False Sharing felaketi yaşanır ve performans 50 kat düşebilir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

High-Impact Interview Trap: 'If Thread A writes to array[0] and Thread B writes to array[1], is there any concurrency penalty?' Senior Answer: 'There is no semantic data race, but there is severe False Sharing. Because array[0] and array[1] share a single 64-byte cache line, CPU cores continually invalidate each other's L1 caches, slashing throughput by up to 50x!'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CPU veriyi RAM'den 64 baytlık Cache Line blokları halinde transfer eder.

• False Sharing, bağımsız değişkenlerin aynı cache satırını paylaşmasıyla oluşur.

• Yüksek performanslı kuyruklar (Disruptor) ve sayaçlar (LongAdder) cache padding kullanır.

🎯 Key Takeaways for the Interview

• CPUs fetch memory in atomic 64-byte cache lines, not single primitive bytes.

• False Sharing occurs when unrelated variables occupy the same 64-byte cache line.

• High-throughput engines (e.g. LMAX Disruptor) pad memory to isolate cache lines.

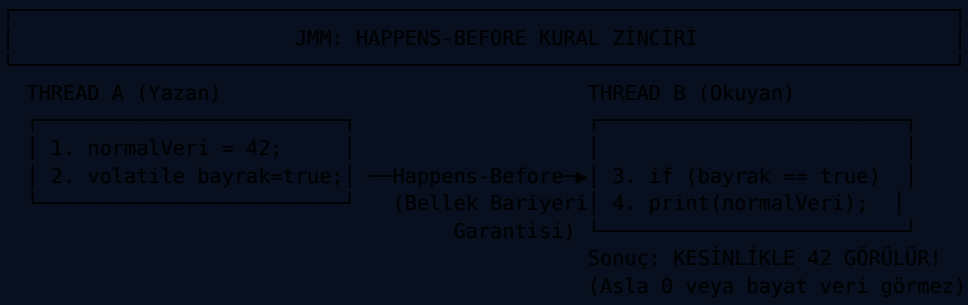
4

BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Java Memory Model (JMM): Happens-Before Garantisi Nedir?

Java Memory Model (JMM): The Happens-Before Relationship Decoded

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



TEMEL HAPPENS-BEFORE KURALLARI:
1. Program Order Rule: Aynı thread içindeki eylemler yazılış sırasıyla gerçekleşir.
2. Monitor Lock Rule: Bir kilidin bırakılması (unlock), sonraki kilitlemeden (lock) önce gelir.
3. Volatile Variable Rule: Bir volatile yazma, sonraki volatile okumadan önce gelir.
4. Thread Start Rule: thread.start() çağrısı, thread içindeki ilk eylemden önce gelir.

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki kişi farklı odalarda kağıda yazı yazıyor olsun. Biri yazdığı kağıdı çekmeceye koymadan (RAM'e basmadan) öbürü o kağıdı göremez. 'Happens-Before' kuralı, iki yabancı işçi arasındaki yasal noter sözleşmesidir: 'A işçisi bayrağı kaldırdığı anda, o bayrağı kaldırmadan önce masaya koyduğu BÜTÜN evraklar B işçisine görünür olmak zorundadır!' Eğer bu yasal sözleşme (Happens-Before) yoksa, B işçisi bayrağı kalkmış görse bile evrakları hala bomboş (0 veya null) görebilir!

🇬🇧 ENGLISH (ELI5)

Imagine two clerks working in separate soundproof rooms. Clerk A drafts a contract, then rings a service bell. The Happens-Before guarantee is the legal protocol ensuring that when Clerk B hears that bell, ALL documents Clerk A drafted prior to ringing MUST be visibly sitting in the shared transfer tray. Without a formal Happens-Before edge, Clerk B might hear the bell but open the tray to find blank paper because Clerk A's edits are still stuck in their private desk drawer (Store Buffer)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

JSR-133 ve JMM Spesifikasyonu: 1) Görünürlük Sorunu (Visibility): Modern CPU'lar yazma işlemlerini hemen ana belleğe yazmaz; Store Buffer'a atar. Happens-Before ilişkisi kurulmazsa, Thread B değişkenin güncel halini dakikalarca göremeyebilir. 2) 8 Temel Happens-Before Kuralı: a) Program Order: Tek thread içi ardışıklık. b) Monitor Lock: `unlock` -> sonraki `lock`. c) Volatile Variable: `volatile write` -> sonraki `volatile read`. d) Thread Start: `t.start()` -> `t` içindeki kodlar. e) Thread Join: t'nin bitişi -> `t.join()` dönüşü. f) Interruption: `interrupt()` -> kesintiyi tespit eden thread. g) Finalizer: Constructor bitişi -> `finalize()` başlangıcı. h) Transitivity (Geçişlilik): Eğer A -> B ve B -> C ise, A -> C'dir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

JSR-133 JMM Formalism: 1) The Memory Visibility Illusion: CPUs utilize out-of-order execution, store buffers, and invalidation queues. Memory operations do not occur globally in instant lockstep. 2) Formal Definition of Happens-Before (HB): An execution order defined over actions. If action X happens-before action Y, then the memory mutations performed by X are guaranteed visible to Y, and X is ordered before Y. 3) Transitivity Property: If action A happens-before B (via program order), and action B happens-before C (via volatile write-read or lock release-acquire), then A transitively happens-before C, safely publishing all non-volatile state preceding B.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Happens-Before Transitivite Güvencesi
public class HappensBeforeDemo {
    private int regularData = 0;
    private volatile boolean ready = false;

    public void writerThread() {
        regularData = 100; // 1. Normal yazma
        ready = true;      // 2. Volatile yazma (Happens-Before Bariyeri kurar!)
    }

    public void readerThread() {
        if (ready) { // 3. Volatile okuma
            // JMM Garantisi: ready=true ise regularData KESİNLİKLE 100'dür!
            IO.println("Veri güvenle okundu: " + regularData);
        }
    }

    void main() throws InterruptedException {
        var demo = new HappensBeforeDemo();
        Thread t1 = new Thread(demo::writerThread);
        Thread t2 = new Thread(demo::readerThread);

        t1.start();
        t1.join();
        t2.start();
        t2.join();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Ölümcül Hata: Happens-Before kurulmazsa, derleyici ve CPU kodun sırasını değiştirebilir! Yani `ready = true` işlemi `regularData = 100` işleminden ÖNCE hafızaya basılabilir! Okuyucu thread `ready == true` görür ama `regularData` hala `0` çıkar!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Fatal Interview Trap: Without an explicit Happens-Before edge, the JIT compiler and hardware can reorder instructions! The CPU might publish `ready = true` to shared memory BEFORE `regularData = 100` drains from the store buffer. The reader sees `ready == true` but reads `regularData == 0`!

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Happens-Before, bir thread'in yazmasının diğerine ne zaman görüneceğini garanti eder.
- Volatile yazma, kendisinden önceki tüm normal yazmaları da görünür kılar (Transitivity).
- Happens-Before yoksa derleyici ve CPU talimatları rastgele yeniden sıralayabilir.

🎯 Key Takeaways for the Interview

- Happens-Before formally dictates when mutations by Thread A become visible to Thread B.
- A volatile write flushes all preceding non-volatile writes via transitive happens-before.
- Without happens-before, hardware and JIT aggressively reorder memory instructions.

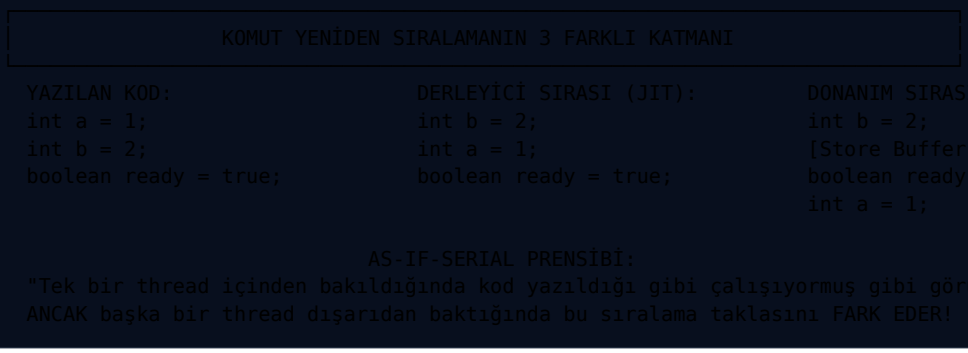
5

BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Komut Yeniden Sıralama (Instruction Reordering) & Derleyici Optimizasyonları

Instruction Reordering & Compiler Optimizations: Why Code Runs Out of Order

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir aşçı düşünün: Tarifte '1. Havuçları soy, 2. Eti marine et, 3. Tencereye yağ dök' yazıyor. Akıllı aşçı bakar ki yağın kızması 3 dakika sürüyor; önce yağı ocağa koyar, o ısınırken havuçları soyar! Tek başına mutfakta olduğu sürece yemek aynı lezzette çıkar (As-If-Serial). Ama mutfağa aniden ikinci bir çırak girip 'Yağ ocakta olduğuna göre havuçlar çoktan doğranmıştır' diye tencereye boş elini atarsa eli yanar! Çoklu thread dünyasında bu sıralama hileleri felakete yol açar!

🇬🇧 ENGLISH (ELI5)

Consider a master chef reading a recipe: '1. Peel carrots, 2. Season steak, 3. Heat oil in skillet'. Because heating oil takes 4 minutes, the chef turns on the stove burner first, then peels carrots while the pan heats up. To the chef working alone, the final meal tastes identical (As-If-Serial rule). But if an assistant rushes in and assumes 'The stove is hot, therefore chopped carrots must already be prepared!', disaster strikes. In multithreading, reordering tricks destroy uncoordinated observers!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üç Seviyeli Yeniden Sıralama: 1) Derleyici Yeniden Sıralaması (Compiler Reordering): javac ve HotSpot JIT (C1/C2), register tahsisini ve döngü verimliliğini optimize etmek için bağımsız talimatların yerini değiştirir. 2) İşlemci Yeniden Sıralaması (CPU Out-of-Order Execution): Modern süperskalar CPU'lar boru hattını (Pipeline) dolu tutmak için bağımsız yönergeleri eşzamanlı veya sıra dışı yürütür. 3) Bellek Sistemi Sıralaması (Memory Hierarchy Reordering): Store Buffer ve Invalidation Queue'lar yüzünden yazma işlemleri hemen RAM'e düşmez; okuma işlemleri bekleyen yazmaların önüne geçebilir (StoreLoad reordering). 4) As-If-Serial Semantiği: JVM ve donanım, tek thread içinde veri bağımlılığı (Data Dependency: Read-After-Write, Write-After-Read, Write-After-Write) olan komutların sırasını ASLA bozmaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Three Tiers of Reordering: 1) Compiler Reordering: HotSpot C2 JIT reorders instructions to maximize register utilization and eliminate redundant loads. 2) Processor Reordering: Superscalar architectures employ reservation stations and out-of-order execution pipelines to avoid stall cycles during cache misses. 3) Memory Subsystem Reordering: Store Buffers allow the CPU to continue processing while memory writes are lazily drained, allowing subsequent loads to bypass stale stores. 4) The As-If-Serial Invariant: Compilers and silicon pledge that single-threaded execution behavior is indistinguishable from strict serial execution. However, they make ZERO guarantees to concurrent observer threads without explicit memory fences.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Instruction Reordering Kanıtı (jcstress Mantığı)
public class ReorderingTest {
    private int x = 0, y = 0;
    private int r1 = 0, r2 = 0;

    // Thread 1 çalıştırır:
    public void actor1() {
        x = 1;   // Yazma 1
        r1 = y;  // Okuma 1
    }

    // Thread 2 çalıştırır:
    public void actor2() {
        y = 1;   // Yazma 2
        r2 = x;  // Okuma 2
    }

    // Olası Sonuçlar:
    // r1=0, r2=1 (Thread 1 önce bitti)
    // r1=1, r2=0 (Thread 2 önce bitti)
    // r1=1, r2=1 (İki yazma da önce bitti)
    // r1=0, r2=0 (İMKANSIZ GİBİ GÖRÜNÜR AMA REORDERING YÜZÜNDEN GERÇEKLEŞİR!)
    void main() {
        IO.println("Instruction Reordering: Donanım StoreLoad taklası r1=0, r2=0 sonucunu
doğurabilir!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Sorusu: 'Tek bir thread içinde `x = 1; y = 2;` yazarsak CPU hangisini önce çalıştırır?' Doğru Yanıt: 'Aralarında veri bağımlılığı olmadığı için CPU veya JIT dilediğini önce çalıştırabilir! Tek thread için sonuç fark etmez ama başka bir thread araya girerse patlar.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Sharp Interview Question: 'In `x = 1; y = 2;`, does the JVM guarantee x is written before y?' Correct Answer: 'No! There is zero data dependency between x and y. The compiler or CPU can flip their execution order freely. To enforce ordering across threads, memory barriers (`volatile` or locks) are mandatory.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- As-if-serial kuralı sıralama garantisini yalnızca tek bir thread için verir.
- Modern CPU'lar bellek gecikmesini gizlemek için komutları sıra dışı (out-of-order) çalıştırır.
- Farklı thread'ler arasındaki sıralama yalnızca Memory Barrier (Fences) ile zorlanabilir.

🎯 Key Takeaways for the Interview

- The as-if-serial contract guarantees preservation of program semantics strictly for single threads.
- Hardware speculatively reorders instructions out-of-order to hide memory latency.
- Cross-thread instruction ordering requires explicit hardware memory barriers.

6

BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Thread Oluşturma Paradigmaları: Thread, Runnable, Callable & Java 25 Builder

Thread Creation Paradigms: Thread, Runnable, Callable & Modern Java 25 Builders

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic




```
1. ESKİ USUL (Kalıtım / Kötü):
   class Worker extends Thread { public void run() { ... } }
   ❌ Java tekli kalıtımını harcar, Thread yaşam döngüsü ile iş mantığı birbirine karışır!

2. AYRIŞTIRILMIŞ İŞ PARÇACIĞI (Composition / Standart):
   Runnable task = () -> { ... }; // Geriye değer dönemez, Exception fırlatamaz!
   Callable<String> task = () -> "Sonuç"; // Değer döner, Checked Exception fırlatabilir!

3. MODERN JAVA 25 BUILDER PATTERN (JDK 21+ & 25):
   Thread.ofPlatform().name("worker-", 1).start(task);
   Thread.ofVirtual().name("loom-", 1).start(task);
   ✓ Tek bir arayüzle hem ağır OS thread'i hem de hafif Virtual Thread açabilme!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eskiden bir kamyon almak istediğinizde şoförü de kamyonun motoruna kaynaklanmış olarak alırdınız (extends Thread). Artık kamyon başka (Thread), şoförün yapacağı iş emri başkadır (Runnable/Callable)! Runnable, 'Git depoyu süpür' diyen ama sana cevap vermeyen bir emirdir. Callable ise 'Kasada kaç para var say ve bana söyle' diyen, sana bir sonuç (Future) getiren emirdir. Java 25'te ise artık 'Bana 1 numaralı platform işçisini ver' veya 'Bana uçan sanal işçi ver' diyebildiğimiz harika bir sipariş menümüz (Thread.ofPlatform() / Thread.ofVirtual()) vardır!

🇬🇧 ENGLISH (ELI5)

In Java 1.0, to buy a delivery truck, you had to weld the human driver directly to the chassis (`extends Thread`). Modern engineering separates the vehicle (Thread) from the cargo assignment (Runnable/Callable). `Runnable` is a 'Fire-and-forget' chore returning `void`. `Callable\<V>` is an order returning an explicit computed value and capable of throwing checked exceptions. In modern Java 25, the fluent `Thread.ofPlatform()` and `Thread.ofVirtual()` builders allow seamlessly stamping out heavyweight OS threads or lightweight user-mode fibers!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 25 Thread Builder Mimarisi: 1) `Thread` sınıfından türetmenin zararları: Object Oriented prensiplerine aykırıdır; 'İş' (Task) ile o işi 'Çalıştıran Motor' (Worker) birbirine sıkı sıkıya bağlanır. 2) `Runnable` vs `Callable`: Runnable `run()` metodu `void` döner ve checked exception fırlatamaz; Callable `call()` ise generik `V` döner ve `throws Exception` imzasına sahiptir. 3) Modern `Thread.Builder`: Java 21 ile gelen ve Java 25'te standartlaşan `Thread.ofPlatform()` ve `Thread.ofVirtual()`, thread konfigürasyonunu (isim öneki, daemon durumu, priority, UncaughtExceptionHandler) fluent API ile güvenle kurar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architectural Analysis of Thread Factories: 1) The Fallacy of Subclassing: Inheritance couples the task logic with OS thread scheduling state, violating the Single Responsibility Principle (SRP). 2) `Runnable` vs `Callable\<V>` Contract: `Runnable.run()` is a void-returning, non-throwing functional interface. `Callable\<V>.call()` enables return value propagation and checked exception handling, foundational to Executor frameworks. 3) Java 25 Fluent Builders: `Thread.ofPlatform()` and `Thread.ofVirtual()` provide type-safe builders for thread naming, daemon flags, uncaught exception handlers, and thread factory generation (`ThreadFactory`).

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Modern Thread Builders ve Callable Entegrasyonu
import java.util.concurrent.Callable;
import java.util.concurrent.FutureTask;

void main() throws Exception {
    // 1. Modern Java 25 Platform Thread Builder
    Thread platformThread = Thread.ofPlatform()
        .name("payment-worker-", 1)
        .daemon(false)
        .start(() -> IO.println("Platform thread çalıştı: " + Thread.currentThread()));
    platformThread.join();

    // 2. Callable ile değer döndüren iş parçacığı
    Callable<Integer> calculateTaxTask = () -> {
        IO.println("Vergi hesaplanıyor...");
        return 1850;
    };

    var futureTask = new FutureTask<>(calculateTaxTask);
    Thread.ofPlatform().name("tax-calc").start(futureTask);

    // get() çağrısı hesaplama bitene kadar bekler (Blocking)
    int result = futureTask.get();
    IO.println("Hesaplanan Vergi: " + result + " TL");
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Thread.run() çağırmak ile Thread.start() çağırmak arasındaki fark nedir?' Yanıt: 'start() metodu JVM üzerinden işletim sistemine yeni bir thread tahsis ettirir (OS thread) ve o yeni thread üzerinde run() metodunu tetikler. Doğrudan run() çağrılırsa yeni thread açılmaz, kod o anki mevcut thread içinde senkron çalışır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Phone Screen Trap: 'What happens if you invoke `thread.run()` instead of `thread.start()`?' Answer: '`start()` prompts the JVM to allocate an underlying OS kernel thread and invoke `run()` asynchronously. Calling `run()` directly merely executes the method synchronously on the current caller's thread, bypassing multithreading entirely.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Thread sınıfını extend etmek yerine Runnable/Callable kompozisyonu kullanılmalıdır.
- Callable geriye değer dönebilir ve Exception fırlatabilir; Runnable dönemez.
- Java 25'te Thread.ofPlatform() ve Thread.ofVirtual() standart builder'dır.

🎯 Key Takeaways for the Interview

- Favor composition (Runnable/Callable) over inheriting from Thread.
- Callable\<V> yields return values and throws checked exceptions; Runnable returns void.
- Java 25 fluent builders Thread.ofPlatform() / ofVirtual() unify thread instantiations.

BÖLÜM 2 / MODULE 2

Volatile Anahtar Kelimesi, Atomiklik & Görünürlük

The volatile Keyword, Atomicity & Visibility

Görünürlük (Visibility) ve Atomiklik farkı, `volatile` bellek bariyerleri (Memory Barriers), Double-Checked Locking (DCL) tuzağı, CAS döngüleri ve `LongAdder` vs `AtomicLong` ölçeklenme mimarisi.

Visibility vs Atomicity, hardware memory barriers backing `volatile`, the Double-Checked Locking (DCL) singleton hazard, lock-free CAS loops, and `LongAdder` vs `AtomicLong` scalability.

7

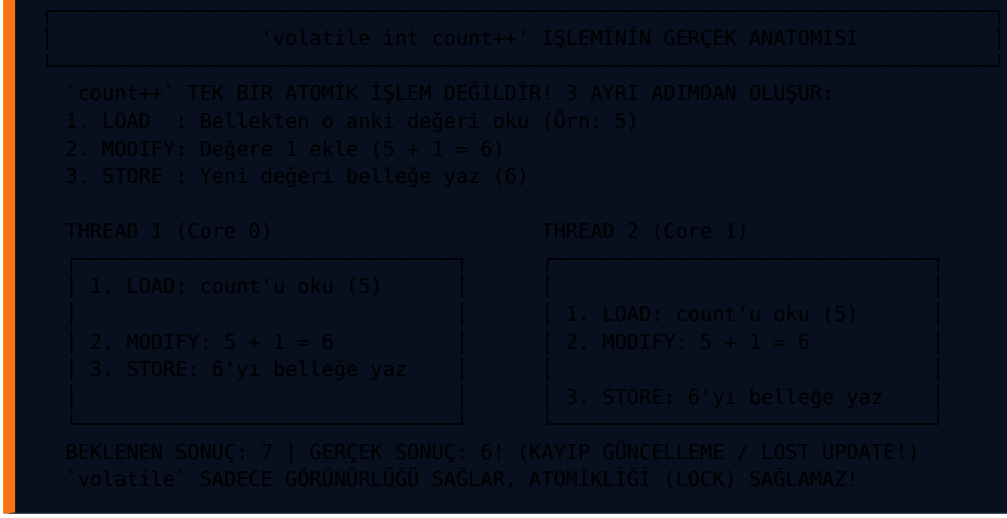
BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Görünürlük (Visibility) vs Atomiklik (Atomicity): 'volatile int counter++' Neden Çöker?

Visibility vs Atomicity: Why 'volatile int counter++' is a Silent Catastrophe

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki tahtaya tebeşirle sayı yazıldığını düşünün. 'volatile' demek, tahtanın üzerine süper parlak bir spot ışığı yakmak gibidir; tahtada ne yazıyorsa bütün oda anında görür (Görünürlük)! Ama iki kişi aynı anda tahtaya koşup 'Mevcut sayıyı silip 1 fazlasını yazayım' derse ne olur? İkisi de aynı anda '5' sayısını görür, ikisi de kafasında 6 hesaplar, ikisi de tahtaya 6 yazar! 2 kere artması gereken sayı sadece 1 kere artar! Spot ışığı (volatile) insanların birbirinin eline çarpmasını (Atomiklik) engellemez!

🇬🇧 ENGLISH (ELI5)

Imagine a shared blackboard in an office. Declaring a variable `volatile` is like installing a 10,000-lumen spotlight directly onto the board: the instant a number is inscribed, every engineer across the room sees it immediately (Visibility). But what happens when two engineers run to the board at the same second to increment the number? Both observe '5', both mentally calculate '5 + 1 = 6', and both scribble '6' over the board! The count increments once instead of twice! Spotlights guarantee sight, not turn-taking (Atomicity)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Read-Modify-Write Döngüsü ve volatile Sınırları: 1) Görünürlük Garantisi: `volatile` bir değişkene yazıldığında, CPU yerel Store Buffer'ı ana belleğe boşaltır (Flush). Okunduğunda ise CPU yerel cache'i geçersiz kılar (Invalidate) ve RAM/L3'ten taze veri çeker. 2) Atomiklik İhlali: Java bytecode seviyesinde `count++` işlemi: `GETFIELD`, `ICONST_1`, `IADD`, `PUTFIELD` olmak üzere 4 talimata ayrılır. İki thread bu talimatları iç içe (interleaved) çalıştırdığında 'Lost Update' meydana gelir. 3) Kural: `volatile` yalnızca ve yalnızca yazılan yeni değer eski değerden tamamen bağımsızsa (örneğin bir boolean bayrak: `running = false`) güvenlidir. Eğer yeni değer eski değere bağlıysa (`count = count + 1`), `volatile` yetersizdir; `AtomicInteger` veya `synchronized` zorunludur.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Dissecting the Read-Modify-Write Hazard: 1) Hardware Visibility Semantics: A `volatile` write issues a memory barrier draining the local store buffer to the cache coherency fabric. A `volatile` read bypasses stale L1 entries. 2) Bytecode Non-Atomicity: The expression `count++` compiles into non-atomic instructions: `getfield`, `iconst_1`, `iadd`, `putfield`. Context switches occurring between `getfield` and `putfield` produce severe data races (Lost Updates). 3) The Axiom of Volatile: `volatile` is thread-safe IF AND ONLY IF state mutations do not depend on current state (e.g. status flags: `volatile boolean shutdownRequested`). When reads dictate writes, CAS primitives (`AtomicInteger`) or mutual exclusion are non-negotiable.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Volatile Atomiklik İllüzyonu ve Kayıp Güncelleme Kanıtı
public class VolatileBrokenCounter {
    private volatile int count = 0;

    public void increment() {
        count++; // TEHLİKELİ! Atomik değildir!
    }

    void main() throws InterruptedException {
        var demo = new VolatileBrokenCounter();
        int threadCount = 10;
        int incrementsPerThread = 1000;

        Thread[] threads = new Thread[threadCount];
        for (int i = 0; i < threadCount; i++) {
            threads[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < incrementsPerThread; j++) {
                    demo.increment();
                }
            });
        }

        for (Thread t : threads) t.join();

        IO.println("Beklenen Sayaç: " + (threadCount * incrementsPerThread)); // 10,000
        IO.println("Gerçek Sayaç  : " + demo.count); // Çoğunlukla 9,200 civarı çıkar!
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Volatile anahtar kelimesi senkronizasyon için ne zaman yeterlidir?' Yanıt: 'Sadece tek bir thread yazıp diğer tüm thread'ler okurken (Single-Writer Multiple-Reader) veya değişken sadece durum bildiren bir bayraksa (`volatile boolean stopped`). Birden fazla thread yazıyorsa volatile tek başına ASLA yetmez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Top Interview Screen: 'When is volatile sufficient for thread safety?' Senior Answer: 'Exclusively in Single-Writer / Multiple-Reader topologies, or for state flag publication (`volatile boolean cancelled`). If multiple concurrent threads execute read-modify-write operations, volatile guarantees zero safety without locks or CAS operations.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- volatile sadece görünürlüğü (visibility) sağlar; atomiklik (atomicity) sağlamaz.
- count++ arka planda 3 ayrı işlem olduğu için lost update hatasına yol açar.
- Değişkene birden fazla thread yazıyorsa Atomic sınıfları veya kilitler zorunludur.

🎯 Key Takeaways for the Interview

- volatile guarantees memory visibility; it provides zero transactional atomicity.
- count++ encompasses 3 discrete CPU instructions, suffering lost updates under concurrency.
- When multiple threads mutate shared state, Atomic types or mutexes are mandatory.

8

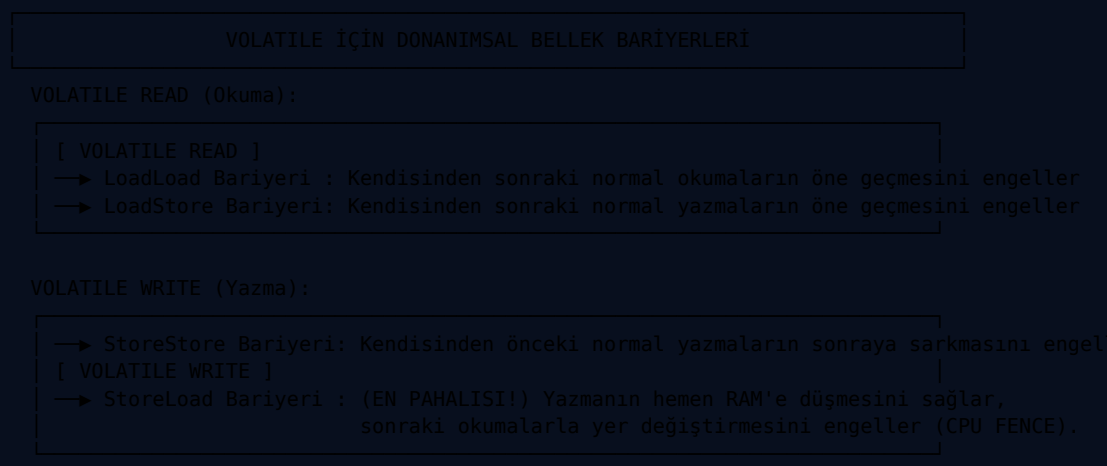
BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

volatile Mekaniği: Donanımsal Bellek Bariyerleri (Memory Barriers / Fences)

The Mechanics of volatile: Hardware Memory Barriers (StoreLoad, StoreStore, LoadLoad)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir postacı düşünün: Mektupları kafasına göre sıraya dizip dağıtıyor (CPU Reordering). Ama patron kapıya bir polis bariyeri (Memory Barrier) dikiyor: 'Bu kırmızı mühürlü mektubu (volatile write) çantandan çıkarmadan önce, elindeki bütün diğer zarfları teslim etmiş olmak zorundasın! Ve bu kırmızı mektubu vermeden sıradaki hiçbir mektubu açamazsın!' İşte Memory Barrier, CPU'nun hızlanmak için yaptığı bütün sıralama hilelerini anında donduran sert bir polis barikatıdır!

🇬🇧 ENGLISH (ELI5)

Think of a courier optimizing deliveries by dropping packages out of order. A Memory Barrier is an armed checkpoint officer standing in the roadway: 'Before you hand over this top-priority VIP document (volatile write), every single regular letter previously in your satchel MUST already be delivered! And you cannot open any future parcels until this VIP document clears!' A Memory Barrier is hardware silicon instructions freezing speculative reordering.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Donanım Seviyesinde 4 Bellek Bariyeri: 1) LoadLoad: `Load1; LoadLoad; Load2` -> Load1 verisi çekilmeden Load2 verisi çekilemez. 2) StoreStore: `Store1; StoreStore; Store2` -> Store1 verisi diğer CPU'lara görünür olmadan Store2 yazılamaz. 3) LoadStore: `Load1; LoadStore; Store2` -> Load1 bitmeden Store2 belleğe basılamaz. 4) StoreLoad (En Ağır Bariyer): `Store1; StoreLoad; Load2` -> Store1'in Store Buffer'dan tamamen temizlenip tüm CPU'lara görünür olmasını garanti eder; aksi takdirde sonraki Load2 bayat veri okuyabilir. x86 mimarisinde donanım zaten güçlü bir bellek modeline (Total Store Order - TSO) sahip olduğu için LoadLoad, LoadStore ve StoreStore bedavadır (NOOP); ancak StoreLoad için pahalı bir `lock addl` veya `mfence` talimatı üretilir. ARM64 ve Apple Silicon zayıf bellek modeline (Weak Memory Model) sahip olduğu için açık `dmb ish` bariyerleri koyar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Hardware Barrier Topologies: 1) LoadLoad & LoadStore: Inhibit subsequent loads/stores from bypassing an initial load. 2) StoreStore: Guarantees preceding stores are flushed to cache hierarchy before subsequent stores become visible. 3) StoreLoad (The Heavyweight Fence): Forces full pipeline serialization. The CPU store buffer must fully drain to L1 before any subsequent load can execute. 4) Architecture Specifics (x86 vs ARM): x86 enforces Total Store Order (TSO) in hardware, making LoadLoad, LoadStore, and StoreStore zero-cost NOOPs; `volatile write` emits a single `lock addl ... 0` instruction to achieve StoreLoad. ARM64/Apple Silicon features a relaxed memory model, emitting explicit `dmb ishld` and `dmb ish` barrier instructions.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Memory Barrier Mantığını Simüle Eden Durum Yayınlama
public class MemoryBarrierPublisher {
    private int configValue = 0;
    private String serverHost = null;
    private volatile boolean initialized = false;

    // Yazan Thread:
    public void initSystem() {
        configValue = 8080;                  // 1. Normal yazma
        serverHost = "api.maang.internal";   // 2. Normal yazma
        // StoreStore Bariyeri buraya yerleşir!
        initialized = true;                  // 3. Volatile yazma (StoreLoad bariyeri tetikler)
    }

    // Okuyan Thread:
    public void handleRequest() {
        if (initialized) { // Volatile okuma (LoadLoad & LoadStore bariyeri)
            // JMM Garantisi: configValue=8080 ve serverHost KESİNLİKLE görünürdür!
            IO.println("Sistem hazır: " + serverHost + ":" + configValue);
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Performans Uyarısı: Mülakatta 'volatile değişken okumak mı yoksa yazmak mı daha pahalıdır?' diye sorulursa; 'Volatile yazma işlemi StoreLoad bariyeri (x86'da mfence / lock prefix) tetiklediği ve CPU Store Buffer'ı boşalmaya zorladığı için yazma işlemi okumadan çok daha pahalıdır!' cevabı verilmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Performance Trap: 'Is a volatile read or a volatile write more expensive?' Senior Answer: 'A volatile write is significantly more expensive on modern CPUs. It forces a StoreLoad memory barrier (e.g. `lock addl` on x86), which stalls execution until the CPU store buffer drains completely to the cache coherency bus.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- volatile anahtar kelimesi CPU seviyesinde Memory Barrier talimatları üretir.
- StoreLoad bariyeri en pahalı bariyerdir; Store Buffer'ı tamamen boşaltır.
- x86 mimarisinde TSO nedeniyle sadece StoreLoad maliyetlidir; ARM64'te açık bariyerler kullanılır.

🎯 Key Takeaways for the Interview

- The volatile keyword compiles into explicit hardware memory barrier instructions.
- The StoreLoad fence is the most expensive barrier, serializing CPU store buffers.
- x86 TSO absorbs most barriers as NOOPs; ARM64 requires explicit fence instructions.

9

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Piggybacking on volatile: Güvenli Durum Yayınlama (Safe Publication)

Piggybacking on volatile: Safe State Publication & Ordering Guarantees

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              PIGGYBACKING (SIRTA BİNME) HAPPENS-BEFORE KURALI │
└──────────────────────────────────────────────────────────────┘

NON-VOLATILE DEĞİŞKENLER:            VOLATILE DEĞİŞKEN (KORUYUCU MELEK):
int a = 10;
String name = "MAANG";
List<String> items = new ArrayList();
items.add("Data");
                                     volatile int trigger = 1; // ◄── YAZMA!
──────────────────────────────────────────────────────────────────
BÜTÜN BU NORMAL DEĞİŞKENLER, TRIGGER'IN ARKASINA TAKILARAK RAM'E BASILIR!

OKUYUCU THREAD:
if (trigger == 1) { // ◄── VOLATILE OKUMA!
    // a, name ve items listesi KESİNLİKLE tam ve güncel olarak görünür!
}
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Havaalanında uçağa binerken pasaportu olmayan 3 çocuk düşünün (Normal değişkenler). Kapıdaki polis tek tek çocukları içeri almaz. Ama yanlarında diplomatik pasaporta sahip bir büyükelçi (volatile değişken) vardır. Büyükelçi kapıdan geçerken 'Bu çocuklar benimle' dediği anda, bütün çocuklar büyükelçinin sırtına binerek (Piggybacking) içeriye güvenle sızarlar! Tek bir volatile değişken, kendisinden önce yazılan onlarca normal değişkeni de beraberinde görünür kılar!

🇬🇧 ENGLISH (ELI5)

Imagine three passengers lacking boarding passes (regular non-volatile fields). Airport security refuses them entry. However, they walk alongside an accredited diplomat holding an all-access pass (a single `volatile` variable). When the diplomat clears the gate, the Happens-Before protocol dictates that everyone preceding the diplomat in line is admitted simultaneously (Piggybacking). A single volatile write flushes all preceding non-volatile state mutations into global visibility!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Piggybacking Mekanizması ve JMM Transitivity Kuralı: 1) Happens-Before Transitivity: Eğer `A -> B` (Program Order) ve `B -> C` (Volatile kuralı) ise, o zaman `A -> C`'dir. Burada A normal değişken yazmaları, B volatile yazma, C ise başka bir thread'in o volatile değişkeni okumasıdır. 2) Java Standart Kütüphanesindeki Yeri: `java.util.concurrent` paketindeki birçok sınıf (ör. `AbstractQueuedSynchronizer` - AQS ve `ConcurrentHashMap`) bu hileyi yoğun şekilde kullanır. Tüm alanları `volatile` yapmak yerine, tek bir `volatile int state` değişkenine yazarak sınıftaki diğer tüm düz alanları görünür kılarlar. Bu sayede gereksiz bellek bariyeri maliyetlerinden kaçınarak inanılmaz bir throughput elde edilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Piggybacking Formalism & JDK Engineering: 1) Transitively Inherited Visibility: JSR-133 guarantees that if Action A (non-volatile writes) happens-before Action B (volatile write), and Action B happens-before Action C (volatile read by Thread 2), then Action A happens-before Action C. 2) High-Performance JDK Exploitation: `AbstractQueuedSynchronizer` (AQS) powers ReentrantLock, Semaphore, and CountDownLatch. AQS does not mark internal queue node pointers as volatile; instead, it relies on piggybacking off mutations to `volatile int state` to safely publish entire linked-node updates, evading catastrophic memory fence penalties.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Piggybacking ile Düz Değişkenleri Güvenle Yayınlama
import java.util.HashMap;
import java.util.Map;

public class PiggybackingPublisher {
    // Bu map normal bir HashMap'tir (Thread-safe DEĞİLDİR!)
    private Map<String, String> internalConfig = new HashMap<>();

    // Sihirli koruyucu: Tek bir volatile tetikleyici
    private volatile boolean published = false;

    public void prepareAndPublish() {
        // 1. Normal, senkronize olmayan map manipülasyonu
        internalConfig.put("db.user", "admin");
        internalConfig.put("db.password", "secret");
        internalConfig.put("cache.ttl", "3600");

        // 2. Piggybacking tetikleyici: Volatile yazma!
        // Kendisinden önceki BÜTÜN map verilerini bellek bariyeriyle RAM'e döker.
        published = true;
    }

    public void readConfig() {
        if (published) { // Volatile okuma
            // internalConfig KESİNLİKLE eksiksiz ve güvenle okunur!
            IO.println("Kullanıcı: " + internalConfig.get("db.user"));
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kritik Kodlama Tuzağı: Piggybacking yaparken kod sıralamasına çok dikkat edilmelidir! Eğer `published = true;` satırını map'e veri eklemeden ÖNCE yazarsanız, reordering yüzünden okuyucu thread boş veya yarım doldurulmuş map ile karşılaşır!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Dangerous Trap: The order of statements is lethal in piggybacking! If you assign `published = true` BEFORE mutating `internalConfig`, the happens-before chain is severed. Observers will read partially constructed, corrupted data structures.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Piggybacking, normal değişkenleri volatile bir yazmanın arkasına takarak görünür kılmaktır.
- JMM'nin Geçişlilik (Transitivity) kuralı sayesinde A -> B ve B -> C ise A -> C olur.
- JDK'nın AQS altyapısı yüksek performans için piggybacking prensibini kullanır.

🎯 Key Takeaways for the Interview

- Piggybacking piggybacks non-volatile updates onto the back of a volatile write.
- Happens-before transitivity legally guarantees that preceding non-volatile state is visible.
- JDK concurrency engines (AQS) leverage piggybacking to minimize memory fence overhead.

10

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Double-Checked Locking (DCL) & Singleton Tuzağı: volatile Neden Zorunlu?

Double-Checked Locking (DCL) & The Singleton Trap: Why volatile is Non-Negotiable

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              DOUBLE-CHECKED LOCKING (DCL) NESNE OLUŞTURMA ADIMLARI              │
└──────────────────────────────────────────────────────────┘

`instance = new Singleton();` İŞLEMİ TEK BİR ADIM DEĞİLDİR!

GERÇEK KOD ADIMLARI:
1. memory = allocate();   // Bellekte yer tahsis et
2. ctorSingleton(memory); // Constructor'ı çalıştır (Alanları doldur)
3. instance = memory;     // Referansı instance değişkenine bağla

CPU / JIT YENİDEN SIRALAMASI (REORDERING):
1. memory = allocate();   // Bellek tahsis edildi
3. instance = memory;     // ⚠ DİKKAT! Referans erkenden bağlandı! (İçi henüz BOŞ!)
2. ctorSingleton(memory); // Constructor hala çalışıyor...

THREAD 2 TAM BU ANDA GELİR:
if (instance == null) ──► FALSE DÖNER! (Çünkü 3. adım çalıştı!)
return instance;      ──► 💣 GÜM! Thread 2 henüz İÇİ BOŞ nesneyi kullanmaya başlar!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir mobilyacıdan gardırop siparişi verdiniz. Usta dükkanda parçaları birleştirip öyle teslim etmelidir. Ama usta hızlanmak için boş gardırop kolisini (Bellek tahsisi) evinize getirip kapıya koyar (Referansı bağlar), raflarını ve kapaklarını ise sonradan takmaya başlar. Tam o sırada evin çocuğu gelip kapağı olmayan, vidaları takılmamış gardırobun içine kıyafet koymaya çalışırsa gardırop çocuğun üstüne devrilir! İşte 'volatile' koymazsanız, Java nesnenin bitmemiş kolisini diğer thread'lere dolu gibi gösterir!

🇬🇧 ENGLISH (ELI5)

You order an assembled oak wardrobe. The carpenter should assemble the doors and drawers before handing over the keys. Instead, the rushed carpenter sets a hollow wooden frame in your bedroom (instance pointer published) while rummaging for screws and shelves in the hallway (constructor executing). If your child immediately tosses coats inside, the unfinished frame collapses! Without `volatile`, the JVM publishes the object reference BEFORE constructor completion, causing threads to access half-baked objects!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

DCL Probleminin Çözümü ve `volatile`: 1) Neden synchronized yetersiz?: Metodun tamamını `synchronized` yapmak her okumada kilit maliyeti yaratır. Bu yüzden 'önce kilitsiz kontrol et, null ise kilide gir, içeride bir daha kontrol et' (Double-Check) tasarlandı. 2) Yarı Yapılandırılmış Nesne Kaçağı (Partially Constructed Object): JIT derleyicisi 2. ve 3. adımların yerini değiştirebilir (Instruction reordering). Thread A henüz constructor'ı bitirmeden `instance` referansı non-null hale gelir. O anda kilit dışındaki Thread B ilk kontrolde `instance != null` görür ve nesneyi alır; ancak nesnenin alanları henüz varsayılan `0` veya `null` değerindedir! 3) volatile Çözümü: `private static volatile Singleton instance;` tanımlandığında, StoreStore ve StoreLoad bariyerleri sayesinde nesnenin constructor'ı tamamen bitmeden referans ASLA yayınlanamaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Deconstructing Double-Checked Locking: 1) The Performance Impetus: Synchronizing the entire `getInstance()` method incurs mutex acquisition on every single read. DCL seeks to synchronize strictly on the initial null check. 2) Reordering Vulnerability: The JVM allows reordering between step 2 (instance initialization) and step 3 (pointer assignment). Thread A executes step 1 then step 3; Thread B executes the outer unsynchronized check, observes `instance != null`, and happily returns the instance while Thread A is still executing step 2! Thread B observes a partially constructed object, crashing with NPEs or corrupt data. 3) The Volatile Fix: Declaring `volatile Singleton instance` forces a StoreStore barrier prior to reference assignment, ensuring all constructor writes flush before publication.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Güvenli Double-Checked Locking (DCL) Singleton
public class SafeDclSingleton {
    // ⚠ KRİTİK: volatile OLMADAN DCL %100 HATALIDIR!
    private static volatile SafeDclSingleton instance;
    private final String connectionString;

    private SafeDclSingleton() {
        // Ağır başlatma simülasyonu
        this.connectionString = "jdbc:postgresql://db.maang.internal:5432/core";
    }

    public static SafeDclSingleton getInstance() {
        // 1. Kontrol (Kilitsiz - Hızlı Okuma)
        SafeDclSingleton localRef = instance;
        if (localRef == null) {
            synchronized (SafeDclSingleton.class) {
                // 2. Kontrol (Kilit İçinde - Kesin Tekillik)
                localRef = instance;
                if (localRef == null) {
                    instance = localRef = new SafeDclSingleton();
                }
            }
        }
        return localRef;
    }

    public String getConnectionString() {
        return connectionString;
    }

    void main() {
        var s = SafeDclSingleton.getInstance();
        IO.println("Bağlantı Başarılı: " + s.getConnectionString());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı Tuzağı: Mülakatçı size DCL kodunu tahtaya yazar ve 'Burada volatile'ı unutsak ne olur?' der. 'NullPointerException veya beklenmedik değerler alırız çünkü başka bir thread henüz constructor'ı tamamlanmamış yarı-yapılandırılmış (partially constructed) bir nesneye erişir' cevabını vermeyen aday elenir!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

The 1 MAANG Concurrency Question: 'What breaks if we omit volatile in Double-Checked Locking?' Answer: 'Threads will observe a partially constructed object! The compiler or CPU can reorder memory assignment before constructor invocation, exposing uninitialized fields to unsynchronized reader threads.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- DCL deseninde volatile kullanımı bir seçenek değil, matematiksel bir zorunluluktur.
- volatile olmadan başka bir thread yarı-yapılandırılmış (partially constructed) nesne görür.
- Modern Java'da alternatif olarak 'Initialization-on-demand holder' idiom tercih edilebilir.

🎯 Key Takeaways for the Interview

- volatile in Double-Checked Locking is a strict requirement, not an optional optimization.
- Without volatile, reader threads risk observing uninitialized, partially constructed state.
- Alternatively, use the Bill Pugh Initialization-on-demand holder idiom.

11

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Atomik İlkel Tipler: AtomicInteger, CAS (Compare-And-Swap) & Kilit-Siz Programlama

Atomic Primitives: AtomicInteger, CAS (Compare-And-Swap) & Lock-Free Computing

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              COMPARE-AND-SWAP (CAS) DONANIMSAL DÖNGÜSÜ   │
└──────────────────────────────────────────────────────────┘

 Thread 1 ve Thread 2 aynı anda count'u (5) artırmak istiyor:

 1. Thread 1 okur: Beklenen Değer = 5, Yeni Değer = 6
 2. Thread 2 okur: Beklenen Değer = 5, Yeni Değer = 6

 CPU DONANIM SEVİYESİNDE ATOMİK TALİMAT: `CMPXCHG` (Compare and Exchange)

 Thread 1 CPU'ya sorar: "Bellekteki değer hala 5 mi?"
 CPU: "Evet, hala 5. Değeri 6 yapıyorum!" ──► [ BAŞARILI (CAS SUCCESS) ]

 Thread 2 CPU'ya sorar: "Bellekteki değer hala 5 mi?"
 CPU: "HAYIR! Değer artık 6 oldu!"        ──► [ BAŞARISIZ (CAS FAIL) ]

 Thread 2 pes etmez (Spin / Retry Loop):
 Yeniden okur: Beklenen = 6, Yeni = 7 ──► Tekrar dener ──► BAŞARILI!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir banko düşünün. Kilitli sistemde (synchronized) veznedar içeri tek bir müşteri alır, kapıyı kilitler, işi bitene kadar dışarıdaki 100 kişi yağmurda bekler. Kilitsiz sistemde (CAS / Atomic) ise kapı her zaman açıktır! Herkes vezneye koşar: 'Hesabımda 100 TL olduğunu biliyorum, onu 150 yap!' der. Veznedar bakar: 'Evet gerçekten 100 TL var, 150 yaptım (Başarılı).' İkinci kişi geldiğinde 'Hesabımda 100 TL vardı 120 yap' derse veznedar: 'Geç kaldın, para artık 150 oldu! Git sıranı tekrar hesapla gel!' der. Müşteri dışarıda beklemez, hemen güncel paraya göre tekrar dener!

🇬🇧 ENGLISH (ELI5)

Consider an amusement park turnstile. Pessimistic locking (synchronized) puts an armed guard at the gate: only one person enters, while 500 people wait frozen in the cold. Optimistic lock-free computing (CAS) removes the guard entirely! Everyone pushes forward: 'I observed the ticket counter was at 100; advance it to 101!' The hardware checks: 'Is it still 100? Yes! Flipping to 101 (Success).' If another person arrives claiming it was 100, hardware rejects: 'False! It is already 101! Recalculate and retry.' Nobody sleeps; losers spin and retry instantly!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CAS Mekanizması ve Donanım Desteği: 1) `CMPXCHG` Talimatı: x86 mimarisinde `lock cmpxchg` tek bir CPU saat çevriminde atomik olarak belleği kontrol eder ve eşitse yazar. Bu işlem işletim sistemi seviyesinde thread askıya alma (park/unpark) gerektirmez; kullanıcı modunda (user space) ultra hızlı gerçekleşir. 2) İyimser Kilitlenme (Optimistic Concurrency): Çatışmanın düşük olduğu sistemlerde CAS, geleneksel mutex kilitlerden 10 kat daha hızlıdır. 3) Spin-Wait Maliyeti: Yüksek çekişmede (high contention), onlarca thread aynı anda CAS denerse CPU döngüleri boşa harcanır (CPU spike) ve throughput düşer.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Hardware CAS Mechanics: 1) `cmpxchg` Assembly Primitive: AtomicInteger relies on `Unsafe` / `VarHandle.compareAndSet()`, executing the hardware-level `lock cmpxchg` instruction. It evaluates `if (memory == expected) { memory = update; return true; } else return false;` in a single un-interruptible silicon cycle. 2) Lock-Free Progress Guarantee: At least one thread is mathematically guaranteed to make forward progress on every CAS collision round, preventing priority inversions and deadlock. 3) Contention Penalty: Under extreme multi-threaded writes, CAS failure rates skyrocket, generating spin-wait CPU burn as threads continuously thrash on cache-line invalidations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: AtomicInteger ile Thread-Safe Lock-Free Sayaç
import java.util.concurrent.atomic.AtomicInteger;

public class LockFreeCounter {
    private final AtomicInteger count = new AtomicInteger(0);

    public void increment() {
        // Dahili olarak compareAndSet döngüsü (CAS loop) işletir
        count.incrementAndGet();
    }

    public int get() {
        return count.get();
    }

    // Manuel CAS Döngüsü Örneği (Koşullu Güncelleme)
    public void multiplyIfLessThan(int multiplier, int limit) {
        int current;
        int next;
        do {
            current = count.get();
            if (current >= limit) return;
            next = current * multiplier;
        } while (!count.compareAndSet(current, next)); // Eşitse güncelle, değilse tekrar dene!
    }

    void main() {
        var counter = new LockFreeCounter();
        counter.increment();
        counter.multiplyIfLessThan(5, 10);
        IO.println("Lock-Free Son Değer: " + counter.get());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'AtomicInteger kullanırsak thread'ler hiçbir zaman bloke olmaz mı?' Yanıt: 'Evet, işletim sistemi seviyesinde BLOCKED olmazlar; ANCAK yüksek çekişmede (High Contention) CAS sürekli başarısız olup döngüye (spin) gireceği için CPU kullanımı %100'e fırlayabilir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Trap: 'Does AtomicInteger eliminate all thread blocking overhead?' Answer: 'Threads never enter the OS BLOCKED state; however, under extreme contention, repeated CAS failures cause aggressive spinning, spiking CPU utilization to 100% while starving other tasks.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CAS (Compare-And-Swap), CPU donanım seviyesinde atomik kontrol ve yazma yapar.
- Kilit kullanılmadığı için Deadlock riski sıfırdır.
- Aşırı çekişmede CAS döngüleri CPU tüketimini tavan yaptırabilir.

🎯 Key Takeaways for the Interview

- CAS (Compare-And-Swap) executes atomic evaluate-and-set in hardware silicon via CMPXCHG.
- Lock-free algorithms carry zero risk of mutual deadlocks.
- Severe contention causes CAS spin-loops to burn excessive CPU cycles.

12

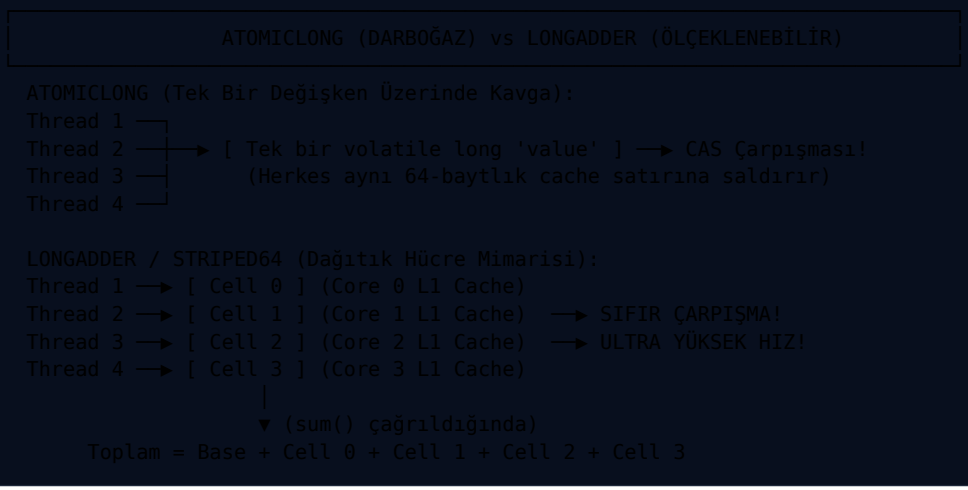
BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Yüksek Ölçekli Sayaçlar: LongAdder vs AtomicLong (Striped64 & Dağıtık Hücreler)

High-Throughput Scalability: LongAdder vs AtomicLong (Striped64 & Cell Arrays)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir süpermarkette tek bir kasa olduğunu düşünün (AtomicLong). 50 müşteri aynı anda tek bir kasiyere para ödemeye çalışırsa birbirini ezer, kuyruk uzar. LongAdder ise süpermarkete 8 ayrı kasa açmak gibidir! Müşteriler farklı kasalara (Cell) dağılır, kimse kimseyi beklemeden parasını öder. Günün sonunda patron kasaların hepsini toplar (sum()) ve toplam ciroyu bulur! Okuma nadir, yazma çoksa LongAdder 100 kat daha hızlıdır!

🇬🇧 ENGLISH (ELI5)

Picture a stadium with a single concession register (AtomicLong). Thousands of fans violently rush that lone cashier simultaneously, resulting in massive bottlenecks and dropped orders. LongAdder opens 16 parallel registers distributed across the stadium (Cell array). Fans transact at their nearest register with zero friction. At halftime, the accountant aggregates balances across all registers (`sum()`). When writes dominate, LongAdder outpaces AtomicLong by orders of magnitude!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Striped64 Mimarisi ve LongAdder Derinliği: 1) Çekişme Azaltma (Contention Mitigation): `AtomicLong` tek bir bellek adresi üzerinde CAS yapar; thread sayısı arttıkça L1 cache geçersiz kılma (cache invalidation) fırtınası kopar. 2) `Cell[]` Dizisi: LongAdder, Doug Lea tarafından yazılan `Striped64` sınıfından türer. Çekişme tespit edildiğinde thread ID'sinin hash'ine göre dinamik olarak `Cell` dizisi büyütülür (2, 4, 8, 16...). Her `Cell` kendi içinde `\@Contended` anotasyonu ile 128 bayt doldurulmuştur (Padding), böylece False Sharing tamamen önlenir! 3) Ne Zaman AtomicLong, Ne Zaman LongAdder?: Eğer atomik bir sıra numarası (Sequence ID: `incrementAndGet()`) gerekiyorsa `AtomicLong` zorunludur. Eğer sadece metrik toplama, sayaç tutma veya istatistik yapılıyorsa kesinlikle `LongAdder` tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Internal Mechanics of Striped64: 1) The Contention Bottleneck of AtomicLong: AtomicLong forces all concurrent cores to serialize writes to a single memory offset, saturating the cache bus with MESI invalidation traffic. 2) Striped64 Cell Dynamism: LongAdder maintains a `base` variable. If a CAS on `base` fails due to contention, it allocates a `Cell[]` table sized to a power of two (capped at CPU core count). Each thread hashes into a specific `Cell`. Crucially, each `Cell` is annotated with `\@jdk.internal.vm.annotation.Contended`, enforcing 128-byte cache-line isolation to eliminate false sharing. 3) Architectural Trade-Off: `LongAdder.sum()` is an eventual snapshot, not an atomic instantaneous freeze. Use `AtomicLong` when sequential order or exact compare-and-set guards are required (`compareAndSet()`); use `LongAdder` for high-frequency telemetry, request counting, and metrics.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Yüksek Ölçekli Metrik Toplama için LongAdder
import java.util.concurrent.atomic.LongAdder;

public class MetricsCollector {
    // Saniyede 1 milyon HTTP isteğini kaydetmek için idealdir
    private final LongAdder requestCount = new LongAdder();

    public void onRequestReceived() {
        requestCount.increment(); // Çarpışmasız, dağıtık hücreye yazar
    }

    public long getTotalRequests() {
        return requestCount.sum(); // Tüm hücreleri toplar
    }

    void main() throws InterruptedException {
        var metrics = new MetricsCollector();
        int threads = 8;
        int reqPerThread = 500_000;

        Thread[] workers = new Thread[threads];
        for (int i = 0; i < threads; i++) {
            workers[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < reqPerThread; j++) {
                    metrics.onRequestReceived();
                }
            });
        }

        for (Thread t : workers) t.join();

        IO.println("Toplam Başarılı İstek: " + metrics.getTotalRequests()); // 4,000,000
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Banka hesap bakiyesi veya sıra numarası için LongAdder kullanabilir miyiz?' Yanıt: 'HAYIR! LongAdder atomik bir `compareAndSet` veya anlık kesin okuma sunmaz. `sum()` çağrısı yapılırken arka planda yazmalar devam edebilir. Sıra numarası için daima AtomicLong kullanılmalıdır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Distinction: 'Can we use LongAdder to issue unique order IDs or manage account balances?' Answer: 'NO! LongAdder does not provide an atomic `compareAndSet()` or transactional read-and-mutate guarantee. `sum()` yields an eventual aggregate, not an isolated lockstep snapshot. Unique sequences require AtomicLong.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LongAdder, yüksek çekişmeli yazma işlemlerinde AtomicLong'dan 10-100 kat daha hızlıdır.
- Striped64 hücreleri \@Contended ile pad edildiği için False Sharing yaşanmaz.
- Benzersiz sıra numarası (Sequence ID) için AtomicLong; metrikler için LongAdder seçilir.

🎯 Key Takeaways for the Interview

- LongAdder achieves 10-100x throughput over AtomicLong under high write contention.
- Striped64 cells are padded to prevent False Sharing across CPU cache lines.
- Use AtomicLong for atomic sequential IDs; use LongAdder for high-volume metrics.

BÖLÜM 3 / MODULE 3

Klasik Senkronizasyon, Monitörler & Kilitler

Classical Synchronization, Monitors & Locks

synchronized iç kilit mekanizması (Intrinsic Lock), HotSpot monitör seviyeleri (Biased, Lightweight, Heavyweight), wait/notify signal tuzakları, ReentrantLock, Condition kuyrukları ve ReentrantReadWriteLock.

JVM intrinsic locks via synchronized, HotSpot lock inflation levels, wait/notify missed signals, ReentrantLock reentrancy, Condition queues, and ReadWriteLock degradation.

13

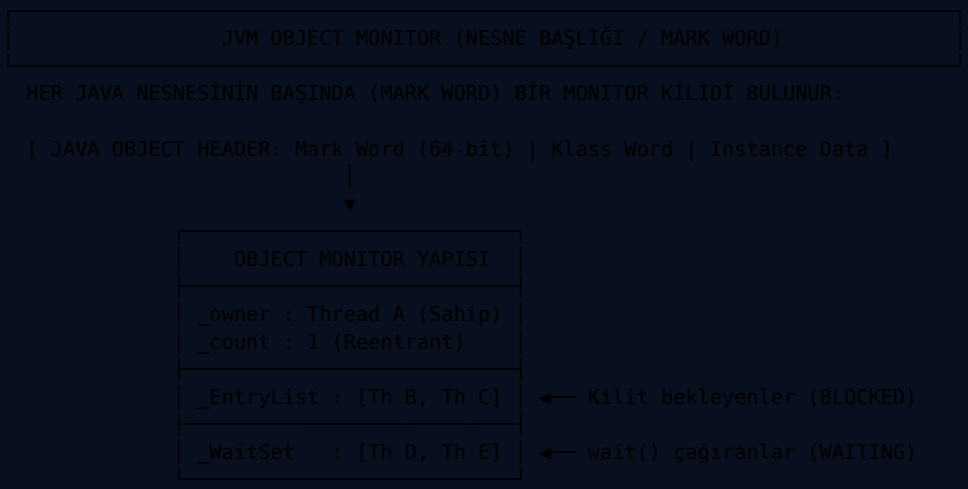
BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

synchronized Anahtar Kelimesi: Intrinsic Lock & JVM Object Monitor Mimarisi

The synchronized Keyword: Intrinsic Locks & JVM Object Monitors

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Her Java nesnesi, kapısında özel bir anahtar deliği olan bir oda gibidir. Bir odaya `synchronized` ile girdiğinizde (monitorenter), içeriden kapıyı kilitlersiniz. Odanın sahibi siz olursunuz (_owner). Kapıda iki tane bekleme salonu vardır: 1) Koridor (_EntryList): İçeri girmek için sıraya girenler (BLOCKED). 2) Balkon (_WaitSet): İçerideyken 'Patron haber verene kadar burada dinleniyorum' diyerek kenara çekilenler (WAITING). Siz odadan çıkarken (monitorexit) anahtarı bırakırsınız ve koridordaki bir kişiyi içeri alırsınız!

🇬🇧 ENGLISH (ELI5)

Every Java object is born with a private hotel room keyhole inside its memory header (Mark Word). When a thread enters a `synchronized` block, it executes `monitorenter`, turning the deadbolt from the inside and claiming ownership (`_owner`). Outside sit two distinct waiting rooms: 1) The Hallway (`_EntryList`): Threads aggressively competing to enter the room (BLOCKED). 2) The Balcony (`_WaitSet`): Threads that were inside, voluntarily surrendered the key via `wait()`, and are resting until signaled (WAITING). Exiting triggers `monitorexit`, unlocking the door for hallway contenders.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Object Monitor ve Bytecode Derinliği: 1) Mark Word: 64-bit HotSpot JVM'de nesne başlığının en alt 2-biti kilit durumunu tutar (01: Kilitsiz, 00: Hafif Kilit, 10: Ağır Monitör Kilidi). 2) Reentrancy (Yeniden Girilebilirlik): Bir thread zaten sahip olduğu bir kilide tekrar girerse (örneğin synchronized metodun içinden başka bir synchronized metot çağırmak), JVM `_count` sayacını 1 artırır; kilitlenip deadlock olmaz. Çıkarken de sayaç düşer. 3) Derleyici Garantisi: `synchronized` bloğu javac tarafından try-finally bloğu gibi derlenir. Metot içinde beklenmedik bir RuntimeException veya Error fırlasa dahi JVM `monitorexit` talimatını kesinlikle çalıştırır; kilit asla askıda kalmaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Monitor Internals & Bytecode: 1) Mark Word State Bits: On 64-bit architectures, the lowest two bits designate monitor states (01 = Unlocked, 00 = Light-weight/Thin, 10 = Heavyweight Inflated Monitor). 2) Reentrancy Mechanism: If Thread A holds a monitor and enters another synchronized block on the same object, the JVM simply increments `_count`. It does not re-acquire or block, preventing self-deadlock. The monitor is released when `_count` hits zero. 3) Bytecode Invariant: javac generates dual `monitorexit` opcodes: one for normal control flow completion and another wrapped in an all-encompassing exception handler, ensuring monitors are guaranteed released even under fatal panics.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Synchronized Reentrancy ve Monitor Davranışı
public class SynchronizedMonitorDemo {
    private final Object lock = new Object();
    private int sharedBalance = 1000;

    public void withdraw(int amount) {
        synchronized (lock) { // monitorenter
            IO.println("Kilit alındı: " + Thread.currentThread().getName());
            auditLog(); // Reentrant çağrı (Aynı kilit, sayaç 2 olur!)
            sharedBalance -= amount;
        } // monitorexit
    }

    private void auditLog() {
        synchronized (lock) { // Aynı thread olduğu için bloke olmaz!
            IO.println("Audit kaydı işlendi. Kalan bakiye: " + sharedBalance);
        }
    }

    void main() {
        var demo = new SynchronizedMonitorDemo();
        demo.withdraw(250);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'synchronized(new Object()) veya synchronized(new String("lock")) yazarsak ne olur?' Yanıt: 'Her thread farklı bir nesne referansı üzerinde kilitleneceği için hiçbir senkronizasyon sağlanamaz! Kilit nesneleri DAİMA `private final Object lock = new Object();` olarak tanımlanmalıdır.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Junior Trap in Interviews: 'What is the catastrophe of `synchronized(new Object())` or `synchronized("literal")`?' Answer: '`new Object()` creates a brand-new monitor per invocation, providing ZERO mutual exclusion. Locking on interned Strings risks accidental cross-library deadlocks. Always declare `private final Object lock = new Object();`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Her Java nesnesi başlığında (Mark Word) bir Object Monitor taşır.
- synchronized kilitleri reentrant'tır; aynı thread aynı kilide defalarca girebilir.
- Kilit nesneleri referans değişimini önlemek için daima private final olmalıdır.

🎯 Key Takeaways for the Interview

- Every Java object carries an intrinsic monitor inside its Mark Word header.
- Intrinsic locks are fully reentrant, incrementing an internal recursion counter.
- Lock target instances must always be private and final to prevent reference mutation.

14

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

HotSpot Kilit Şişmesi: Biased Locking, Lightweight Locking & Heavyweight Monitor

HotSpot Lock Optimization: Biased, Lightweight (Thin) & Heavyweight Lock Inflation

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                HOTSPOT KİLİT ŞİŞMESİ (LOCK INFLATION) AŞAMALARI │
└──────────────────────────────────────────────────────────────┘

1. BIASED LOCKING (Taraflı Kilit - Tarihsel / Tek Thread):
   [ Nesne Başlığı ──► Thread ID yazılır ]
   Kilit maliyeti = 0 (Sadece tek bir thread giriyorsa sıfır senkronizasyon maliyeti).

2. LIGHTWEIGHT LOCK (Hafif Kilit / Çekişmesiz Çoklu Thread):
   Farklı thread'ler giriyor ama AYNI ANDA DEĞİL (Alternating execution).
   [ Nesne Başlığı ──► Thread'in kendi Stack'indeki BasicObjectLock'a işaret eder (CAS) ]
   İşletim sistemi kernel'ına İNİLMEZ, tamamen user-space CAS ile çözülür!

3. HEAVYWEIGHT MONITOR (Ağır Kilit / Çekişmeli Durum):
   İki thread AYNI ANDA kilide saldırır! CAS başarısız olur.
   [ Nesne Başlığı ──► OS seviyesinde ObjectMonitor C++ nesnesine işaret eder ]
   Thread'ler OS kernel seviyesinde BLOCKED durumuna alınır ve uyutulur (Park).
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirketteki toplantı odasını kiralama adımları gibidir: 1) Taraflı Kilit: Odayı hep aynı genel müdür kullanıyorsa, kapıya genel müdürün resmi asılır. Müdür kapıyı çalmadan doğrudan girer (Sıfır maliyet). 2) Hafif Kilit: Odayı ara sıra farklı çalışanlar kullanıyor ama hiç kimse birbiriyle çakışmıyor. Kapıya basit bir mandal konur (Kullanıcı seviyesinde CAS). 3) Ağır Kilit: İki yönetici aynı anda odaya girmeye çalışır ve kapıda kavga çıkar! Güvenlik (OS Kernel) çağrılır, yöneticilerden biri zorla bekleme salonuna oturtulup uyutulur (Context Switch maliyeti)!

🇬🇧 ENGLISH (ELI5)

HotSpot optimizes locking like an escalating conference room protocol: 1) Biased Locking: If Manager A is the sole user, their nameplate stays on the door; they walk in without checking locks (zero cost). 2) Lightweight (Thin) Locking: Different team members book the room, but their schedules never collide. They flip a simple physical door latch using atomic CAS in user space without security guards. 3) Heavyweight Monitor: Two directors arrive simultaneously and collide at the door threshold! Armed security (the OS kernel) intervenes, putting one director into forced sleep in the waiting lounge.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Kilit Optimizasyon Kademeleri: 1) Kilit Şişmesi (Inflation) Tek Yönlüdür: HotSpot'ta kilit Hafif'ten Ağır'a şiştiğinde (Inflatable), genellikle JVM yeniden başlatılana kadar ağır kalır. 2) Lightweight Locking (CAS Lock): Thread, kendi thread stack'inde bir `BasicObjectLock` açar ve nesnenin Mark Word'ünü CAS ile bu stack kaydına yönlendirir. Çekişme yoksa tek bir CAS ile kilit alınır ve bırakılır. 3) Spin-Locking (Uyarlamalı Dönme / Adaptive Spinning): Ağır kilide geçmeden önce HotSpot thread'i hemen uyutmaz; kısa bir süre döngüde (spin) bekler. Eğer kilidi tutan diğer thread hemen işini bitirirse, pahalı işletim sistemi context-switch maliyetinden kurtulunur! 4) Biased Locking Durumu: Modern JDK sürümlerinde (JDK 15+) Biased Locking, safepoint maliyetleri nedeniyle varsayılan olarak kapatılmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Lock Optimization Architecture: 1) One-Way Inflation: Once contention forces an object's Mark Word to point to an inflated C++ `ObjectMonitor`, deflation rarely occurs until safepoint GC phases. 2) Lightweight Locking via Stack Displaced Mark Word: A thread copies the target object's Mark Word to its local stack frame (Displaced Mark Word) and performs a CAS to store the stack address in the object header. If successful, acquisition completes with zero kernel transitions. 3) Adaptive Spinning: Before invoking heavy `pthread_mutex_lock` (which suspends the thread), the JVM spins the core for several clock cycles. If the holding thread releases during the spin, execution proceeds seamlessly without context-switch latency.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Kilit Çekişmesi ve Adaptif Spinning Simülasyonu
public class LockInflationDemo {
    private final Object monitor = new Object();
    private int counter = 0;

    // Çekişmesiz: Hafif Kilit (Lightweight CAS) olarak çalışır
    public void uncontendedOperation() {
        synchronized (monitor) {
            counter++;
        }
    }

    // Yüksek Çekişme: Ağır Kilit (Heavyweight Monitor Inflation) tetikler
    public void contendedOperation() {
        synchronized (monitor) {
            try {
                Thread.sleep(1); // Bilerek kilit içinde beklenir, diğerleri şişer!
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            counter++;
        }
    }

    void main() {
        var demo = new LockInflationDemo();
        demo.uncontendedOperation();
        IO.println("Hafif kilit başarıyla çalıştı: " + demo.counter);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Thread.sleep() kilit içindeyken kilidi bırakır mı?' Yanıt: 'KESİNLİKLE BIRAKMAZ! sleep() kilidi tutmaya devam ederken diğer tüm thread'leri kapıda bloke eder. Kilidi geçici olarak bırakan metot Object.wait()'tir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Crucial Interview Question: 'Does Thread.sleep() release held synchronized monitors?' Answer: 'ABSOLUTELY NOT! `Thread.sleep()` sleeps while retaining ownership of every single held lock, starving other threads. Only `Object.wait()` atomically releases the monitor.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- HotSpot kilitleri çekişmeye göre Lightweight'ten Heavyweight'e şişer (Inflate).
- Adaptif dönme (Adaptive Spinning), thread uyutma (park) maliyetini önler.
- Thread.sleep() kilidi ASLA bırakmaz; Object.wait() ise kilidi bırakır.

🎯 Key Takeaways for the Interview

- HotSpot locks dynamically inflate from Lightweight CAS to Heavyweight OS monitors.
- Adaptive spinning attempts to acquire monitors during spin loops before sleeping.
- Thread.sleep() retains locks; Object.wait() atomically surrenders them.

15

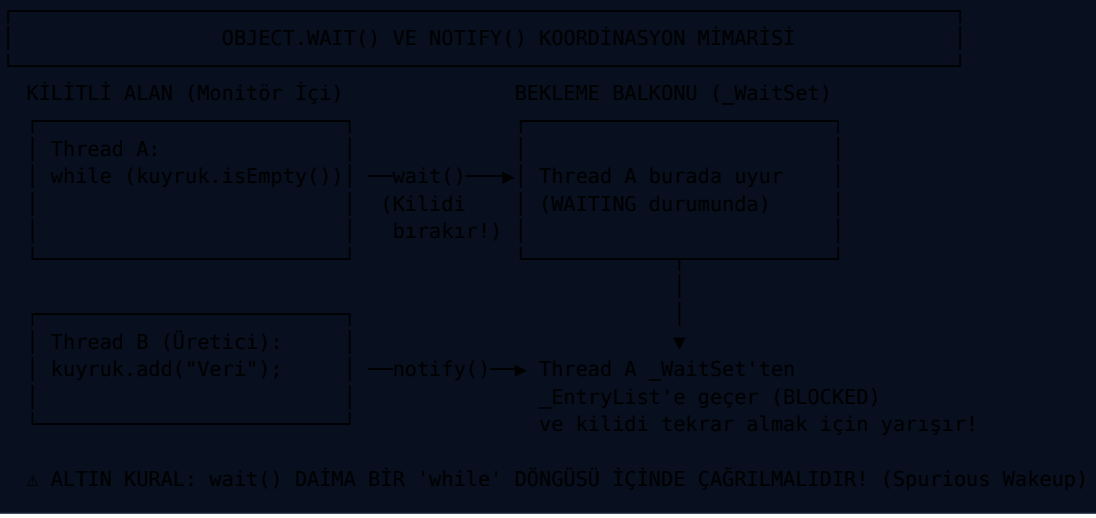
BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

Thread'ler Arası İletişim: wait(), notify(), notifyAll() & Kaçan Sinyal (Missed Signal)

Inter-Thread Signaling: wait(), notify(), notifyAll() & The Missed Signal Hazard

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda müşteri (Tüketici) masaya oturuyor. Garsona 'Bana yemek getir' diyor. Mutfakta yemek yoksa müşteri masayı meşgul etmez (wait() çağırıp kilidi bırakır) ve kenardaki bekleme salonuna geçer. Aşçı (Üretici) yemeği pişirdiği anda 'Yemek hazır!' diye zili çalar (notify()). Müşteri zil sesini duyunca hemen gidip masaya oturmaz; önce tabağa bakar: 'Gerçekten yemek geldi mi, yoksa garson yanlışlıkla mı zile bastı?' (while döngüsü). Eğer yemek varsa yer; yoksa tekrar beklemeye geçer!

🇬🇧 ENGLISH (ELI5)

Imagine a patron at a diner. If the soup pot is empty, the customer cannot sit occupying the single service stool. They step aside to the lounge via `wait()`, atomically relinquishing the stool. When the chef brews fresh soup, the chef rings the service bell via `notify()`. The patron awakes, but does NOT blindly eat. They inspect the pot inside a `while` loop: 'Is there soup, or did another patron grab it first, or did lightning ring the bell (Spurious Wakeup)?' If soup exists, they eat; otherwise, they wait again.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

wait/notify Koordinasyonunda 3 Hayati Kural: 1) Neden synchronized içinde olmalıdır?: `wait()` veya `notify()` kilitsiz çağrılırsa JVM anında `IllegalMonitorStateException` fırlatır; çünkü paylaşılan durumu kontrol etmek ve sinyal göndermek atomik bir bütün olmak zorundadır. 2) Spurious Wakeup (Sahte Uyanma): İşletim sistemi sinyalleri veya kernel kesintileri yüzünden bir thread hiçbir `notify()` çağrılmasa dahi `wait()`ten uyanabilir. Bu yüzden `if (condition) wait();` yazmak ölümcül bir hatadır; DAİMA `while (condition) wait();` yazılmalıdır! 3) notify() vs notifyAll(): `notify()` rastgele tek bir thread uyandırır; uyandırılan thread işi yapamazsa tüm sistem sonsuza dek kilitlenebilir (Deadlock / Missed Signal). Bu yüzden özel bir optimizasyon gerekmiyorsa her zaman `notifyAll()` kullanılmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Mechanics of Condition Signaling: 1) Prerequisite Monitor Ownership: Invoking `wait()` or `notify()` outside of a synchronized block targeting that exact object immediately throws `IllegalMonitorStateException`. 2) Spurious Wakeups: Under POSIX threading implementations, waiting threads can awaken absent any explicit notification due to OS-level signal handling. Guarding `wait()` with an `if` statement causes race conditions where unverified invariants execute; `while (!condition) { wait(); }` is mathematically required. 3) The `notify()` vs `notifyAll()` Dilemma: `notify()` signals an arbitrary thread from `_WaitSet`. If that thread is unable to satisfy state requirements, the notification is lost, hanging the entire pool. `notifyAll()` broadcasts to all waiting threads, shifting them to `_EntryList`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Klasik Producer-Consumer (Üretici-Tüketici) Kuyruğu
import java.util.LinkedList;
import java.util.Queue;

public class BoundedBlockingQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public BoundedBlockingQueue(int capacity) {
        this.capacity = capacity;
    }

    public synchronized void put(T item) throws InterruptedException {
        // ⚠ DAİMA WHILE DÖNGÜSÜ! Spurious Wakeup koruması
        while (queue.size() == capacity) {
            wait(); // Kuyruk dolu, yer açılana kadar kilidi bırakıp bekle
        }
        queue.add(item);
        notifyAll(); // Tüketicilere haber ver: Veri geldi!
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) {
            wait(); // Kuyruk boş, veri gelene kadar bekle
        }
        T item = queue.remove();
        notifyAll(); // Üreticilere haber ver: Yer açıldı!
        return item;
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Neden if yerine while kullanıyoruz?' diye sorarsa: '1) Sahte uyanmalar (Spurious Wakeup) yüzünden thread sinyal olmadan uyanabilir. 2) notifyAll çağrıldığında uyandırılan diğer bir thread araya girip kuyruktaki elemanı kapmış olabilir. Bu yüzden uyanan her thread koşulu tekrar doğrulamak zorundadır!' denmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory Interview Answer: 'Why must wait() always sit in a while loop rather than an if block?' Answer: '1) Spurious wakeups allow threads to awaken without signals. 2) When notifyAll awakens multiple threads, another thread may race ahead and consume the state first. The awakening thread must re-evaluate the invariant before proceeding.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- wait(), notify() ve notifyAll() sadece synchronized blok içinde çağrılabilir.
- Sahte uyanmalara (Spurious Wakeup) karşı wait() daima while döngüsü içinde olmalıdır.
- Kaçan sinyal (Missed Signal) hatalarını önlemek için varsayılan olarak notifyAll seçilir.

🎯 Key Takeaways for the Interview

- wait() and notify() require active monitor ownership; otherwise IllegalMonitorStateException.
- Always guard wait() with while(!condition) to defend against spurious wakeups.
- Favor notifyAll() over notify() to prevent lost signal deadlocks.

16

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

ReentrantLock: Açık Kilitler, Adil (Fair) vs Haksız (Non-Fair) & Kesilebilirlik

ReentrantLock: Explicit Locking, Fair vs Non-Fair & Lock Interruptibility

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
SYNCHRONIZED vs REENTRANTLOCK MİMARİ KARŞILAŞTIRMASI

ÖZELLİK                 SYNCHRONIZED              REENTRANTLOCK
-------------------------------------------------------------------
Kilit Bırakma           Otomatik (Compiler)       Manuel (finally { unlock(); })
Zaman Aşımı (Timeout)   YOK (Sonsuza dek bekler)  VAR: tryLock(5, SECONDS)
Kesilebilirlik          YOK (Kesilemez)           VAR: lockInterruptibly()
Adillik (Fairness)      Sadece Haksız (Barging)   Fair / Non-Fair seçilebilir
Çoklu Kuyruk            Tek bir WaitSet           Birden fazla Condition nesnesi

NON-FAIR LOCK (Haksız Kilit - Yüksek Throughput):
Yeni gelen thread kuyrukta bekleyenleri beklemeden doğrudan kilide saldırır (Barging)!
Context switch maliyetinden kaçar, %400 daha hızlıdır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Geleneksel anahtarlar (synchronized) basittir; kapıyı açar girersiniz, çıkınca kapı arkadan otomatik kilitlenir. Ama diyelim ki kapının önünde 3 saat beklemek istemiyorsunuz: 'Eğer 10 saniye içinde açılmazsa ben gidiyorum' demek istiyorsunuz (`tryLock(timeout)`). Ya da yangın çıktığında bekleyen adamı kolundan çekip dışarı çıkarmak istiyorsunuz (`lockInterruptibly()`). İşte ReentrantLock, cebinizde zamanlayıcısı, acil durum butonu ve sıra numarası olan son teknoloji akıllı kilit sistemidir!

🇬🇧 ENGLISH (ELI5)

An intrinsic lock (`synchronized`) is an old-fashioned deadbolt: you wait until it opens, with no escape. ReentrantLock is an enterprise smart lock: 1) You can say: 'If the door doesn't unlock in 5 seconds, I'm leaving' (`tryLock(timeout)`). 2) In a fire alarm, a thread waiting for the lock can be canceled immediately (`lockInterruptibly()`). 3) You can enforce a strict first-come-first-served line (Fairness) or allow energetic newcomers to barge in immediately for 4x higher throughput (Non-Fair).

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ReentrantLock ve AQS Derinliği: 1) AbstractQueuedSynchronizer (AQS): ReentrantLock, temelinde çift yönlü bir FIFO bekleme kuyruğu (CLH lock varyantı) ve bir `volatile int state` tutar. 2) Fair vs Non-Fair Kilit: Non-fair kilit (varsayılan) 'Barging' uygular. Yeni gelen thread, kuyrukta bekleyenler olmasına rağmen önce bir CAS ile kilidi kapmaya çalışır. Eğer o an kilit boştaysa, kuyruk uyandırma (unpark) ve context-switch gecikmesini atlayarak kilidi alır; throughput dramatik yükselir. Fair lock ise daima kuyruk başındaki thread'e verir, adaleti sağlar ancak gecikmeyi katlar. 3) finally Bloğu Zorunluluğu: `lock.unlock()` daima `try-finally` bloğunun finally kısmında olmalıdır; aksi halde bir exception fırladığında kilit sonsuza dek kilitli kalır ve sistem felç olur!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architectural Internals of ReentrantLock: 1) AQS Foundation: ReentrantLock wraps `Sync extends AbstractQueuedSynchronizer`. State 0 represents unlocked; state >= 1 denotes reentrant holds by the owning thread. 2) The Non-Fair Barging Advantage: `NonfairSync` attempts an immediate CAS on `state` before enqueuing. If an unparking thread has not yet acquired the CPU core, a newly arriving active thread sneaks in, eliminating two context-switch penalties. This yields massive throughput gains at the expense of potential thread starvation. 3) Non-Negotiable Idiom: Because the compiler cannot enforce automatic release, `lock.lock()` must immediately precede a `try` block, with `lock.unlock()` residing strictly inside `finally`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ReentrantLock ile Zaman Aşımlı (tryLock) ve Kesilebilir Kilit
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

public class AdvancedLockingDemo {
    private final ReentrantLock lock = new ReentrantLock(false); // Non-fair (Hızlı)

    public void processPayment() {
        lock.lock();
        try {
            IO.println("Ödeme işlemi kilitlendi: " + Thread.currentThread().getName());
        } finally {
            lock.unlock(); // ⚠ KESİNLİKLE finally içinde olmalıdır!
        }
    }

    public boolean tryProcessWithTimeout() throws InterruptedException {
        // 2 saniye içinde kilit alamazsa vazgeç (Deadlock önleyici!)
        if (lock.tryLock(2, TimeUnit.SECONDS)) {
            try {
                IO.println("Zaman aşımlı kilit alındı!");
                return true;
            } finally {
                lock.unlock();
            }
        } else {
            IO.println("Kilit meşguldü, işlem timeout'a uğradı!");
            return false;
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Ölümcül Kodlama Hatası: `lock.lock()` satırını `try` bloğunun İÇİNE yazmayın! Eğer `lock()` anında bir hata fırlatırsa, `finally` bloğu çalışır ve sahip olunmayan bir kilit için `unlock()` çağrılır (`IllegalMonitorStateException`). Kural: `lock.lock(); try { ... } finally { lock.unlock(); }`!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Severe Anti-Pattern: Never place `lock.lock()` INSIDE the `try` block! If `lock()` throws before acquisition, `finally` executes `unlock()`, triggering an `IllegalMonitorStateException` that masks the root cause. Idiom: Invoke `lock.lock()` strictly on the line preceding `try`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ReentrantLock; tryLock(), timeout ve interruptibility esnekliği sunar.
- Non-fair kilitler context switch maliyetini baypas ettiği için çok daha yüksek throughput üretir.
- unlock() çağrısı DAİMA finally bloğu içerisinde yer almalıdır.

🎯 Key Takeaways for the Interview

- ReentrantLock provides tryLock(), timeouts, and interruption capabilities.
- Non-fair locks bypass context switches via barging, delivering superior throughput.
- unlock() must strictly execute inside a finally block to prevent resource leaks.

17

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

Condition Değişkenleri: Çoklu Bekleme Kuyrukları (notFull & notEmpty)

Condition Variables: Multi-Queue Coordination (notFull & notEmpty)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
TEK BİR REENTRANTLOCK İÇİNDE ÇİFT BEKLEME KUYRUĞU

ReentrantLock lock = new ReentrantLock();
Condition notFull  = lock.newCondition(); // Üreticilerin bekleme kuyruğu
Condition notEmpty = lock.newCondition(); // Tüketicilerin bekleme kuyruğu

[ ÜRETİCİ (PUT) ]:
Kuyruk doluysa ──► notFull.await() ──► Sadece notFull kuyruğunda bekler!
Eleman ekleyince ──► notEmpty.signal() ──► YALNIZCA Tüketicileri uyandırır!

[ TÜKETİCİ (TAKE) ]:
Kuyruk boşsa ──► notEmpty.await() ──► Sadece notEmpty kuyruğunda bekler!
Eleman çekince ──► notFull.signal() ──► YALNIZCA Üreticileri uyandırır!

AVANTAJ: synchronized gibi herkesi tek bir torbaya atıp uyandırmaz (Sıfır İtişme)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski sistemde (synchronized wait/notify) lokantada hem yemek bekleyen aç müşteriler hem de boş tabak bekleyen bulaşıkçılar aynı odada bekliyordu. Aşçı 'Yemek hazır' dediğinde herkesi aynı anda uyandırıyordu (notifyAll). Bulaşıkçılar uyanıp 'Bize yemek lazım değil ki' diyip tekrar uyuyordu (Boşa enerji). Condition ise lokantaya iki ayrı kapı koymaktır: 1. Kapı: 'Yemek Bekleyenler' (notEmpty). 2. Kapı: 'Temiz Tabak Bekleyenler' (notFull). Aşçı yemek yapınca sadece müşterilerin kapısını çalar; bulaşıkçıları boş yere uyandırmaz!

🇬🇧 ENGLISH (ELI5)

In intrinsic `wait/notify`, producers and consumers share a single noisy waiting room. When a producer delivers an item and calls `notifyAll()`, both idle producers and consumers wake up, frantically thrashed from sleep only to realize the notification wasn't for them. `Condition` splits the room into targeted queues: `notFull` for producers and `notEmpty` for consumers. Producers wake only consumers; consumers wake only producers. Zero wasted CPU cycles!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Condition Mimarisi ve ArrayBlockingQueue: 1) Çoklu Bekleme Seti: Tek bir kilit üzerinden `lock.newCondition()` ile onlarca farklı koşul kuyruğu oluşturulabilir. 2) await() ve signal(): `await()` kilidi bırakıp thread'i o Condition'a özel AQS bekleme kuyruğuna sokar. `signal()` ise sadece o kuyruğun başındaki tek bir thread'i kilit için yarışmaya gönderir. 3) Neden signal() güvenlidir?: Çünkü `notEmpty.signal()` çağrıldığında uyandırılan kişinin KESİNLİKLE bir tüketici olduğu garanti altındadır! `notify()` gibi yanlışlıkla başka bir üreticiyi uyandırıp sistemi kilitleme tehlikesi yoktur. JDK'nın `ArrayBlockingQueue` sınıfı tamamen bu desenle yazılmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Condition Queues and ArrayBlockingQueue Topology: 1) Dedicated AQS Wait Nodes: Each `ConditionObject` instance maintains its own first/last FIFO node pointers. When a thread executes `await()`, AQS saves the hold count, fully releases the lock, and enqueues the thread in the condition queue. 2) Targeted Signaling: Calling `signal()` unlinks the first node from the condition queue and transfers it to the AQS sync queue (`_EntryList` equivalent). 3) Safety of `signal()` over `signalAll()`: Because distinct conditions cleanly isolate producers from consumers, calling `notEmpty.signal()` is 100% immune to the missed-signal hazards that plague intrinsic `notify()`. This is the exact implementation powering `java.util.concurrent.ArrayBlockingQueue`.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Condition Değişkenleri ile Yüksek Başarımlı Bounded Queue
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class ConditionQueue<T> {
    private final Object[] items;
    private int count, putIndex, takeIndex;

    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull  = lock.newCondition(); // Üretici kuyruğu
    private final Condition notEmpty = lock.newCondition(); // Tüketici kuyruğu

    public ConditionQueue(int capacity) {
        this.items = new Object[capacity];
    }

    public void put(T x) throws InterruptedException {
        lock.lock();
        try {
            while (count == items.length) {
                notFull.await(); // Yer yoksa üretici bekler
            }
            items[putIndex] = x;
            if (++putIndex == items.length) putIndex = 0;
            count++;
            notEmpty.signal(); // Sadece tüketicilere sinyal ver!
        } finally {
            lock.unlock();
        }
    }

    @SuppressWarnings("unchecked")
    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (count == 0) {
                notEmpty.await(); // Veri yoksa tüketici bekler
            }
            T x = (T) items[takeIndex];
            if (++takeIndex == items.length) takeIndex = 0;
            count--;
            notFull.signal(); // Sadece üreticilere sinyal ver!
            return x;
        } finally {
            lock.unlock();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Condition.await() çağrılmadan önce lock.lock() çağrılmazsa ne olur?' Yanıt: 'Tıpkı synchronized olmadan wait() çağrıldığında olduğu gibi IllegalMonitorStateException fırlatılır! Condition'lar ait oldukları ReentrantLock'a göbekten bağlıdır.'

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Requirement: Invoking `condition.await()` or `condition.signal()` without holding the associated `ReentrantLock` immediately throws `IllegalMonitorStateException`. The calling thread must hold the lock that owns the condition.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Condition, tek bir kilit altında birden fazla bağımsız bekleme kuyruğu oluşturur.

• notFull ve notEmpty ayrımı sayesinde signal() güvenle kullanılabilir.

• JDK'nın ArrayBlockingQueue sınıfı Condition mimarisi üzerine inşa edilmiştir.

🎯 Key Takeaways for the Interview

• Condition variables decouple wait-sets into distinct semantic queues.

• Condition queues make signal() completely safe by targeting specific thread roles.

• ArrayBlockingQueue is the canonical enterprise implementation of Condition queues.

18

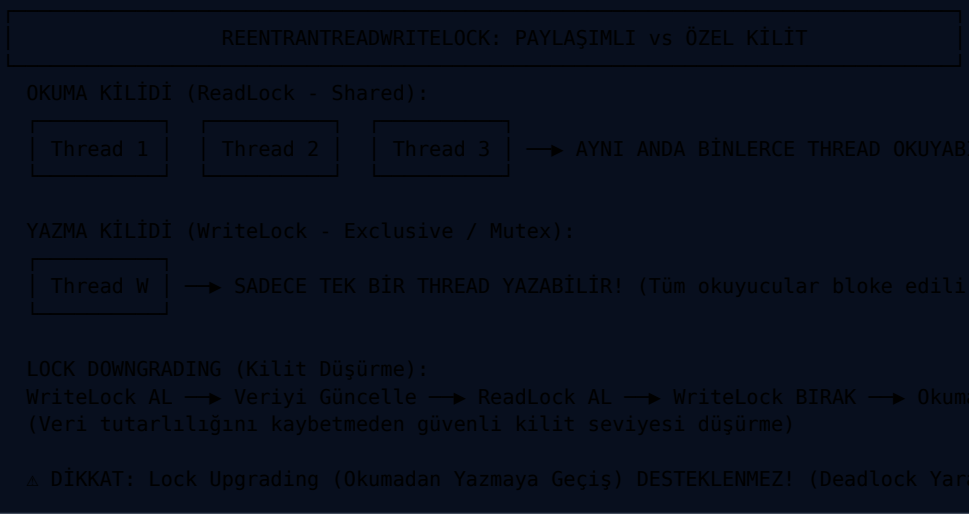
BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

ReentrantReadWriteLock: Okuma Yoğun Sistemler & Kilit Düşürme (Lock Downgrading)

ReentrantReadWriteLock: Read-Intensive Scaling & Lock Downgrading Mechanics

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki büyük ansiklopedi düşünün. Aynı anda 100 öğrenci gelip ansiklopediyi sessizce okuyabilir (Okuma Kilidi / ReadLock); kimse kimseyi engellemez. Ama ansiklopedideki hatalı bir sayfayı yırtıp yenisini yapıştıracak olan matbaacı (Yazma Kilidi / WriteLock) geldiğinde, kütüphanedeki bütün öğrencileri dışarı çıkartır ve kapıyı kilitler! İşini bitirince kapıyı açar ve öğrenciler tekrar içeri dolar. Okuma çok, yazma azsa sistem roket gibi hızlanır!

🇬🇧 ENGLISH (ELI5)

Imagine a public stock ticker display. Thousands of traders can stare at the display concurrently without interfering with one another (Shared Read Lock). However, when the exchange operator updates the asset prices (Exclusive Write Lock), everyone must pause until the prices settle. If 99% of requests are reads and 1% are writes, a ReadWriteLock boosts concurrency a thousandfold compared to a blunt mutex!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ReadWriteLock Mimarisi ve Kilit Düşürme Kuralları: 1) Durum Yönetimi (State Split): AQS'in 32-bitlik tek `state` tamsayısı ikiye bölünür: Üst 16 bit okuma kilidi sayısını (Shared count), alt 16 bit ise yazma kilidi sayısını (Exclusive count) tutar. 2) Lock Downgrading (Kilit Düşürme): Yazma kilidini tutarken önce okuma kilidi alınır, ardından yazma kilidi bırakılır. Bu sayede yazma ile okuma arasına başka bir yazıcının girmesi engellenerek veri tazeliği garanti edilir. 3) Lock Upgrading Neden Yoktur?: Eğer iki okuyucu thread aynı anda yazma kilidine yükselmek (upgrade) isterse, ikisi de birbirinin okuma kilidini bırakmasını bekler ve anında kilitlenir (Deadlock). Bu yüzden Java'da kilit yükseltme yasaktır!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ReadWriteLock Internals & Downgrade Protocols: 1) 32-Bit State Partitioning: AQS encodes dual locks within a single atomic integer: the high 16 bits record shared read acquisitions; the low 16 bits record exclusive write hold counts. 2) Lock Downgrade Sequence: While holding the write lock, acquire the read lock, then release the write lock. This preserves atomic continuity; no competing writer can slip in between the write release and read acquisition. 3) The Ban on Lock Upgrading: Upgrading from read lock to write lock is strictly disallowed. If Thread A and Thread B both hold read locks and attempt to upgrade, both freeze waiting for the other to release, creating an immediate deadlock.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Lock Downgrading (Kilit Düşürme) Deseni
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class CacheWithLockDowngrade {
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
    private String cachedData = "Initial Data";
    private boolean cacheValid = false;

    public void processData() {
        rwLock.readLock().lock();
        if (!cacheValid) {
            // Önce okuma kilidini bırakmak zorundayız!
            rwLock.readLock().unlock();

            // Yazma kilidini al
            rwLock.writeLock().lock();
            try {
                if (!cacheValid) { // Re-check
                    cachedData = "Fresh Data from DB";
                    cacheValid = true;
                }
                // ⚠ KİLİT DÜŞÜRME (LOCK DOWNGRADE) BURADA BAŞLAR:
                rwLock.readLock().lock(); // 1. Okuma kilidini AL
            } finally {
                rwLock.writeLock().unlock(); // 2. Yazma kilidini BIRAK (Kilit düşürüldü!)
            }
        }

        try {
            // Veriyi güvenle oku
            IO.println("İşlenen Veri: " + cachedData);
        } finally {
            rwLock.readLock().unlock(); // 3. Son okuma kilidini BIRAK
        }
    }

    void main() {
        new CacheWithLockDowngrade().processData();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'ReentrantReadWriteLock kullanırken okuma kilidi içindeyken yazma kilidi alabilir miyiz (Lock Upgrading)?' diye sorar. Yanıt: 'HAYIR! Kilit yükseltme (Lock Upgrading) Deadlock riski nedeniyle Java'da desteklenmez; thread sonsuza dek kilitlenir. Ancak Kilit Düşürme (Lock Downgrading) tamamen desteklenir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic MAANG Trap: 'Can a thread holding a read lock directly upgrade to a write lock?' Senior Answer: 'NO! Lock upgrading is strictly prohibited in Java to prevent deadlocks. Attempting to acquire a write lock while holding a read lock deadlocks the thread forever. However, Lock Downgrading (write to read) is fully supported.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ReadWriteLock, okuma yoğun (%90+ okuma) sistemlerde devasa verim artışı sağlar.
- Kilit Düşürme (Downgrade: Write -> Read) güvenle desteklenir.
- Kilit Yükseltme (Upgrade: Read -> Write) deadlock nedeniyle yasaktır.

🎯 Key Takeaways for the Interview

- ReadWriteLock shines in read-heavy workloads (90%+ reads).
- Lock Downgrading (WriteLock -> ReadLock -> unlock WriteLock) is fully supported.
- Lock Upgrading is strictly prohibited due to inherent cyclic deadlocks.

BÖLÜM 4 / MODULE 4

İleri Düzey Senkronizasyon & Kilit-Siz Araçlar

Advanced Synchronization & Lock-Free Primitives

StampedLock iyimser okuma mekanizması, CountDownLatch tek seferlik kapılar, CyclicBarrier tekrar kullanılabilir buluşma noktaları, Semaphore kaynak havuzları, Exchanger ve dinamik Phaser mimarisi.

StampedLock optimistic reading stamps, CountDownLatch fan-out gates, CyclicBarrier reusable rendezvous, Semaphore resource pools, pairwise Exchanger, and dynamic multi-phase Phaser.

19

BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

StampedLock: İyimser Okuma (Optimistic Reading) & Stamp Doğrulama

StampedLock: Optimistic Reading, Stamp Validation & Upgrades

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                STAMPEDLOCK: İYİMSER OKUMA (OPTIMISTIC READ) DÖNGÜSÜ

 1. İYİMSER OKUMA BAŞLAT:
    long stamp = sl.tryOptimisticRead(); // SIFIR KİLİT! Bellek durumu snapshot'ı al.

 2. YEREL DEĞİŞKENLERE KOPYALA:
    double currentX = x;
    double currentY = y;

 3. STAMP'I DOĞRULA (validate):
    if (!sl.validate(stamp)) { ──► "Biri araya girip yazdı mı?"
        // EVET! Araya bir yazıcı girdi, veriler tutarsız olabilir!
        // PES ETME: Karamsar (Pessimistic) Okuma Kilidine geç:
        stamp = sl.readLock();
        try {
            currentX = x;
            currentY = y;
        } finally {
            sl.unlockRead(stamp);
        }
    }
 AVANTAJ: Yazıcı yoksa %100 KİLİTSİZ ve sıfır CAS maliyetiyle okuma tamamlanır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir müzedeki tarihi bir saatin akrep ve yelkovanına bakmak istiyorsunuz. Normal sistemde saatçiyi durdurup saati kilitlersiniz. StampedLock'un 'İyimser Okuması'nda ise saati hiç kilitlemezsiniz! Yanından geçerken akrep ile yelkovana hızlıca göz ucuyla bakarsınız (tryOptimisticRead). Sonra elinizdeki bilete bakıp sorarsınız: 'Ben bakarken saatçi saati kurcaladı mı?' (validate). Saatçi dokunmadıysa saati öğrendiniz, bitti! Dokunduysa hemen geri döner, bu sefer saati durdurup garantiye alarak okursunuz!

🇬🇧 ENGLISH (ELI5)

Imagine checking the time on a grand clock. Under standard locking, you halt the clockwork mechanism while looking. StampedLock's 'Optimistic Read' never halts the clockwork! You glance at the hands as you walk past (`tryOptimisticRead`). Then you check your ticket validation stamp: 'Did the clockmaker wind the hands while I was glancing?' (`validate`). If not, your read is mathematically valid with ZERO lock overhead! If the stamp invalidated, you gracefully fall back to a full pessimistic lock.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

StampedLock Mimarisi ve Reentrancy Uyarısı: 1) İyimser Okuma (Optimistic Read): `tryOptimisticRead()` bir kilit almaz; yalnızca sürüm numarasını (Stamp) döner. Okuma sırasında herhangi bir yazıcı thread `writeLock()` alırsa stamp sürümü değişir ve `validate(stamp)` false döner. 2) ReentrantReadWriteLock'tan Farkı: RWLock'ta okuyucular yazıcıları tamamen bloke eder; bu da 'Writer Starvation' (yazıcıların aç kalması) sorununa yol açar. StampedLock'ta iyimser okuyucular yazıcıları ASLA bloke etmez! 3) ⚠️ EN BÜYÜK TUZAK (Reentrant DEĞİLDİR!): StampedLock `Reentrant` DEĞİLDİR! Bir thread tuttuğu `readLock()` varken tekrar `readLock()` almaya çalışırsa veya kilit içinde kendini çağıran metotlar varsa DEADLOCK meydana gelir!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

StampedLock Internal Topology: 1) Stamp Encoding: StampedLock encodes lock state as a 64-bit integer combining sequence bits and reader counts. An optimistic read returns the current sequence stamp with zero memory writes. 2) Writer Starvation Immunity: In `ReentrantReadWriteLock`, continuous readers starve pending writers indefinitely. StampedLock allows writers to acquire the exclusive lock instantly, invalidating optimistic readers without stalling. 3) ⚠️ Critical Fatal Constraint: StampedLock is NON-REENTRANT! If a thread holding a read or write lock calls another method attempting to acquire the same StampedLock, it deadlocks itself immediately. It also does NOT implement the standard `Lock` interface.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: StampedLock ile İyimser Okumalı 2D Nokta Modeli
import java.util.concurrent.locks.StampedLock;

public class Point2D {
    private double x, y;
    private final StampedLock sl = new StampedLock();

    public void move(double deltaX, double deltaY) {
        long stamp = sl.writeLock(); // Karamsar Yazma Kilidi
        try {
            x += deltaX;
            y += deltaY;
        } finally {
            sl.unlockWrite(stamp);
        }
    }

    public double distanceFromOrigin() {
        // 1. İyimser Okuma Dene (Kilit YOK!)
        long stamp = sl.tryOptimisticRead();
        double curX = x, curY = y;

        // 2. Araya yazıcı girdi mi kontrol et
        if (!sl.validate(stamp)) {
            // Yazıcı girmiş! Karamsar okuma kilidine düş:
            stamp = sl.readLock();
            try {
                curX = x;
                curY = y;
            } finally {
                sl.unlockRead(stamp);
            }
        }
        return Math.sqrt(curX * curX + curY * curY);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı StampedLock Sorusu: 'StampedLock reentrant mıdır?' Yanıt: 'HAYIR! Kesinlikle reentrant değildir. Aynı thread aynı kilide ikinci kez girmeye çalışırsa anında self-deadlock yaşanır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory StampedLock Interview Check: 'Is StampedLock reentrant?' Senior Answer: 'NO! StampedLock is strictly NON-REENTRANT. Attempting reentrant acquisition on the same thread will permanently deadlock the application.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- StampedLock iyimser okumada hiçbir kilit almaz; yazıcıları asla bloke etmez.
- validate(stamp) başarısız olursa karamsar readLock'a geri dönülür.
- StampedLock REENTRANT DEĞİLDİR; kendini çağıran kilitler deadlock olur.

🎯 Key Takeaways for the Interview

- Optimistic reads in StampedLock acquire no lock and never block writers.
- If validate() fails, execution falls back gracefully to a pessimistic readLock.
- StampedLock is NOT reentrant; self-acquisitions cause instant deadlocks.

20

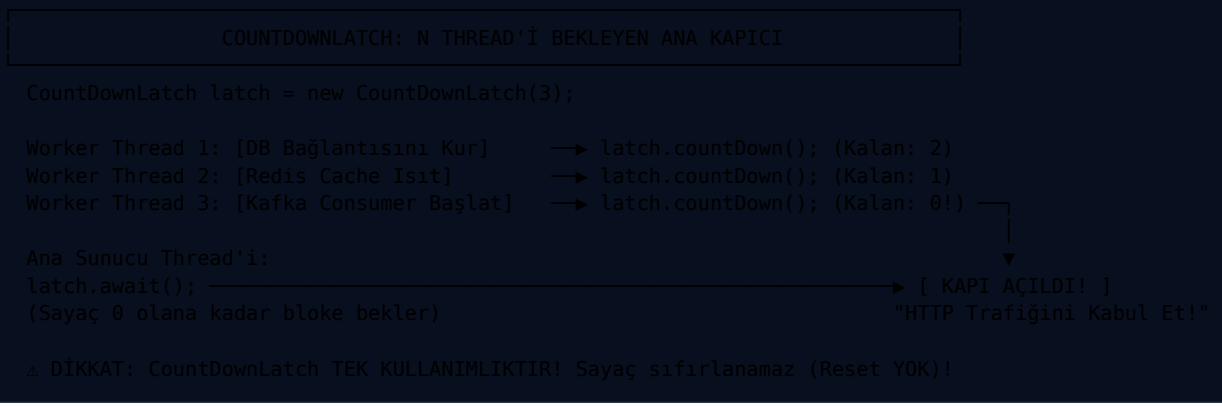
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

CountDownLatch: Tek Seferlik Geri Sayım Kapıları & Dağıtık Fan-Out

CountDownLatch: One-Shot Countdown Gates for Distributed Fan-Out

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir roket fırlatması düşünün. Ana kumandanın (Main Thread) 'ATEŞLE' düğmesine basması için 3 uzmanın yeşil ışık yakması gerekir: 1. Yakıt uzmanı onaylar (countDown), 2. Motor uzmanı onaylar (countDown), 3. Rota uzmanı onaylar (countDown). Sayaç 3'ten geriye sayar. Sayaç sıfıra ulaştığı anda fırlatma kapısı açılır (`await()` biter) ve roket uzaya fırlar! Bu kilit tek kullanımlıktır; bir kere sıfıra indi mi bir daha kurulamaz!

🇬🇧 ENGLISH (ELI5)

Think of a rocket launch protocol. The mission commander (`main` thread) holds their finger over the ignition switch (`latch.await()`). Three independent engineers must report ready: Telemetry (`countDown()`), Propulsion (`countDown()`), and Life Support (`countDown()`). As the latch decrements from 3 to 0, the gate springs open, releasing the commander to initiate liftoff. CountDownLatch is strictly single-use; once the count hits zero, it cannot be reset.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CountDownLatch ve AQS Paylaşımlı Durum Mimarisi: 1) AQS Shared Mode: Latch, AQS'in `acquireSharedInterruptibly` ve `releaseShared` metotlarını kullanır. İlk sayaç AQS'in `state` değişkenine yazılır. 2) countDown() Atomikliği: Her `countDown()` çağrısı `state` değişkenini CAS ile 1 azaltır. Sayaç tam 0 olduğu anda, AQS bekleme kuyruğundaki tüm thread'leri `LockSupport.unpark()` ile aynı anda uyandırır. 3) Fan-Out / Fan-In Deseni: MAANG mikromimarisinde bir ana API isteği geldiğinde 5 farklı alt servise (Fiyat, Stok, Kullanıcı, Öneri, Kargo) paralel istek atılır. Ana thread `latch.await(500, TimeUnit.MILLISECONDS)` ile bekler; tüm servisler yanıt verince sonuçlar birleştirilir (Fan-In).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CountDownLatch Mechanics in AQS: 1) Shared Synchronizer: Unlike ReentrantLock which acquires exclusive state, CountDownLatch employs AQS Shared Mode. The initial constructor parameter sets the internal AQS `state`. 2) Atomic Decrement: `countDown()` executes CAS loops decrementing `state`. When `state` transitions to 0, it calls `doReleaseShared()`, cascading down the CLH queue unparking every thread blocked on `await()`. 3) One-Shot Invariant: Once `state == 0`, subsequent calls to `countDown()` are inert NOOPs, and calls to `await()` return immediately. It cannot be rewound; reusable synchronization requires `CyclicBarrier`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: CountDownLatch ile Paralel Mikroservis Fan-Out / Fan-In
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.TimeUnit;

public class MicroserviceAggregator {
    void main() throws InterruptedException {
        int serviceCount = 3;
        var latch = new CountDownLatch(serviceCount);

        IO.println("Ana Servis: Alt servis çağrıları başlatılıyor...");

        // 3 paralel servis çağrısı başlat
        Thread.ofPlatform().name("PriceService").start(() -> {
            try { Thread.sleep(200); IO.println("✔ Fiyat Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        Thread.ofPlatform().name("InventoryService").start(() -> {
            try { Thread.sleep(150); IO.println("✔ Stok Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        Thread.ofPlatform().name("RecommendationService").start(() -> {
            try { Thread.sleep(300); IO.println("✔ Öneri Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        // Maksimum 500ms bekle (SLA Koruması)
        boolean allDone = latch.await(500, TimeUnit.MILLISECONDS);
        if (allDone) {
            IO.println("🎉 Tüm alt servisler yanıt verdi! Yanıt birleştirildi.");
        } else {
            IO.println("⚠ Bazı servisler zaman aşımına uğradı, fallback yanıtı dönülüyor!");
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kritik Mülakat Hatası: `countDown()` metodunu `try-finally` bloğu içine almamak! Eğer worker thread içinde beklenmedik bir RuntimeException fırlarsa ve `countDown()` çağrılamazsa, ana thread sonsuza dek `await()`te asılı kalır! Kural: `try { ... } finally { latch.countDown(); }`!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Pitfall: Omitting `latch.countDown()` from a `finally` block! If a worker thread throws an unhandled RuntimeException before reaching `countDown()`, the counter never decrements to zero, hanging the coordinating thread permanently.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CountDownLatch tek kullanımlıktır; sayaç sıfıra ulaştığında bir daha sıfırlanamaz.
- countDown() çağrıları thread çökmelerine karşı DAİMA finally içinde olmalıdır.
- Mikroservis paralel sorgulamalarında (Fan-Out/Fan-In) zaman aşımlı await kullanılır.

🎯 Key Takeaways for the Interview

- CountDownLatch is strictly single-use and cannot be reset once drained.
- Always wrap countDown() invocations inside a finally block to prevent hangs.
- Use await(timeout) for resilient SLA enforcement in parallel fan-out architectures.

21

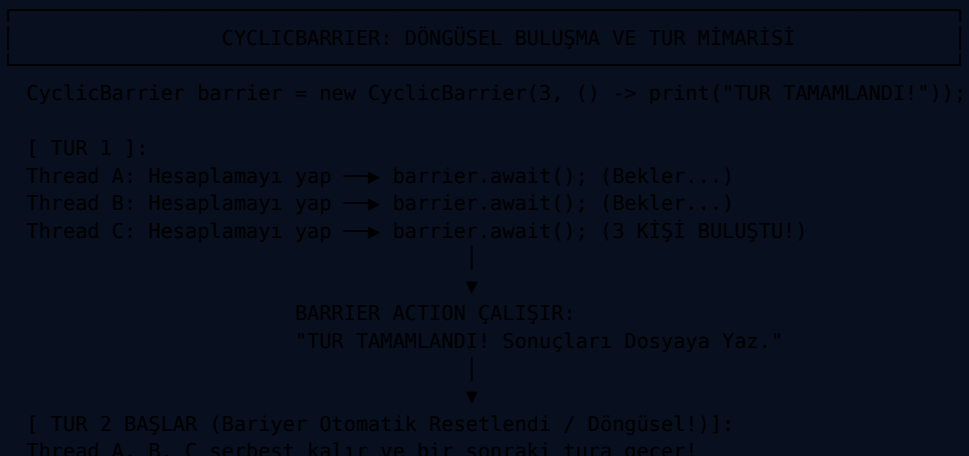
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

CyclicBarrier: Tekrar Kullanılabilir Buluşma Noktaları & Bariyer Eylemi

CyclicBarrier: Reusable Multi-Thread Rendezvous & Barrier Actions

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir lunaparktaki 4 kişilik gondol düşünün. Gondol 4 kişi dolmadan ASLA hareket etmez. İlk binen 3 kişi gondolun içinde oturup 4. kişiyi bekler (`await()`). 4. kişi bindiği anda kilitler kapanır ve gondol sallanır (Barrier Action). Tur bitince herkes iner ve gondol TEK-RAR boşalır; sonraki 4 kişi için baştan kullanılır (Cyclic / Döngüsel)! CountDownLatch gibi çöpe atılmaz!

🇬🇧 ENGLISH (ELI5)

Picture a 4-person whitewater rafting crew. The raft cannot depart until all 4 rowers take their seats. The first three climbers sit with paddles ready (`await()`). The instant the fourth rower steps aboard, the team pushes off down the rapids together (Barrier Action). At the bottom of the river, the raft is carried back up and re-used for the next round! Unlike single-use countdown latches, CyclicBarrier automatically resets for subsequent cycles.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CyclicBarrier ve CountDownLatch Farkları: 1) Tekrar Kullanılabilirlik (Cyclic): CyclicBarrier, tüm thread'ler buluştuğunda iç sayacını otomatik olarak ilk değerine geri sıfırlar. Matris çarpımları, genetik algoritmalar ve çok adımlı paralel simülasyonlar için biçilmiş kaftandır. 2) Barrier Action: Tüm thread'ler bariyere ulaştığında, içlerinden rastgele bir tanesi verilen `Runnable` görevini (ör. tur sonuçlarını diskte birleştirme) tek başına çalıştırır; bitince tüm thread'ler bir sonraki adıma serbest bırakılır. 3) Kırık Bariyer (BrokenBarrierException): Eğer bekleyen thread'lerden biri zaman aşımına uğrar veya `interrupt` edilirse, tüm bariyer kırılır (Broken state). Diğer tüm bekleyen thread'ler `BrokenBarrierException` alarak çöker; böylece eksik veriyle yanlış işlem yapılması donanımsal olarak engellenir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CyclicBarrier Mechanics & Failure Handling: 1) Cyclic Reset: CyclicBarrier resets its internal count automatically once the parties count reaches zero, advancing to a new 'Generation'. It is specifically tuned for iterative algorithms (e.g. parallel Jacobi relaxation or map-reduce matrix passes). 2) Barrier Action Execution: When the last thread trips the barrier, it synchronously executes the barrier action Runnable on its own stack before releasing the awaiting parties. 3) Broken Barriers: If any thread in the barrier is interrupted or times out, the barrier enters the 'Broken' state, unparking all other threads with a `BrokenBarrierException` to prevent deadlock and state corruption.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Çok Turlu Paralel Simülasyonda CyclicBarrier Kullanımı
import java.util.concurrent.CyclicBarrier;

public class ParallelMatrixSimulation {
    void main() {
        int workerCount = 3;

        // 3 işçi buluştuğunda çalışacak ortak bariyer eylemi
        var barrier = new CyclicBarrier(workerCount, () -> {
            IO.println("=== TUR BİTTİ: Ara sonuçlar birleştirildi. Yeni tura geçiliyor! ===\n");
        });

        for (int i = 1; i <= workerCount; i++) {
            final int id = i;
            Thread.ofPlatform().name("Worker-" + id).start(() -> {
                try {
                    for (int round = 1; round <= 2; round++) {
                        IO.println("İşçi " + id + " | Tur " + round + " hesaplıyor...");
                        Thread.sleep(100 * id); // Farklı hızlarda hesaplarlar

                        IO.println("İşçi " + id + " bariyere vardı, diğerlerini bekliyor...");
                        barrier.await(); // 3 kişi tamamlanana kadar bekle
                    }
                } catch (Exception e) {
                    Thread.currentThread().interrupt();
                }
            });
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'CountDownLatch ile CyclicBarrier arasındaki en temel iki fark nedir?' diye sorarsa: '1) CountDownLatch tek kullanımlıktır, CyclicBarrier sonsuz döngüde tekrar kullanılabilir (resetlenir). 2) CountDownLatch thread'lerin bir olayı beklemesini sağlar; CyclicBarrier ise thread'lerin birbirini beklemesini sağlar!' denmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Top MAANG Difference Question: 'What are the two foundational differences between CountDownLatch and CyclicBarrier?' Senior Answer: '1) CountDownLatch is single-use and cannot be reset; CyclicBarrier is cyclic and auto-resets per generation. 2) CountDownLatch waits for events/tasks to finish (threads don't wait for each other); CyclicBarrier forces concurrent threads to wait for each other at a mutual rendezvous.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CyclicBarrier her turda kendini sıfırlayarak sonsuz kez tekrar kullanılabilir.
- Barrier Action, son gelen thread tarafından tek başına yürütülür.
- Bir thread kesilirse bariyer Broken state'e geçer ve tüm thread'leri korur.

🎯 Key Takeaways for the Interview

- CyclicBarrier auto-resets its count, enabling iterative multi-phase processing.
- The barrier action is executed by the final arriving thread before unparking the pool.
- Thread interruption breaks the barrier, throwing BrokenBarrierException across all parties.

22

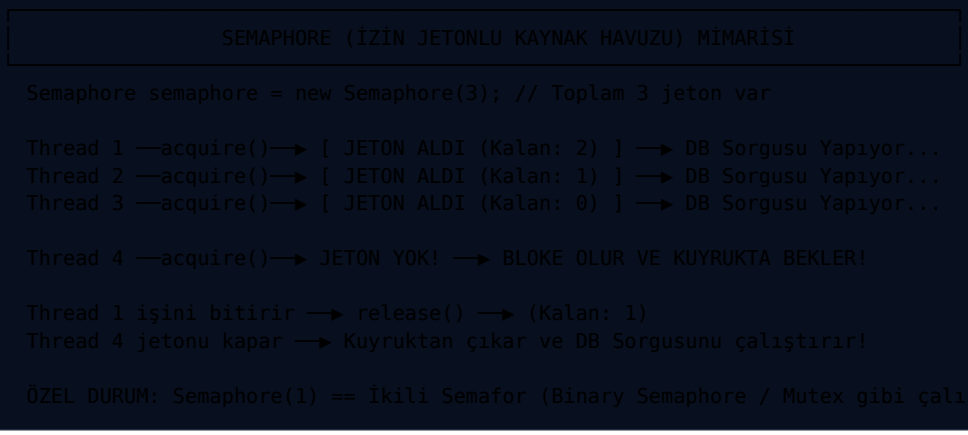
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Semaphore: İzin Tabanlı Kaynak Havuzları & Backpressure Sınırlama

Semaphore: Permitted Resource Pooling & Backpressure Throttling

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir gece kulübünün otoparkını düşünün. Otoparkta sadece 3 araçlık yer var (Semaphore permits = 3). İlk gelen 3 araba içeri girer ve güvenlik bariyeri kapanır. 4. araba kapıda bekler. İçerideki arabalardan biri çıkış yaptığı anda (`release()`), kapıdaki arabaya yeşil ışık yanar (`acquire()`) ve içeri girer. Semafor, pahalı bir kaynağı (veritabanı bağlantısı, harici API kotası) aynı anda aşırı yüklenmeden koruyan jetonlu güvenlik turnikesidir!

🇬🇧 ENGLISH (ELI5)

Think of a nightclub coat check with exactly 3 plastic hangers (`Semaphore(3)`). The first 3 patrons receive hangers and enter. The fourth patron waits in line at the counter. The moment an existing patron checks out and returns their hanger (`release()`), the clerk hands it to the waiting patron (`acquire()`). A Semaphore acts as a token-based rate-limiter, throttling concurrent access to bounded resources (database pools, third-party REST quotas) to prevent system brownouts.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Semafor Mimarisi ve Mutex Farkı: 1) AQS State = İzin Sayısı: Semaphore, AQS'in `state` alanını kalan izin (permit) sayısı olarak kullanır. 2) Semaphore vs Mutex (Önemli Mülakat Farkı!): Bir Mutex kilitte kilidi KİM ALDIYSA YALNIZCA O BIRAKABİLİR (Ownership prensibi). Ancak Semaphore'da SAHİPLİK YOKTUR! Thread A `acquire()` çağırıp izni alabilir; Thread B ise `release()` çağırarak o izni havuza geri verebilir! Bu özellik thread'ler arası sinyalleşmede kullanılır. 3) Backpressure ve Rate Limiting: Veritabanına aynı anda en fazla 20 sorgu gitmesini istiyorsanız, arkaya 1000 thread gelse bile `semaphore.tryAcquire(timeout)` ile veritabanı OOM ve çökmeden korunur.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Semaphore Internals & Mutex Distinction: 1) AQS Permit Representation: AQS `state` holds the available permit count. `acquire()` executes a CAS loop decrementing state; if state <= 0, the thread parks on the AQS shared sync queue. 2) Crucial Ownership Difference vs Mutex: A Mutex or `ReentrantLock` has strict ownership: only the thread that acquired the lock may release it. A Semaphore HAS NO CONCEPT OF OWNERSHIP. Thread 1 can invoke `acquire()`, and Thread 2 can invoke `release()`! 3) Dynamic Permit Inflation: Calling `release()` without preceding acquires actually INCREASES permits beyond initial capacity, a feature used in reactive backpressure signaling but a hazardous bug if done accidentally.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Semaphore ile Veritabanı Bağlantı Havuzu Throttling
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;

public class DatabaseConnectionThrottler {
    // Veritabanı maksimum 3 eşzamanlı bağlantıya izin veriyor
    private final Semaphore semaphore = new Semaphore(3, true); // Adil (Fair)

    public void executeQuery(String queryName) {
        try {
            // 1 saniye içinde izin alamazsa vazgeç (Fail-Fast)
            if (semaphore.tryAcquire(1, TimeUnit.SECONDS)) {
                try {
                    IO.println("Bağlantı alındı: " + queryName + " yürütülüyor...");
                    Thread.sleep(300); // DB işlemi simülasyonu
                } finally {
                    semaphore.release(); // İzni havuza geri ver
                    IO.println("Bağlantı bırakıldı: " + queryName);
                }
            } else {
                IO.println("⚠ BAĞLANTI HAVUZU DOLU! " + queryName + " reddedildi (Backpressure).");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Bir semafora fazladan release() çağrılırsa ne olur?' diye sorar. Yanıt: 'Hata fırlatmaz! İzin sayısı başlangıçtaki kapasitenin üzerine çıkar (Permit Inflation)! Örneğin 3 izinli semafora 5 kere release() derseniz artık 5 thread aynı anda içeri girebilir. Bu durum beklenmedik kaynak tükenmelerine yol açabilir.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Subtle Interview Trap: 'What happens if you invoke `semaphore.release()` more times than `acquire()`?' Answer: 'No exception is thrown! The permit count increases monotonically past the initial constructor capacity (Permit Inflation). Calling release() 5 times on a Semaphore(3) raises available permits to 5, potentially overwhelming downstream resources.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Semaphore, eşzamanlı erişilebilecek kaynak sayısını (permits) sınırlar.
- Semaforlarda sahiplik (ownership) yoktur; bir thread alıp başka thread bırakabilir.
- İkili Semafor (Binary Semaphore / permits=1), mutex alternatifi olarak kullanılabilir.

🎯 Key Takeaways for the Interview

- Semaphore limits the maximum number of concurrent threads accessing a resource.
- Semaphores have no thread ownership; Thread A can acquire and Thread B can release.
- A binary semaphore (permits=1) functions as a non-reentrant mutual exclusion lock.

23

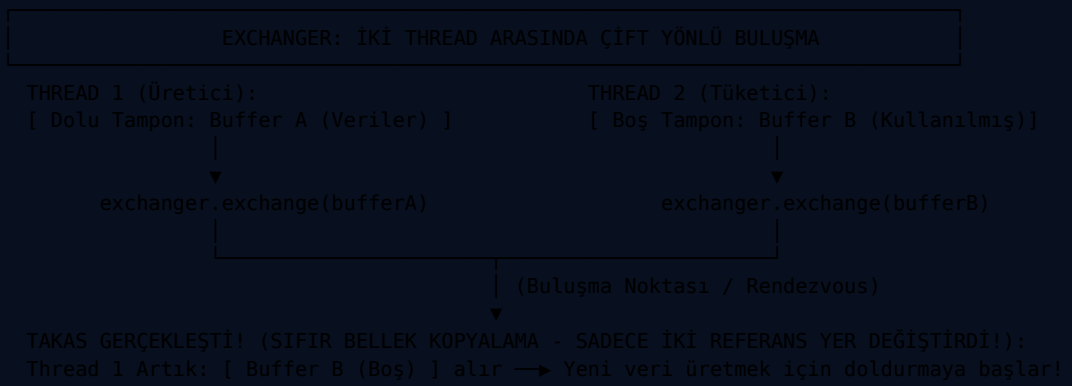
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Exchanger: İki Thread Arasında Çift Yönlü Sıfır Kopyalama Veri Takası

Exchanger: Pairwise Zero-Copy Buffer Swapping Between Producer and Consumer

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Casus filmlerindeki köprü sahnesini düşünün. Bir taraftan A ajanı elinde dolu para çantasıyla gelir, diğer taraftan B ajanı elinde boş çantayla gelir. Köprünün ortasında buluşurlar (`exchange()`). Biri gelmeden öbürü çantasını bırakmaz. Tam karşılaştıkları anda çantaları tokuştururlar! A ajanı boş çantayı alıp eve döner, B ajanı para dolu çantayı alıp gider. Bellekte tek bir bayt bile kopyalanmaz; sadece çantalar el değiştirir!

🇬🇧 ENGLISH (ELI5)

Picture a spy bridge exchange. Agent A arrives from the West holding a briefcase packed with cash. Agent B arrives from the East holding an empty briefcase. Neither departs until both arrive at the center span (`exchange()`). At that exact millisecond, they swap cases! Agent A walks away with the empty case to collect more funds; Agent B walks away with the cash. Zero data copying occurs; only two object references instantly swap ownership.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Exchanger Mekanizması ve Ring Buffer Optimizasyonu: 1) Rendezvous (Buluşma) Mantığı: `Exchanger\<V>` iki thread'in çift yönlü senkronize olmasını sağlar. İlk gelen thread, ikinci thread gelene kadar `LockSupport.park()` ile bekletilir. İkincisi geldiğinde referanslar atomik olarak takas edilir ve her iki thread uyandırılır. 2) Sıfır Çöp Üretimi (Zero-Allocation GC Friendly): Yüksek frekanslı finansal sistemlerde (HFT) sürekli `new byte[]` tahsis etmek GC pause yaratır. Exchanger ile önceden ayrılmış iki bellek tamponu (Double Buffering) sürekli el değiştirir; GC baskısı sıfıra iner! 3) Çoklu Çiftler (Arena Desteği): Birden fazla üretici ve tüketici varsa Exchanger içsel olarak bir Arena yapısı kullanarak thread çiftlerini farklı slotlarda eşleştirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Exchanger Architecture & Cache Locality: 1) Symmetric Rendezvous: Exchanger provides a bidirectional synchronization point. When Thread 1 calls `exchange(V)`, it registers its slot and parks. When Thread 2 arrives, it executes a CAS swapping references, and unparks Thread 1. 2) Eliminating Garbage Collection Churn: High-Frequency Trading (HFT) and telemetry engines avoid constant buffer re-allocations via Double-Buffering. Producer fills Buffer A; Consumer drains Buffer B. At the boundary, they swap references in sub-microsecond time with ZERO heap allocations. 3) Multi-Thread Arena Scaling: Modern JDK Exchangers implement an Arena of slots to eliminate contention when multiple thread pairs exchange concurrently.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Exchanger ile Çift Tamponlu (Double-Buffering) Log Yazıcı
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Exchanger;

public class DoubleBufferingLogger {
    void main() {
        var exchanger = new Exchanger<List<String>>();

        // Üretici Thread (Log toplayan)
        Thread.ofPlatform().name("LogProducer").start(() -> {
            List<String> currentBuffer = new ArrayList<>();
            try {
                for (int i = 1; i <= 5; i++) {
                    currentBuffer.add("Log Mesajı #" + i);
                    Thread.sleep(50);
                }
                IO.println("Üretici: Tampon doldu, takas bekleniyor...");
                currentBuffer = exchanger.exchange(currentBuffer); // Dolu ver, boş al!
                IO.println("Üretici: Boş tampon alındı, boyut: " + currentBuffer.size());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Tüketici Thread (Diske yazan)
        Thread.ofPlatform().name("LogConsumer").start(() -> {
            List<String> emptyBuffer = new ArrayList<>();
            try {
                IO.println("Tüketici: Veri gelene kadar bekleniyor...");
                List<String> fullBuffer = exchanger.exchange(emptyBuffer); // Boş ver, dolu al!
                IO.println("Tüketici: Dolu tampon alındı! Diske yazılıyor: " + fullBuffer);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kullanım Kuralı: Exchanger tam olarak İKİ thread arasındaki takaslar için tasarlanmıştır. Eğer 3 thread aynı Exchanger'ı kullanmaya kalkarsa, hangi iki thread'in eşleşeceği öngörülemez ve veri akış sırası bozulabilir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Design Limitation: Exchanger is explicitly optimized for pairwise (2-thread) cooperation. Utilizing an Exchanger across odd numbers of threads or uncoordinated pools yields non-deterministic pairing, corrupting ordering protocols.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Exchanger, iki thread arasında çift yönlü ve sıfır kopyalama ile veri takas eder.
- Double-Buffering deseninde GC baskısını sıfırlamak için idealdir.
- İlk gelen thread, ikinci thread gelene kadar park edilerek bekletilir.

🎯 Key Takeaways for the Interview

- Exchanger facilitates bidirectional, zero-copy object swapping between two threads.
- Ideal for Double-Buffering architectures to eliminate GC allocation churn.
- The first arriving thread parks until the second arrives, ensuring atomic handoff.

24

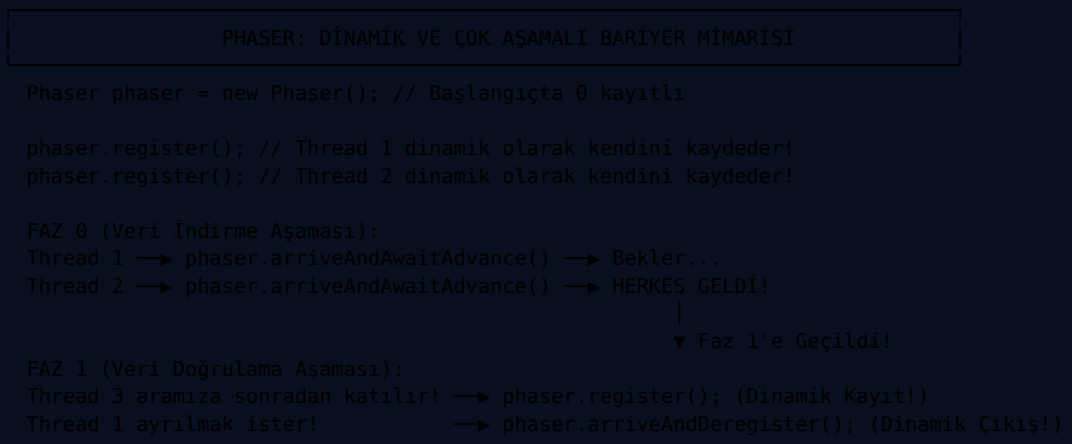
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Phaser: Dinamik Thread Kaydı & Çok Fazlı İleri Düzey Bariyer

Phaser: Dynamic Thread Registration & Multi-Phase Barrier Architecture

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir maraton düşünün. Ama bu maratonda yarışmacı sayısı baştan belli değildir! Koşu sırasında yeni koşucular yarışa dahil olabilir (`register()`), yorulan koşucular yarıştan çekilebilir (`arriveAndDeregister()`). Maraton 3 etaptan oluşur: 1. Etap (Faz 0), 2. Etap (Faz 1), 3. Etap (Faz 2). Her etabın kontrol noktasına gelen koşucu diğerlerini bekler; herkes geldiğinde hakem düdüğü çalar ve bir sonraki etap başlar! Phaser, katılımcı sayısı dinamik değişen çok etaplı bir yarış hakemidir!

🇬🇧 ENGLISH (ELI5)

Imagine a multi-stage endurance race where the participant roster is fluid. Mid-race, new runners can dynamically register (`register()`), and injured runners can withdraw (`arriveAndDeregister()`). The race spans discrete stages: Stage 0 (Mud Run), Stage 1 (Swim), Stage 2 (Sprint). At each checkpoint, runners wait until all registered participants arrive; the referee then advances the phase counter. Phaser is a dynamic, multi-phase CyclicBarrier supporting variable party counts.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Phaser vs CyclicBarrier / CountDownLatch: 1) Dinamik Katılımcı Sayısı (Dynamic Parties): CyclicBarrier'da katılımcı sayısı `new CyclicBarrier(N)` ile sabittir ve değiştirilemez. Phaser'da ise çalışma anında thread'ler `register()`, `bulkRegister(n)` veya `arriveAndDeregister()` ile serbestçe katılıp ayrılabilir. 2) Çok Fazlı Yapı (Phases): Phaser içsel olarak `getPhase()` ile artan bir faz sayacı tutar (0, 1, 2...). 3) Ağaç Yapılı Ölçekleme (Tiered Phasers): Yüzlerce thread'in tek bir senkronizöre saldırmasını önlemek için Phaser'lar hiyerarşik (Parent-Child) ağaçlar halinde kurulabilir; bu da devasa çok çekirdekli sunucularda çekişmeyi azaltır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Phaser Architecture & Scalability: 1) Dynamic Party Mutation: Unlike CyclicBarrier, party counts are mutable. Threads register on the fly (`register()`) and deregister upon early task completion (`arriveAndDeregister()`), preventing abandoned barriers. 2) Phase Counter Progression: Phaser tracks phase numbers monotonically via a 64-bit state field. `onAdvance(int phase, int registeredParties)` can be overridden to execute barrier logic or terminate the phaser. 3) Hierarchical Tree Phasers: To overcome memory bus contention across hundreds of cores, Phasers support hierarchical parent-child configurations, reducing synchronization contention via subtrees.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Phaser ile Dinamik Katılımlı Çok Fazlı İş Akışı
import java.util.concurrent.Phaser;

public class MultiPhaseTaskRunner {
    void main() {
        var phaser = new Phaser(1); // Main thread kendini kaydeder (Party: 1)

        for (int i = 1; i <= 2; i++) {
            phaser.register(); // Dinamik olarak yeni worker kaydet
            final int workerId = i;
            Thread.ofPlatform().name("Worker-" + workerId).start(() -> {
                IO.println("Worker " + workerId + " | Faz 0 (Veri İndirme) başladı.");
                phaser.arriveAndAwaitAdvance(); // Faz 0 tamamlandı, bekle!

                IO.println("Worker " + workerId + " | Faz 1 (Veri İşleme) başladı.");
                phaser.arriveAndAwaitAdvance(); // Faz 1 tamamlandı, bekle!

                phaser.arriveAndDeregister(); // İş bitti, kaydı sil ve ayrıl!
            });
        }

        IO.println("Main: Faz 0 bekleniyor...");
        phaser.arriveAndAwaitAdvance(); // Main thread Faz 0'ı onaylar

        IO.println("Main: Faz 1 bekleniyor...");
        phaser.arriveAndAwaitAdvance(); // Main thread Faz 1'i onaylar

        phaser.arriveAndDeregister(); // Main ayrılır
        IO.println("Tüm fazlar başarıyla tamamlandı!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Sorusu: 'CyclicBarrier varken neden Phaser kullanmalıyız?' Yanıt: '1) Thread sayısı dinamik değişiyorsa, 2) Görevini erken bitiren thread'lerin havuzdan düşmesi gerekiyorsa, 3) Hiyerarşik ağaç yapısıyla binlerce thread'e ölçeklenmek gerekiyorsa Phaser tercih edilir.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Comparative Analysis: 'Why select Phaser over CyclicBarrier?' Senior Answer: '1) Dynamic thread registration/deregistration at runtime. 2) Programmable termination conditions via `onAdvance()`. 3) Tiered hierarchical tree structures to mitigate cache coherency contention under extreme thread counts.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Phaser, çalışma anında dinamik thread kaydına (register/deregister) izin verir.
- getPhase() ile aşama aşama ilerleyen çok adımlı iş akışlarını koordine eder.
- Hiyerarşik (Parent-Child) Phaser yapıları binlerce çekirdeğe ölçeklenebilir.

🎯 Key Takeaways for the Interview

- Phaser supports dynamic registration and deregistration of parties at runtime.
- Tracks sequential workflow phases via an incrementing phase counter.
- Hierarchical phasers eliminate bus contention across massive core counts.

BÖLÜM 5 / MODULE 5

Thread Havuzları, Executor'lar & Work-Stealing

Thread Pools, Executors & Work-Stealing Algorithms

ThreadPoolExecutor iç yapısı, 4 ret politikası (Rejection Policies), Executors sınıfının üretim felaketleri (OOM riskleri), ScheduledThreadPool, Fork/Join work-stealing algoritması ve Graceful Shutdown.

ThreadPoolExecutor mechanics, the 4 task rejection policies, production OOM hazards of Executors factory methods, ScheduledExecutor delay drift, Fork/Join work-stealing, and graceful shutdown.

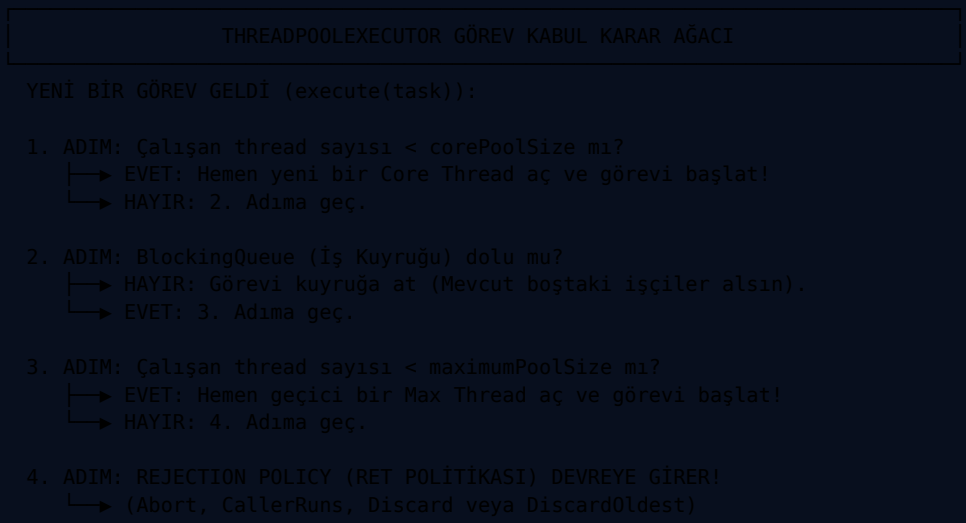
25

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

ThreadPoolExecutor Anatomisi: CorePool, MaxPool, Kuyruk & Büyüme Algoritması

ThreadPoolExecutor Anatomy: Core vs Max Pool, Queues & Growth Dynamics

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kargo şubesini düşünün. Şubede 3 daimi kadrolu personel (corePoolSize) çalışır. İlk 3 kargo gelince hemen bu personellere verilir. 4. kargo geldiğinde müdür yeni personel tutmaz; kargoyu arkadaki depoya (BlockingQueue) koyar! Ancak depo ağzına kadar dolup taştığında (Kuyruk dolduğunda), müdür geçici acil durum personelleri (maximumPoolSize) tutmaya başlar. Eğer depo da dolu, tüm geçici personeller de meşgulse; kapıya gelen müşteriye 'Kargonuzu alamıyoruz' denir (Rejection Policy)!

🇬🇧 ENGLISH (ELI5)

Think of a busy post office. It employs 3 full-time clerks (`corePoolSize`). The first 3 parcels go directly to them. When the 4th parcel arrives, the manager does NOT hire new staff; the parcel is placed into the backroom storage shelf (`BlockingQueue`). Only when that shelf is 100% physically packed full does the manager hire auxiliary temporary workers (`maximumPoolSize`). If temporary workers hit the legal ceiling AND the shelf is overflowing, incoming parcels are turned away at the door (`RejectedExecutionHandler`)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ThreadPoolExecutor Parametreleri ve HotSpot Yaşam Döngüsü: 1) corePoolSize vs maximumPoolSize: Birçok mühendis görevler geldikçe havuzun `maximumPoolSize`'a kadar büyüyeceğini sanır; bu yanlıştır! Standart mantıkta görev önce kuyruğa konur. Yalnızca kuyruk DOLDUĞUNDA `maximumPoolSize`'a kadar yeni thread açılır. 2) keepAliveTime ve Unit: Boşta kalan (idle) geçici thread'lerin ne kadar süre sonra sonlandırılacağını belirler (`allowCoreThreadTimeOut(true)` ile core thread'ler de sonlandırılabilir). 3) ctl Atomik Değişkeni: Doug Lea, havuz durumunu (RUNNING, SHUTDOWN, STOP, TIDYING, TERMINATED) ve aktif thread sayısını tek bir 32-bit `AtomicInteger ctl` içinde bit-masking ile tutar (Üst 3 bit durum, alt 29 bit thread sayısı).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ThreadPoolExecutor Architecture & Bit-Packing: 1) The Growth Order Axiom: Tasks route to: `corePool -> workQueue -> maxPool -> RejectedExecutionHandler`. Extra threads past `corePoolSize` are ONLY created if the queue rejects the `offer()`. 2) Atomic Control Word (`ctl`): HotSpot uses a single `AtomicInteger ctl` to pack both worker count and run state. The high 3 bits represent states (RUNNING = −1, SHUTDOWN = 0, STOP = 1, TIDYING = 2, TERMINATED = 3); the lower 29 bits encode the active worker count (supporting up to 536 million threads). 3) Worker Encapsulation: Internal `Worker` classes extend AQS to implement non-reentrant mutual exclusion locks, preventing interrupts during active task execution.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Üretim Kalitesinde ThreadPoolExecutor Konfigürasyonu
import java.util.concurrent.*;

public class ProductionThreadPoolFactory {
    public static ThreadPoolExecutor createCustomPool() {
        int corePoolSize = 4;
        int maxPoolSize = 8;
        long keepAliveTime = 60L;

        // ⚠ Sınırlı (Bounded) kuyruk OOM'u kesinlikle önler!
        BlockingQueue<Runnable> queue = new ArrayBlockingQueue<>(100);

        ThreadFactory threadFactory = Thread.ofPlatform()
            .name("order-processor-", 1)
            .factory();

        // 4. Adım: Kuyruk ve Max dolunca çağıran thread çalıştırsın (CallerRuns)
        RejectedExecutionHandler handler = new ThreadPoolExecutor.CallerRunsPolicy();

        return new ThreadPoolExecutor(
            corePoolSize,
            maxPoolSize,
            keepAliveTime,
            TimeUnit.SECONDS,
            queue,
            threadFactory,
            handler
        );
    }

    void main() {
        var pool = createCustomPool();
        pool.execute(() -> IO.println("Görev çalıştı: " + Thread.currentThread().getName()));
        pool.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı Sorusudur: 'corePoolSize=2, maxPoolSize=10, queueCapacity=100 olan bir havuza aynı anda 50 görev gönderirsek kaç thread açılır?' Yanıt: 'SADECE 2 THREAD AÇILIR! Çünkü ilk 2 görev core thread'leri alır, kalan 48 görev 100 kapasiteli kuyruğa atılır. Kuyruk dolmadığı için 3. thread bile açılmaz!' Bu soruyu bilen aday doğrudan öne geçer.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

The 1 MAANG Pool Question: 'If core=2, max=10, and queueCapacity=100, and 50 tasks arrive concurrently, how many threads exist?' Answer: 'EXACTLY 2 THREADS! The first 2 tasks spawn the core workers. The remaining 48 tasks are buffered in the 100-capacity queue. Because the queue never overflows, temporary workers are never spawned.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Havuz kuyruk dolmadan maximumPoolSize'a kadar ASLA genişlemez.
- ctl değişkeni tek bir 32-bit integer içinde hem havuz durumunu hem thread sayısını tutar.
- Üretimde sınırsız (unbounded) kuyruk kullanmak OOM çöküşlerine davetiyedir.

🎯 Key Takeaways for the Interview

- Pools never expand past corePoolSize until the bounded work queue is 100% full.
- HotSpot packs pool run state and thread count into a single 32-bit atomic integer.
- Unbounded work queues cause OutOfMemoryErrors under sustained traffic spikes.

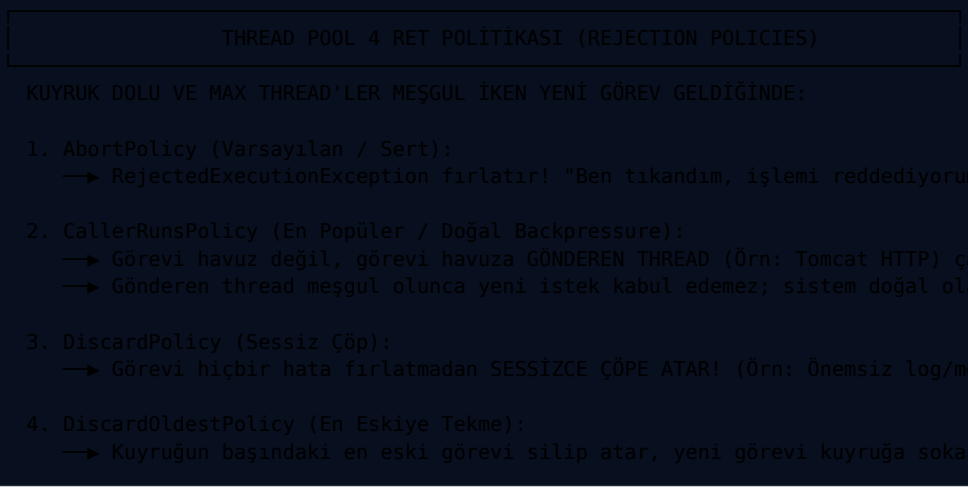
26

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Ret Politikaları (Rejection Policies): Abort, CallerRuns, Discard & DiscardOldest

Task Rejection Policies: AbortPolicy, CallerRuns, Discard & DiscardOldest

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir pizzacı dükkanı düşünün. Fırınlar dolu, masalar dolu, kuryeler meşgul. Kapıdan yeni bir müşteri girip pizza istediğinde pizzacı ne yapar? 1) Abort: Müşterinin yüzüne kapıyı çarpar: 'Sipariş alamam!' der (Hata fırlatır). 2) CallerRuns: Müşterinin eline önlük tutuşturur: 'Çok istiyorsan mutfağa gir kendin pişir!' der (İsteği getiren kişi kendi işini kendi yapar, böylece kapıda yeni müşteri birikmez). 3) Discard: Sipariş fişini alır ve müşteri arkasını döndüğü anda çöpe atar (Sessizce yutar). 4) DiscardOldest: Bekleyen en eski müşterinin siparişini iptal eder, yeni gelen müşteriyi sıraya alır!

🇬🇧 ENGLISH (ELI5)

Imagine a bakery operating at maximum capacity. The oven is full, counters overflowing. When a customer orders a cake: 1) AbortPolicy: The baker slams the shutter and shouts 'Denied!' (`RejectedExecutionException`). 2) CallerRunsPolicy: The baker hands the customer an apron: 'Bake it yourself!' The caller executes the task on their own thread, naturally throttling new orders. 3) DiscardPolicy: The baker silently drops the order slip into the trash (silent drop). 4) DiscardOldestPolicy: The baker evicts the oldest stale ticket waiting in line and enqueues the fresh order.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CallerRunsPolicy ile Doğal Backpressure (Geri Basınç): 1) Neden Üretimde CallerRuns Tercih Edilir?: Web sunucularında (Tomcat, Netty, Spring Boot) arka plandaki thread havuzu tıkandığında `CallerRunsPolicy` seçilirse, görevi havuza atan HTTP Acceptor thread'i görevi kendi üzerinde koşturmaya başlar. Bu süre zarfında soketten yeni HTTP istekleri kabul edilemez; işletim sistemi TCP SYN kuyruğunda bekletir veya yük dengeleyiciye (Load Balancer) 'Ben doluyum' sinyali gider. Bu, sistemin OOM çöküşünü önleyen en zarif mimari savunmadır. 2) DiscardOldest Tehlikesi: `PriorityBlockingQueue` ile kullanılırsa, en eski görev yerine en yüksek öncelikli görev çöpe atılabilir!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CallerRunsPolicy & The Elegance of Organic Backpressure: 1) Mitigating Cascading Failures: When a downstream worker pool saturates, `CallerRunsPolicy` delegates execution to the invoking thread (e.g. Tomcat/Jetty worker). Because the calling thread is now occupied executing computation, it stops accepting new requests from the network socket. The TCP receive buffer fills, and upstream load balancers detect backpressure, rerouting traffic away from the saturated node. 2) DiscardOldest Hazard: When paired with a `PriorityBlockingQueue`, `DiscardOldest` evicts the head of the queue, which is the HIGHEST priority task, not the oldest chronologically!

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: CallerRunsPolicy ile Doğal Backpressure Gösterimi
import java.util.concurrent.*;

public class RejectionPolicyDemo {
    void main() {
        // 1 Thread, 1 Kuyruk Kapasitesi (Toplam 2 görev alabilir)
        var executor = new ThreadPoolExecutor(
            1, 1, 0L, TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(1),
            new ThreadPoolExecutor.CallerRunsPolicy() // Backpressure kalkanı
        );

        for (int i = 1; i <= 4; i++) {
            final int taskId = i;
            executor.execute(() -> {
                IO.println("Görev #" + taskId + " çalıştırılıyor. Thread: " +
Thread.currentThread().getName());
                try { Thread.sleep(200); } catch (InterruptedException ignored) {}
            });
        }

        executor.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'CallerRunsPolicy kullandığımızda ana thread kilitlenebilir mi?' diye sorar. Yanıt: 'Evet! Eğer havuza atılan görev uzun süren bir işlemse veya bloke oluyorsa, ana thread (ör. UI thread veya Netty event-loop) o görevi çalıştırırken donar ve tüm uygulamanın tepki süresi (responsiveness) durur!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Architectural Hazard: 'Can CallerRunsPolicy degrade the application main loop?' Answer: 'YES! If the calling thread is a Netty EventLoop or a UI dispatcher, forcing it to run long-running CPU or blocking I/O tasks halts the entire event-loop, starving all other multiplexed channels.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• AbortPolicy varsayılandır ve RejectedExecutionException fırlatır.

• CallerRunsPolicy, çağıran thread'i çalıştırarak doğal bir backpressure kalkanı oluşturur.

• DiscardPolicy sessizce görevi yutar; yalnızca önemsiz telemetri loglarında kullanılmalıdır.

🎯 Key Takeaways for the Interview

• AbortPolicy is the default, throwing RejectedExecutionException on saturation.

• CallerRunsPolicy creates natural backpressure by forcing the submitting thread to execute the work.

• DiscardPolicy silently drops tasks and should only be applied to dispensable metrics.

27

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Executors Fabrika Tuzakları: newFixedThreadPool Neden Üretimde OOM Yaratır?

Executors Factory Traps: Why newFixedThreadPool Triggers Production OOM

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              EXECUTORS.NEWFIXEDTHREADPOOL() ÖLÜMCÜL KODU     │
└──────────────────────────────────────────────────────────────┘

Executors.newFixedThreadPool(10) ÇAĞRISININ ARKA PLANI:

return new ThreadPoolExecutor(
    10, 10, 0L, TimeUnit.MILLISECONDS,
    new LinkedBlockingQueue<Runnable>() // 💣 SAATLİ BOMBA BURADA!
);

LinkedBlockingQueue'nun Varsayılan Kapasitesi:
Integer.MAX_VALUE = 2,147,483,647 (2 MİLYAR GÖREV!)

GÖREVLER HAVUZA AKARKEN:
10 Core Thread kilitlenir veya yavaşlar ──► Gelen görevler kuyrukta birikir:
[Task 1, Task 2, Task 3, ... Task 1,000,000, ... Task 5,000,000]
                                        │
                                        ▼
HEAP BELLEK BİTER: java.lang.OutOfMemoryError: Java heap space ──► SİSTEM ÇÖKER!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir mağaza açıyorsunuz. İçeriye 10 tezgahar koyuyorsunuz (newFixedThreadPool(10)). Ama bekleme salonunu 'sonsuz' büyüklükte yapıyorsunuz (2 milyar kişilik kuyruk)! Tezgahlarda işler biraz yavaşladığında kapıdaki 5 milyon insan mağazanın koridorlarına doluşur. Sonunda binanın kolonları bu ağırlığı taşıyamaz ve bina herkesin üstüne çöker (OutOfMemoryError)! İşte `Executors.newFixedThreadPool` kullanmak, mağazanın kapısına sınırsız insan alıp binayı çökertmektir!

🇬🇧 ENGLISH (ELI5)

Imagine opening a bank branch with 10 tellers (`newFixedThreadPool(10)`). However, you build a waiting lobby capable of holding 2.14 billion humans (`LinkedBlockingQueue`). If the tellers suffer an external network delay, millions of customers flood into the building. Eventually, the floor collapses under the physical weight, demolishing the entire bank (`OutOfMemoryError`). Never configure an infinite lobby!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Executors Metotlarının MAANG Üretim Analizi: 1) newFixedThreadPool & newSingleThreadExecutor Felaketi: İkisi de unbounded `LinkedBlockingQueue` (kapasite: 2^31-1) kullanır. Tüketici hızı üretici hızından yavaş olduğunda görev nesneleri Heap'i doldurur ve JVM OOM ile çöker. 2) newCachedThreadPool Felaketi: `corePoolSize = 0`, `maximumPoolSize = Integer.MAX_VALUE` ve `SynchronousQueue` kullanır. Yoğun trafikte her gelen istek için yeni bir OS thread'i açmaya çalışır; binlerce thread açıldığında işletim sistemi seviyesinde `java.lang.OutOfMemoryError: unable to create new native thread` patlar. 3) MAANG Kuralı: `Executors` statik fabrikalarının kullanımı kurumsal kod tabanlarında kesinlikle yasaklanmalıdır (Checkstyle / SonarQube kuralı); daima parametreleri açıkça belirlenmiş `ThreadPoolExecutor` kurulmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Why Static Executors Factories are Banned at MAANG: 1) `newFixedThreadPool` Unbounded Queue Doom: It wraps an unbounded `LinkedBlockingQueue` (capacity `Integer.MAX_VALUE`). Under downstream latency spikes, tasks accumulate on heap memory until a catastrophic `OutOfMemoryError: Java heap space` kills the JVM process. 2) `newCachedThreadPool` Thread Explosion Doom: It sets `maximumPoolSize = Integer.MAX_VALUE` over a zero-capacity `SynchronousQueue`. In traffic surges, it spawns thousands of OS platform threads until memory or OS limits trip, crashing with `OutOfMemoryError: unable to create native thread`. 3) Staff+ Guideline: Enforce linter rules prohibiting `java.util.concurrent.Executors` factory methods. Always instantiate `ThreadPoolExecutor` with explicit bounded queues and custom rejection handlers.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Güvenli Bounded Havuz vs Tehlikeli Executors Karşılaştırması
import java.util.concurrent.*;

public class SafeVsUnsafePools {
    // ❌ TEHLİKELİ (Üretimde KESİNLİKLE kullanmayın!):
    public static ExecutorService createDangerousPool() {
        return Executors.newFixedThreadPool(10); // Unbounded kuyruk = OOM riski!
    }

    // ✔ GÜVENLİ (MAANG Standardı):
    public static ExecutorService createEnterpriseSafePool() {
        return new ThreadPoolExecutor(
            8,                          // Core pool size
            16,                         // Max pool size
            30L, TimeUnit.SECONDS,      // Idle timeout
            new ArrayBlockingQueue<>(5000), // ⚠ Sınırlı Kuyruk (Bounded Queue)
            Thread.ofPlatform().name("safe-worker-", 1).factory(),
            new ThreadPoolExecutor.CallerRunsPolicy() // Backpressure
        );
    }

    void main() {
        var safePool = createEnterpriseSafePool();
        safePool.execute(() -> IO.println("Güvenli kurumsal havuz aktif!"));
        safePool.shutdown();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Executors.newCachedThreadPool() ne zaman çöker?' Yanıt: 'Gelen istek hızı thread'lerin işi bitirme hızından yüksek olduğunda, havuz sonsuza kadar yeni OS thread'i açmaya çalışır. 10.000 thread açıldığında sistem RAM'i tükenir ve unable to create new native thread hatasıyla çöker!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Distinction: 'What is the failure mode of `newCachedThreadPool`?' Senior Answer: 'Unbounded thread creation! Because maximumPoolSize is Integer.MAX_VALUE, any traffic burst spawns thousands of native threads. Each consumes 1MB stack plus OS kernel metadata, crashing the JVM with `OutOfMemoryError: unable to create new native thread`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- newFixedThreadPool sınırsız LinkedBlockingQueue yüzünden Heap OOM yaratır.
- newCachedThreadPool sınırsız thread açtığı için Native Thread OOM patlatır.
- Üretimde DAİMA sınırlı kuyruklu (ArrayBlockingQueue) açık ThreadPoolExecutor kullanılır.

🎯 Key Takeaways for the Interview

- newFixedThreadPool uses an unbounded LinkedBlockingQueue, causing Heap OOM crashes.
- newCachedThreadPool spawns infinite threads, causing Native Thread OOM panics.
- In production, always construct explicit ThreadPoolExecutors with bounded queues.

28

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

ScheduledThreadPoolExecutor: Zamanlanmış Görevler, Drift & İptal Mekanizmaları

ScheduledThreadPoolExecutor: Timer Scheduling, Delay Drift & Task Cancellation

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic




```
1. scheduleAtFixedRate (Sabit Hızlı Frekans):
   Periyot: 5 saniye | Görev süresi: 2 saniye
   [ Görev 1 (2s) ] ──3s bekle──▶ [ Görev 2 (2s) ] ──3s bekle──▶ [ Görev 3 ]
   ⚠ Eğer görev 7 saniye sürerse? Sonraki görev GECİKMELİ olarak anında başlar!

2. scheduleWithFixedDelay (Sabit Aralık Molası):
   Gecikme: 5 saniye | Görev süresi: 2 saniye
   [ Görev 1 (2s) ] ──TAM 5 SANİYE BEKLE──▶ [ Görev 2 (2s) ] ──TAM 5s BEKLE──▶
   İş ne kadar sürerse sürsün, iki işin bitişi ile başlangıcı arası DAİMA 5 saniyedir!

İÇ MİMARİ: `DelayedWorkQueue` (Min-Heap / Öncelik Kuyruğu)
En yakın sürede çalışacak görev kuyruğun tepesindedir (O(log N) ekleme).
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki tip ilaç saati gibi düşünün: 1) FixedRate (Sabit Saat): Doktor 'Her sabah saat tam 08:00'de ilaç iç' dedi. Kahvaltıyı uzatsanız bile saat 08:00 olunca ilacı almak zorundasınız. 2) FixedDelay (Sabit Mola): Doktor 'Yemeğin bittikten TAM 2 saat sonra ilacını iç' dedi. Kahvaltı ister 10 dakika ister 1 saat sürsün; kahvaltı bittiği andan itibaren kronometre 2 saat saymaya başlar! Eğer bir işin önceki iş bittikten sonra belirli bir süre dinlenmesini istiyorsanız `scheduleWithFixedDelay` seçmelisiniz!

🇬🇧 ENGLISH (ELI5)

Compare two medicine schedules: 1) `scheduleAtFixedRate`: 'Take medicine at 08:00, 12:00, 16:00'. If lunch runs late, the schedule still demands delivery at the clock strike. 2) `scheduleWithFixedDelay`: 'Take medicine exactly 4 hours AFTER finishing your previous meal'. The timer starts only when the prior task completes. For periodic database synchronization or cleanup jobs, `scheduleWithFixedDelay` prevents overlapping runs if a task takes longer than expected!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ScheduledThreadPoolExecutor İç Mimarisi ve İstisna Tuzağı: 1) DelayedWorkQueue (Min-Heap): Görevler `ScheduledFutureTask` nesnelerine sarılır. Kuyruk, çalışma zamanına göre sıralanmış bir ikili yığın (Binary Min-Heap) tutar. 2) Sessiz Ölüm Tuzağı (Silent Death): Periyodik çalışan bir görev (`Runnable`) içinde yakalanmamış bir `RuntimeException` fırlarsa, `ScheduledExecutorService` o görevi SESSİZCE İPTAL EDER ve bir daha asla çalıştırmaz! Konsola hiçbir hata basılmaz! Görev arka planda ölmüştür ve kimsenin haberi olmaz. 3) Çözüm: Periyodik görevlerin `run()` metodu en dışta `try-catch(Throwable t)` ile sarılmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ScheduledExecutor Deep-Dive & The Silent Death Hazard: 1) DelayedWorkQueue Topology: Backed by a concurrent Fibonacci/Min-Heap, ordering scheduled tasks by trigger epoch (`getDelay(TimeUnit.NANOSECONDS)`). 2) FixedRate vs FixedDelay Under Overrun: If a task under `scheduleAtFixedRate` exceeds the interval, subsequent executions do not overlap (single-thread constraint per task instance), but they stack up without pause, causing high CPU storms. `scheduleWithFixedDelay` strictly guarantees `delay` elapsed time between execution completion and next launch. 3) ⚠️ The Fatal Silent Death Trap: If an unhandled `Throwable` escapes a scheduled Runnable, the executor halts all future executions of that task permanently, suppressing the stack trace! Always envelop scheduled loops with `try-catch (Throwable t)`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Güvenli Periyodik Görev Zamanlayıcı
import java.util.concurrent.*;

public class ResilientScheduler {
    void main() {
        ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

        // Güvenli Periyodik Görev (FixedDelay)
        scheduler.scheduleWithFixedDelay(() -> {
            try {
                IO.println("Cache temizliği başladı: " + System.currentTimeMillis());
                Thread.sleep(100);
            } catch (Throwable t) {
                // ⚠ KESİNLİKLE THROWABLE YAKALANMALIDIR! Aksi halde scheduler görevi öldürür!
                IO.println("Hata yakalandı, görev ölmedi: " + t.getMessage());
            }
        }, 0, 1, TimeUnit.SECONDS);

        // 3 saniye sonra durdur
        scheduler.schedule(() -> {
            IO.println("Scheduler kapatılıyor...");
            scheduler.shutdown();
        }, 3, TimeUnit.SECONDS);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Üretim Kabusu: 'ScheduledExecutorService'e attığım cron görevi 3 gün çalıştıktan sonra durdu, hata da vermedi, neden?' Yanıt: 'Görev içinde bir NullPointerException fırlamıştır ve catch edilmemiştir. Scheduler görevi kalıcı olarak iptal etmiştir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Real-World Production Horror: 'Our periodic cache refresher ran for 4 days then mysteriously vanished without crashing the JVM. Why?' Answer: 'An uncaught NullPointerException escaped the Runnable. ScheduledExecutorService silently terminated future iterations without logging to stdout.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- scheduleAtFixedRate sabit frekansla çalışır; scheduleWithFixedDelay iş bittikten sonra bekler.
- Yakalanamayan RuntimeException periyodik görevleri SESSİZCE ÖLDÜRÜR.
- Periyodik görevlerin gövdesi daima try-catch(Throwable) ile sarılmalıdır.

🎯 Key Takeaways for the Interview

- scheduleAtFixedRate targets fixed intervals; scheduleWithFixedDelay enforces delay between runs.
- Uncaught exceptions silently cancel future executions of scheduled tasks permanently.
- Always wrap scheduled task bodies inside a defensive try-catch(Throwable) block.

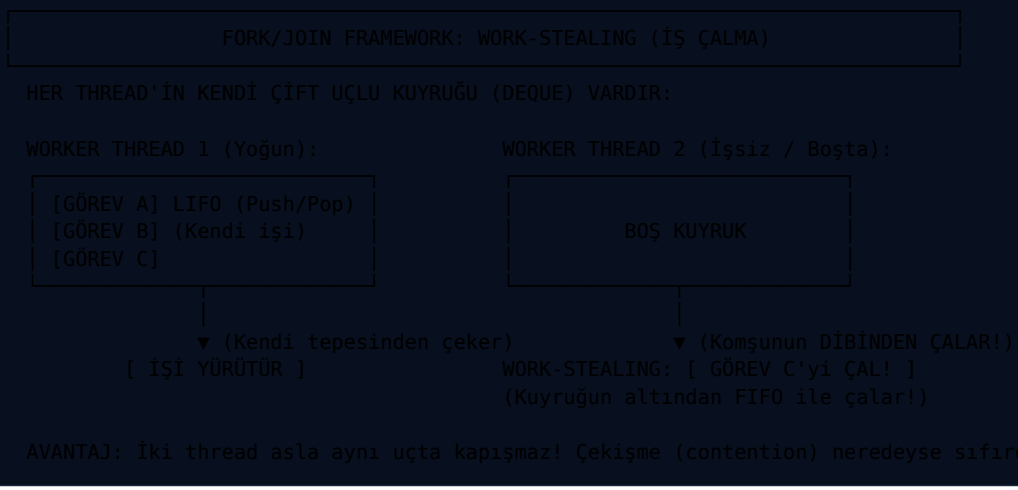
29

BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Fork/Join Çerçevesi & Work-Stealing Algoritması: İş Çalma & Çift Uçlu Kuyruklar

Fork/Join Framework & The Work-Stealing Algorithm: Deque Poaching & Recursion

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki işçi düşünün: Ali ve Veli. İkisinin de önünde kendi özel iş tepsisi var. Ali'nin tepsisinde 100 tane evrak yığılı, Veli ise kendi işlerini hemen bitirip boşta kalmış. Veli oturup tembellik yapmaz; hemen Ali'nin tepsisine uzanır! Ama Ali tepsisinin tepesinden en taze evrakı alırken, Veli tepsisinin en altındaki en eski evrakı çalar (Work-Stealing). İkisi tepsisinin farklı uçlarından tuttukları için birbirlerinin eline çarpmazlar! Böylece hiçbir CPU çekirdeği boş kalmaz!

🇬🇧 ENGLISH (ELI5)

Imagine two postal sorters: Alice and Bob. Each owns a private Double-Ended Queue (Deque). Alice has 100 letters stacked; Bob clears his pile and sits idle. Under the Work-Stealing algorithm, Bob does not stay idle. He peeks at Alice's tray! Alice pushes and pops small subtasks from the TOP of her deque (LIFO, cache-friendly). Bob steals large root tasks from the BOTTOM of her deque (FIFO). Because they operate on opposite ends, thread contention is mathematically minimized!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ForkJoinPool ve Work-Stealing Derinliği: 1) Doug Lea'nın Work-Stealing Algoritması: Her worker thread kendi `WorkQueue` (çift uçlu kuyruk - Deque) yapısına sahiptir. Kendi görevlerini LIFO (Last-In-First-Out) sırasıyla işler; bu sayede CPU L1 önbellek uyumluluğu (Cache Locality) maksimize edilir. 2) Hırsızlık (Stealing): Boşta kalan thread başka bir rastgele worker'ın kuyruk dibinden FIFO ile çalar; en büyük iş parçacığı genellikle dipte yattığı için çalınan iş hırsızı uzun süre meşgul eder. 3) Java Paralel Akışlar (Parallel Streams): `list.parallelStream()` arka planda ortak `ForkJoinPool.commonPool()`'u kullanır! Eğer paralel stream içinde bloklayıcı bir I/O (ör. HTTP isteği veya DB sorgusu) yaparsanız, ortak havuzdaki tüm çekirdekleri kilitler ve tüm JVM'i çökertirsiniz!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ForkJoinPool Mechanics & The Parallel Streams Hazard: 1) Dual-Ended Contention Isolation: Workers interact with their local array deque via LIFO push/pop, minimizing synchronization and maximizing CPU cache locality. Idle workers steal from victim deques via FIFO CAS operations on the opposite tail end. Large coarse-grained un-split tasks naturally reside at the tail, giving thieves substantial work chunks. 2) RecursiveTask vs RecursiveAction: Subclasses override `compute()`. `fork()` pushes asynchronously; `join()` executes cooperative work while waiting rather than blocking. 3) The `ForkJoinPool.commonPool()` Anti-Pattern: Java Parallel Streams execute on a shared global common pool sized to CPU cores. Executing blocking I/O inside parallel streams starves the common pool, deadlocking unrelated library components across the JVM.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Fork/Join ile Büyük Dizi Toplamı (Böl ve Yönet / Divide and Conquer)
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

public class ParallelSumTask extends RecursiveTask<Long> {
    private static final int THRESHOLD = 10_000; // Eşik değer
    private final long[] array;
    private final int start, end;

    public ParallelSumTask(long[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Long compute() {
        if (end - start <= THRESHOLD) {
            long sum = 0;
            for (int i = start; i < end; i++) sum += array[i];
            return sum;
        }

        int mid = start + (end - start) / 2;
        var leftTask = new ParallelSumTask(array, start, mid);
        var rightTask = new ParallelSumTask(array, mid, end);

        leftTask.fork(); // Sol parçayı başka bir thread'e çalması için bırak
        long rightResult = rightTask.compute(); // Sağ parçayı kendin hesapla
        long leftResult = leftTask.join();     // Sol parçayı bekle ve birleştir

        return leftResult + rightResult;
    }

    void main() {
        long[] data = new long[100_000];
        for (int i = 0; i < data.length; i++) data[i] = 1;

        try (var pool = new ForkJoinPool()) {
            long total = pool.invoke(new ParallelSumTask(data, 0, data.length));
            IO.println("Paralel Toplam Sonucu: " + total); // 100,000
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

MAANG Mülakat Kuralı: 'parallelStream() içine HTTP isteği veya veritabanı sorgusu yazabilir miyiz?' Yanıt: 'KESİNLİKLE HAYIR! parallelStream() tüm JVM'de paylaşılan ForkJoinPool.commonPool()'u kullanır. Birkaç yavaş I/O çağrısı havuzdaki tüm çekirdekleri tüketir ve sistemdeki diğer tüm paralel akışları felç eder!'

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Fatal Architecture Sin: 'Can we run HTTP calls or database queries inside parallelStream()?' Answer: 'NEVER! Parallel streams default to the shared ForkJoinPool.commonPool(). Blocking calls tie up common pool carrier threads, causing system-wide thread starvation across the entire JVM.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Fork/Join çerçevesi işlemci yoğun (CPU-bound) böl-ve-yönet algoritmaları içindir.

• Work-stealing, boşta kalan işçilerin diğer kuyrukların dibinden iş çalmasıdır.

• Bloklayıcı I/O işlemleri asla parallelStream() ve commonPool içine konulmamalıdır.

🎯 Key Takeaways for the Interview

• ForkJoinPool is optimized strictly for CPU-bound recursive divide-and-conquer workloads.

• Work-stealing allows idle workers to steal tasks from the tail of busy workers' deques.

• Never execute blocking I/O inside parallel streams or the shared common pool.

30

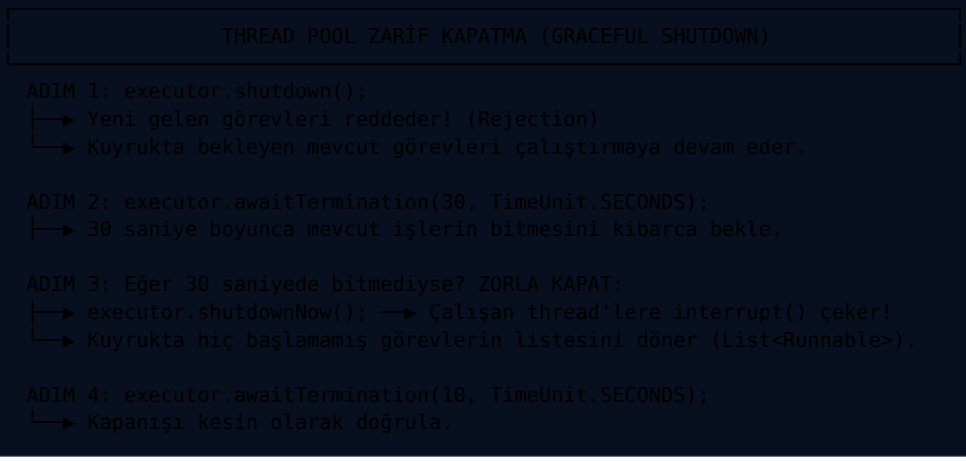
BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Zarif Kapatma (Graceful Shutdown): shutdown, shutdownNow & Zehirli Hap (Poison Pill)

Graceful Shutdown Patterns: shutdown(), shutdownNow() & The Poison Pill Pattern

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranın kapanış saati gibidir: 1. Adım (`shutdown()`): Kapı kilitlenir, yeni müşteri alınmaz. Ama masada yemeğini yiyen müşteriler kovulmaz; yemeklerini bitirmeleri beklenir. 2. Adım (`awaitTermination()`): Garsonlar masaların kalkmasını 30 dakika bekler. 3. Adım (`shutdownNow()`): 30 dakika geçmesine rağmen kalkmayan inatçı müşteri olursa ışıklar söndürülür, müzik kapatılır ve güvenlik herkesi dışarı çıkarır (Thread Interrupt)! İşte zarif kapatma, veri kaybı yaşamadan sistemi güvenle kapatma sanatıdır!

🇬🇧 ENGLISH (ELI5)

Shutting down a thread pool is like closing a restaurant: Step 1 (`shutdown()`): Lock the front door to new customers. Do not evict seated patrons eating dinner; let them finish. Step 2 (`awaitTermination()`): Wait patiently for 30 minutes for tables to clear. Step 3 (`shutdownNow()`): If stubborn diners refuse to leave after 30 minutes, cut the power, turn off the music, and usher them out via security (`Thread.interrupt()`). Graceful shutdown guarantees zero data loss.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üretim Seviyesinde 2-Aşamalı Kapatma Deseni (Two-Phase Termination): 1) `shutdown()` vs `shutdownNow()`: `shutdown()` havuzun durumunu SHUTDOWN yapar; yeni iş almaz ama kuyruktaki işleri tamamlar. `shutdownNow()` havuzu STOP durumuna çeker, kuyruktaki bekleyen görevleri çalıştırmadan bir `List\<Runnable>` olarak geri döner ve çalışan işçilere `interrupt()` sinyali gönderir. 2) Poison Pill Deseni: Kuyruk tabanlı Producer-Consumer sistemlerinde özel bir 'Zehirli Nesne' (ör. `POISON_PILL = new Task(-1)`) kuyruğa atılır. Tüketici bu nesneyi gördüğü anda döngüden çıkarak kendini sonlandırır; böylece `interrupt()` zorlamasına gerek kalmadan tamamen zarif bir kapanış sağlanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Oracle Standard 2-Phase Termination Protocol: 1) State Transitions: `shutdown()` transitions state from RUNNING to SHUTDOWN, refusing incoming tasks while executing buffered queue items. `shutdownNow()` transitions state directly to STOP, actively calling `Thread.interrupt()` on workers and draining unstarted tasks into a `List\<Runnable>` return payload. 2) The Poison Pill Architecture: In high-throughput messaging pipelines, a designated Sentinel Object (the Poison Pill) is injected into the queue. Consumers break their processing loops upon encountering the pill, achieving deterministic, exception-free termination without relying on thread interrupts.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Resilient Two-Phase Graceful Shutdown Deseni
import java.util.concurrent.*;

public class GracefulShutdownManager {
    public static void shutdownGracefully(ExecutorService pool, long timeoutSeconds) {
        pool.shutdown(); // 1. Yeni iş kabulünü durdur
        try {
            // 2. Mevcut işlerin bitmesini bekle
            if (!pool.awaitTermination(timeoutSeconds, TimeUnit.SECONDS)) {
                IO.println("⚠ İşler zamanında bitmedi, zorla kapatılıyor (shutdownNow)...");
                var droppedTasks = pool.shutdownNow(); // Çalışanlara interrupt çek
                IO.println("İptal edilen kuyruk görev sayısı: " + droppedTasks.size());

                if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
                    IO.println("❌ Havuz tamamen kapatılamadı!");
                }
            } else {
                IO.println("✔ Havuz tüm görevleri başarıyla tamamlayıp kapandı.");
            }
        } catch (InterruptedException e) {
            pool.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }

    void main() {
        var pool = Executors.newFixedThreadPool(2);
        pool.execute(() -> {
            try { Thread.sleep(500); } catch (InterruptedException ignored) {}
        });
        shutdownGracefully(pool, 2);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'shutdownNow() çağrıldığında çalışan thread anında pat diye durur mu?' Yanıt: 'HAYIR! Java'da thread'leri dışarıdan zorla öldüren bir mekanizma yoktur (Thread.stop() deprecated ve tehlikelidir). shutdownNow() yalnızca thread'e interrupt() sinyali gönderir; eğer çalışan kod InterruptedException yakalamıyor veya Thread.interrupted() kontrol etmiyorsa çalışmaya devam eder!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Java Truth: 'Does shutdownNow() kill running threads instantly?' Senior Answer: 'NO! Java has no safe pre-emptive thread-killing primitive (`Thread.stop()` is deprecated and dangerous). `shutdownNow()` merely issues a cooperative `interrupt()` signal. If the executing code does not poll `Thread.interrupted()` or sleep, it runs forever!'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- shutdown() kuyruktaki işleri bitirir; shutdownNow() çalışanlara interrupt çeker.
- Standart 2-aşamalı kapatma deseni: shutdown() -> awaitTermination() -> shutdownNow().
- Java thread'leri zorla öldürülemez; cooperative interruption mantığına uymak zorundadır.

🎯 Key Takeaways for the Interview

- shutdown() drains the queue; shutdownNow() interrupts active workers and returns unexecuted tasks.
- Always apply the 2-phase termination protocol: shutdown() -> await -> shutdownNow().
- Threads cannot be forcefully killed; they must cooperatively handle interruption.

BÖLÜM 6 / MODULE 6

Eşzamanlı Koleksiyonlar & Thread-Safe Veri Yapıları

Concurrent Collections & Thread-Safe Data Structures

ConcurrentHashMap iç mekaniği (CAS ve Kova Kilitleme), CopyOnWriteArrayList anlık görüntü kopyalaması, BlockingQueue varyantları, SynchronousQueue sıfır kapasite takası ve Lock-Free kuyruklar.

ConcurrentHashMap internals (CAS and Bin-Level locking), CopyOnWriteArrayList snapshot iteration, BlockingQueue variants, SynchronousQueue zero-capacity handoffs, and lock-free queues.

31

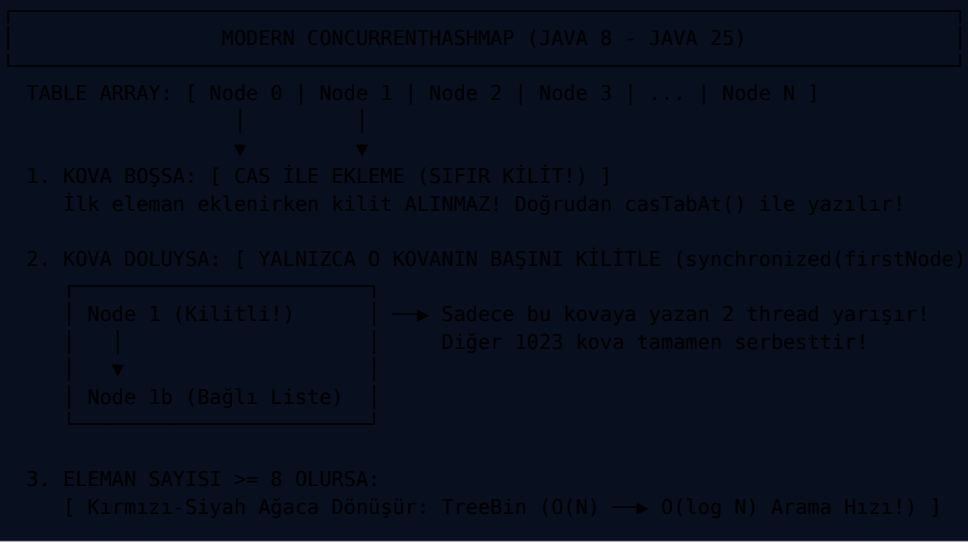
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

ConcurrentHashMap İç Mimarisi: Segment Yok! Node CAS, Bin Kilitleri & TreeBin

ConcurrentHashMap Internals: Node CAS, Bin-Level Locking & TreeBins

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski usul senkronize haritalar (Hashtable), 1000 çekmeceli bir kütüphanenin kapısına tek bir kilit takmak gibidir. Biri 1. çekmeceden kitap alırken diğer 999 kişi kapıda bekler. ConcurrentHashMap ise her çekmecenin başına küçük bir şifreli kilit takar! Biri 1. çekmeceyi açarken, diğeri 500. çekmeceden rahatça kitabını alır; birbirlerini hiç görmezler. Eğer bir çekmecede çok fazla dosya birikirse, çekmece otomatik olarak açılıp alfabetik bir fihrist ağacına (TreeBin) dönüşür!

🇬🇧 ENGLISH (ELI5)

A synchronized map (`Hashtable`) locks the entire bank vault whenever someone accesses safety deposit box 1 ConcurrentHashMap locks ONLY safety deposit box 1 (`synchronized(firstNode)`). Customers accessing boxes 2 through 1024 proceed concurrently with ZERO friction! Even better: if a box is empty, it inserts the item using atomic lock-free CAS without locking at all! If a single bucket accumulates more than 8 colliding items, it dynamically transmutes into a Red-Black Tree (`TreeBin`), preserving O(log N) search speed.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 8+ ve Java 25 ConcurrentHashMap İnovasyonları: 1) Segmentlerin Kaldırılması: Java 7'deki 16'lı `Segment[]` mimarisi çöpe atıldı. Artık doğrudan `Node\<K,V>[] table` dizisi kullanılır. 2) CAS ile İlk Ekleme: Kova (Bucket) boşsa, `Unsafe.compareAndSetObject` (modern JDK'da VarHandle) kullanılarak kilit almadan tek çevrimde yazılır. 3) Kova Düzeyinde Kilitleme: Kova doluysa sadece o kovanın ilk elemanı (`synchronized(node)`) kilitlenir; haritanın geneli açık kalır. 4) Treeify Eşiği: Tek bir kovadaki eleman sayısı 8'i geçerse ve tablo boyutu >= 64 ise bağlı liste kırmızı-siyah ağaca (`TreeBin`) dönüştürülür. Eleman sayısı 6'ya düşerse tekrar listeye (Untreeify) döner. 5) size() Hesabı: LongAdder mantığında `CounterCell[]` kullanılarak kilit almadan hesaplanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot ConcurrentHashMap Topology: 1) Demise of Segmented Locking: Java 8 discarded the historical 16-Segment structure. The map operates directly on a volatile `Node\<K,V>[] table`. 2) CAS-Driven Empty Bucket Ingestion: If `tabAt(i) == null`, the entry is inserted via `casTabAt()` with zero lock acquisition. 3) Bin-Level Synchronized Locks: When collisions occur, the thread synchronizes strictly on the root `Node` of that specific hash bin. Concurrent threads hashing to distinct bins execute in parallel. 4) TreeBin Transformation: If bin collisions reach `TREEIFY_THRESHOLD = 8` and table capacity >= 64, the singly-linked list transmutes into a balanced Red-Black Tree (`TreeNode`), bounding worst-case lookup from $O(N)$ to O(log N). 5) High-Concurrency Size Aggregation: `mappingCount()` leverages internal `CounterCell` arrays, mirroring `LongAdder` to prevent CAS contention.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ConcurrentHashMap ile Atomik computeIfAbsent ve İstatistikler
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.LongAdder;

public class HighThroughputUrlTracker {
    // URL bazlı istek sayaçları
    private final ConcurrentHashMap<String, LongAdder> accessCounters = new ConcurrentHashMap<>();

    public void trackVisit(String url) {
        // ⚠ ATOMİK: URL yoksa LongAdder oluştur, varsa eskisini getir (KİLİTSİZ!)
        accessCounters.computeIfAbsent(url, _ -> new LongAdder()).increment();
    }

    public long getVisitCount(String url) {
        var adder = accessCounters.get(url);
        return adder != null ? adder.sum() : 0L;
    }

    void main() {
        var tracker = new HighThroughputUrlTracker();
        tracker.trackVisit("https://maang.com/careers");
        tracker.trackVisit("https://maang.com/careers");
        IO.println("Sayfa Ziyaret Sayısı: " + tracker.getVisitCount("https://maang.com/careers"));
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı CHM Tuzağı: 'ConcurrentHashMap içine `null` key veya `null` value koyabilir miyiz?' Yanıt: 'KESİNLİKLE HAYIR! HashMap null kabul eder ama ConcurrentHashMap null key veya null value'ya ASLA izin vermez (NPE fırlatır). Çünkü çoklu thread ortamında map.get(k) == null döndüğünde 'Anahtar gerçekten yok mu, yoksa değeri mi null?' belirsizliği (ambiguity) kilit-siz çözülemez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory Interview Axiom: 'Can ConcurrentHashMap store null keys or null values?' Senior Answer: 'NEVER! `HashMap` permits nulls, but `ConcurrentHashMap` explicitly forbids null keys and null values, throwing `NullPointerException`. Doug Lea designed this because in a multithreaded map, `map.get(key) == null` cannot disambiguate whether the key is missing or the value is legitimately null without holding a lock across `containsKey()`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ConcurrentHashMap boş kovalara kilit almadan CAS ile eleman ekler.
- Kova dolduğunda tüm tablo değil, sadece o kovanın ilk elemanı kilitlenir.
- ConcurrentHashMap null key ve null value kabul etmez (NullPointerException).

🎯 Key Takeaways for the Interview

- ConcurrentHashMap inserts into empty bins via lock-free CAS operations.
- Only the root node of the target hash bin is synchronized during collisions.
- ConcurrentHashMap strictly forbids null keys and values to prevent race ambiguities.

32

BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

CopyOnWriteArrayList: Anlık Görüntü (Snapshot) İterasyonu & Yazma Cezası

CopyOnWriteArrayList: Snapshot Iteration, Write Penalties & When to Use It

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              COPY-ON-WRITE (COW) DİZİ KLONLAMA MEKANİZMASI              │
└──────────────────────────────────────────────────────────┘

 MEVCUT DİZİ (İç Array): [ "A", "B", "C" ] ◄── Binlerce Okuyucu Thread
                                               (SIFIR KİLİT! ULTRA HIZLI!)

 BİR YAZICI THREAD GELDİ (add("D")):
 1. ReentrantLock AL
 2. Dizinin YEPYENİ BİR KOPYASINI oluştur: new Object[eski.length + 1]
 3. Elemanları kopyala ve yeni elemanı ekle: [ "A", "B", "C", "D" ]
 4. İçsel volatile referansı yeni diziye bağla: array = newArray;
 5. ReentrantLock BIRAK

 SONUÇ: Okuyucular gezinirken (Iterator) ASLA ConcurrentModificationException ALMAZ!
 ANCAK sık yazma yapılırsa devasa bellek kopyalama maliyeti oluşur!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirketin telefon rehberini düşünün. 1000 çalışan gün boyunca rehbere bakıp numaraları arar (Okuyucular). Kimsenin birbirini beklemesine gerek yoktur. Ama şirkete yeni bir personel girdiğinde, sekreter mevcut rehberin üzerine karalama yapmaz! Masasındaki fotokopi makinesinden rehberin koca bir kopyasını çıkarır, yeni personeli o yeni kağıda yazar ve panodaki eski kağıdı yenisiyle değiştirir! Eski kağıdı okumakta olanlar rahatsız olmaz. Ama her saniye yeni personel giriyorsa, fotokopi makinesi yanar!

🇬🇧 ENGLISH (ELI5)

Think of a published corporate phone directory. Hundreds of employees browse it simultaneously without blocking (Readers). When an employee joins the company, the receptionist does not scribble on the active sheet. Instead, the receptionist photocopies the entire directory, appends the new name to the duplicate, and magnetically snaps the new copy onto the board! Readers actively holding the old sheet finish unperturbed. But if you have 10,000 updates a second, the photocopier catches fire!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CopyOnWrite Mimarisi ve Kullanım Alanları: 1) Snapshot İteratör Garantisi: `iterator()` çağrıldığı anda o anki dizinin referansını alır. Başka bir thread listeye 1 milyon eleman eklese veya silse bile, iteratör kendi anlık görüntüsünü (Snapshot) taramaya devam eder; asla `ConcurrentModificationException` fırlatmaz. 2) Ağır Yazma Cezası (O(N) Allocation): Her `add()`, `set()`, `remove()` işlemi dizinin tamamını `Arrays.copyOf` ile yeniden tahsis eder. 3) Ne Zaman Kullanılmalıdır?: 'Okuma Çok (%99+), Yazma Çok Nadir (%1)' olan sistemlerde: Event Listener listeleri, filtre zincirleri ve güvenlik kuralı tabloları için mükemmeldir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CopyOnWrite Architectural Economics: 1) Fail-Safe Snapshot Iteration: The iterator holds a reference to the exact array state at the moment `iterator()` was invoked. Mutations occurring during iteration swap the internal `volatile Object[] array` reference without modifying the iterator's snapshot array, eliminating `ConcurrentModificationException` and the need for locks during reads. 2) O(N) Write Amplification: Every write operation acquires an exclusive `ReentrantLock` and clones the underlying backing array (`Arrays.copyOf()`), inducing substantial GC allocation pressure. 3) Domain Fit: Exclusively viable for read-dominant collections such as observer listener lists, routing tables, and security filter chains.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Event Listener Havuzunda CopyOnWriteArrayList Kullanımı
import java.util.concurrent.CopyOnWriteArrayList;

public class SecurityAuditNotifier {
    // Listener listesi: Okuma çok sık, yeni listener ekleme çok nadirdir
    private final CopyOnWriteArrayList<String> securitySubscribers = new CopyOnWriteArrayList<>();

    public void subscribe(String serviceName) {
        securitySubscribers.add(serviceName); // Dizi kopyalanarak yazılır
    }

    public void broadcastSecurityAlert(String alertMessage) {
        // SIFIR KİLİT! İterasyon anında liste değişse bile patlamaz!
        for (String subscriber : securitySubscribers) {
            IO.println("Alert yollandı -> " + subscriber + ": " + alertMessage);
        }
    }

    void main() {
        var notifier = new SecurityAuditNotifier();
        notifier.subscribe("FraudService");
        notifier.subscribe("AuthGateway");
        notifier.broadcastSecurityAlert("CVE-2026-9999 tespit edildi!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı '100.000 elemanlı bir listeye sürekli veri ekleyeceğimiz bir sistemde CopyOnWriteArrayList kullanabilir miyiz?' diye sorar. Yanıt: 'KESİNLİKLE HAYIR! Her eleman eklemede 100.000 elemanlık dizi baştan kopyalanır; bellek ve işlemci tükenir, sistem OOM veya Garbage Collection duraklamalarıyla felç olur!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Performance Trap: 'Can we use CopyOnWriteArrayList for high-throughput stream ingestion?' Answer: 'ABSOLUTELY NOT! Appending 100,000 items creates 100,000 full array allocations and copies, consuming gigabytes of transient heap and collapsing GC pause times. Use ConcurrentLinkedQueue or Collections.synchronizedList instead.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CopyOnWriteArrayList okuma işlemlerinde sıfır kilit maliyetiyle çalışır.
- İteratörler anlık görüntü (snapshot) üzerinden gezinir; CME hatası imkansızdır.
- Yazma işlemleri diziyi baştan kopyaladığı için sık yazılan sistemlerde kullanılmaz.

🎯 Key Takeaways for the Interview

- CopyOnWriteArrayList provides lock-free, volatile-read performance for all queries.
- Iterators traverse an isolated array snapshot, guaranteeing immunity to CME.
- Writes incur full array re-allocation; strictly avoid in write-heavy environments.

33

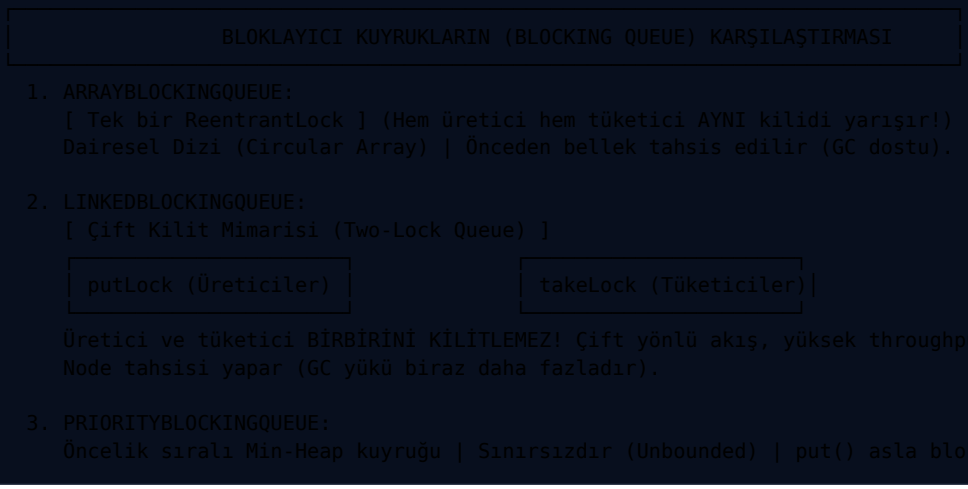
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

Bloklayıcı Kuyruklar: ArrayBlockingQueue vs LinkedBlockingQueue vs PriorityBlockingQueue

Blocking Queues: ArrayBlockingQueue vs LinkedBlockingQueue vs PriorityBlockingQueue

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Fabrika çıkışındaki 3 farklı konveyör bandı gibidir: 1) ArrayBlockingQueue: Sabit 10 kutuluk metal bir tepsidir. Tepsi baştan ayrılmıştır, yeri sabittir. Ama tepsinin başında tek bir kilit vardır; aşçı yemek koyarken garson yemek alamaz. 2) LinkedBlockingQueue: Çift kapılı tüneldir. Tünelin girişinde ayrı kilit (putLock), çıkışında ayrı kilit (takeLock) vardır! Aşçı arkadan yemekleri yüklerken garson önden tepsileri alır; birbirlerinin ayağına basmazlar! 3) PriorityBlockingQueue: Acil servis kuyruğudur; durumu en acil olan hasta sıranın en önüne geçer!

🇬🇧 ENGLISH (ELI5)

Compare three factory conveyor belts: 1) ArrayBlockingQueue: A fixed 10-slot circular steel tray. Pre-allocated and bounded. However, it relies on a SINGLE lock shared between producers and consumers. 2) LinkedBlockingQueue: A two-door tunnel using 'Two-Lock Queue' topology. Producers contend for `putLock` at the entrance; consumers contend for `takeLock` at the exit. Producers and consumers operate concurrently without blocking each other! 3) PriorityBlockingQueue: A hospital triage queue where highest-priority elements jump to the head of the line via a binary heap.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

İki Kilitli Kuyruk Algoritması (Two-Lock Queue): 1) LinkedBlockingQueue'nun Başarımı: H.S. Chao ve Doug Lea'nın uyguladığı bu algoritmada, baş (head) ve kuyruk (tail) düğümleri iki ayrı `ReentrantLock` (`putLock` ve `takeLock`) ile korunur. Üreticiler sadece kuyruğun sonuna eleman eklerken, tüketiciler kuyruğun başından çeker. Aralarındaki tek bağ `AtomicInteger count` değişkenidir. Bu sayede eşzamanlı okuma ve yazma işlemlerinde `ArrayBlockingQueue`'dan 2-3 kat daha yüksek throughput üretir. 2) Bellek Farkı: ArrayBlockingQueue diziyi baştan tek parça tahsis eder, sıfır GC çöpü üretir. LinkedBlockingQueue her eleman için bir `Node` nesnesi tahsis eder; yüksek frekansta Young Generation GC baskısı yaratır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Two-Lock Queue Mechanics & Heap Trade-Offs: 1) LinkedBlockingQueue Concurrency Isolation: Employs a variation of the Michael & Scott Two-Lock Queue algorithm. Enqueue operations synchronize exclusively on `putLock` modifying `last`; dequeue operations synchronize exclusively on `takeLock` modifying `head`. Atomicity of capacity is preserved via `AtomicInteger count`. Producers and consumers rarely serialize against one another. 2) Garbage Collection Overhead: `ArrayBlockingQueue` allocates an `Object[]` array upfront, maintaining zero allocation churn at runtime. `LinkedBlockingQueue` instantiates a `Node\<E>` wrapper for every single item enqueued, generating GC churn in high-throughput allocations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: LinkedBlockingQueue ile Yüksek Hızlı Producer-Consumer Pipeline
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;

public class HighSpeedPipeline {
    void main() throws InterruptedException {
        // ⚠ Üretimde mutlaka bounded kapasite verilmelidir!
        var queue = new LinkedBlockingQueue<String>(10_000);

        // Üretici (Producer) - putLock üzerinde çalışır
        Thread.ofPlatform().name("IngestionWorker").start(() -> {
            try {
                for (int i = 1; i <= 3; i++) {
                    queue.put("Mesaj-" + i);
                    IO.println("Kuyruğa eklendi: Mesaj-" + i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Tüketici (Consumer) - takeLock üzerinde çalışır (Çakışma YOK!)
        Thread.ofPlatform().name("ExportWorker").start(() -> {
            try {
                for (int i = 1; i <= 3; i++) {
                    String msg = queue.poll(1, TimeUnit.SECONDS);
                    IO.println("Kuyruktan işlendi: " + msg);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'ArrayBlockingQueue mu yoksa LinkedBlockingQueue mu daha az bellek tüketir ve GC dostudur?' diye sorar. Yanıt: 'ArrayBlockingQueue çok daha GC dostudur çünkü baştan tek bir dizi tahsis eder, çalışma anında hiç yeni nesne (Node) üretmez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Interview Nuance: 'Which queue exhibits superior GC profile under extreme load?' Senior Answer: 'ArrayBlockingQueue! It allocates its backing array once during construction, generating zero runtime node allocations. LinkedBlockingQueue allocates a Node instance for every put, inducing significant allocation churn.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ArrayBlockingQueue tek kilit kullanır; LinkedBlockingQueue çift kilit (put/take) kullanır.
- LinkedBlockingQueue üretici ve tüketiciyi ayırdığı için daha yüksek throughput sunar.
- ArrayBlockingQueue baştan dizi tahsis ettiği için sıfır GC çöpü üretir.

🎯 Key Takeaways for the Interview

- ArrayBlockingQueue uses a single lock; LinkedBlockingQueue uses independent put/take locks.
- Dual locks grant LinkedBlockingQueue higher concurrency by decoupling producers from consumers.
- ArrayBlockingQueue avoids GC churn by reusing its preallocated array storage.

34

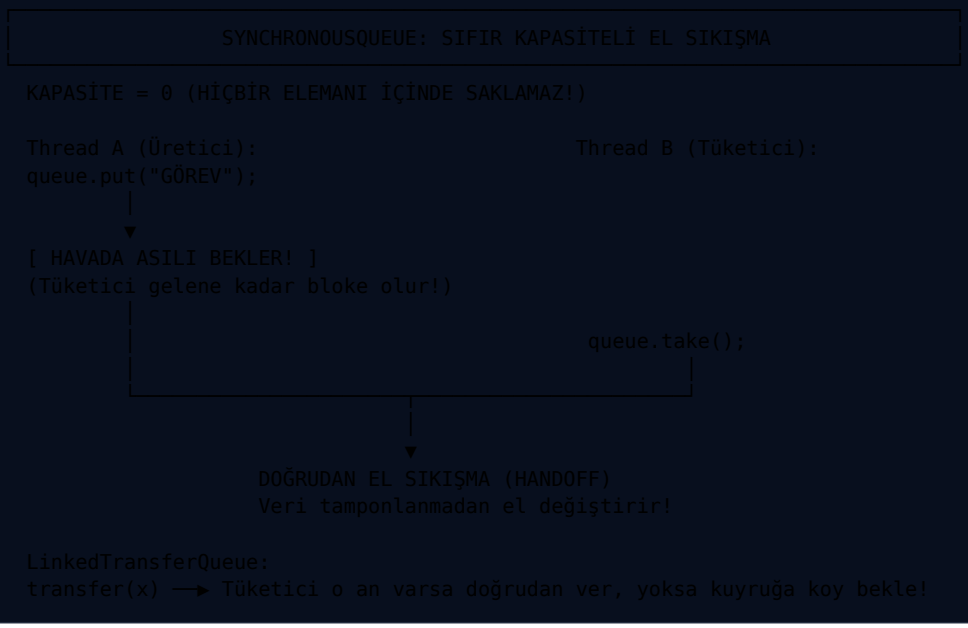
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

Sıfır Kapasiteli El Sıkışma: SynchronousQueue vs TransferQueue

Zero-Capacity Direct Handoffs: SynchronousQueue vs TransferQueue

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Normal bir restoranda aşçı yemeği yapar, tezgahın üzerine koyar (Tamponlu kuyruk). SynchronousQueue'da ise tezgah YOKTUR (Kapasite = 0)! Aşçı sıcak tabağı elinde havada tutar (`put()`). Bir garson gelip tabağı aşçının elinden alana kadar (`take()`) aşçı kaskatı bekler! Tabak elden ele doğrudan teslim edilir. Arada hiçbir bekleme masası yoktur. Eğer hemen teslim alamayacaksan, aşçı yemeği pişirmeye bile kalkışmaz!

🇬🇧 ENGLISH (ELI5)

In a standard cafeteria, chefs set plates onto a warming counter (Buffering). A SynchronousQueue HAS NO COUNTER (Capacity = 0). The chef holds the scalding plate in mid-air (`put()`). The chef stands frozen in place until a server arrives to take the plate directly from their hands (`take()`). The transfer is an immediate atomic handoff from one thread's registers to another. If no server is waiting, the producer blocks instantly.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Doğrudan El Sıkışma (Direct Handoff) ve Dual Stack/Queue Mimarisi: 1) Sıfır Kapasite Felsefesi: `SynchronousQueue.size()` daima 0 döner; `peek()` daima null döner. Bir elemanı kuyruğa koymak ile o elemanın başka bir thread tarafından teslim alınması aynı zaman diliminde gerçekleşmek zorundadır. 2) Non-blocking Dual Queue (Scherer-Scott Algoritması): SynchronousQueue içsel olarak kilit kullanmaz; bekleyen üreticileri veya tüketicileri temsil eden düğümleri (QNode) lock-free bir yığında (TransferStack) veya kuyrukta (TransferQueue) CAS ile yönetir. 3) LinkedTransferQueue: Java 7 ile gelen bu yapı hem `BlockingQueue` hem de `TransferQueue` özelliklerini birleştirir. `tryTransfer(e)` metodu eğer bekleyen tüketici varsa anında el sıkışır, yoksa beklemeden false döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Zero-Capacity Handoff Mechanics: 1) The Non-Buffering Invariant: `SynchronousQueue` has zero internal storage. `put(e)` blocks until a concurrent `take()` arrives; `take()` blocks until a concurrent `put(e)` arrives. 2) Scherer-Scott Dual Stack/Queue: Implemented via lock-free dual data structures. Nodes represent either data requests or data fulfillment. Threads match against opposite-mode nodes using atomic CAS operations, eliminating monitor overhead. 3) `LinkedTransferQueue`: Blends `LinkedBlockingQueue` buffering with `SynchronousQueue` handoffs. The `transfer()` method blocks until a consumer accepts the item; `tryTransfer()` offers instant zero-wait handoffs.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: SynchronousQueue ile Doğrudan Handoff Gösterimi
import java.util.concurrent.SynchronousQueue;

public class DirectHandoffDemo {
    void main() throws InterruptedException {
        var syncQueue = new SynchronousQueue<String>();

        // Tüketici Thread (1 saniye geç başlıyor)
        Thread.ofPlatform().name("Consumer").start(() -> {
            try {
                Thread.sleep(1000);
                IO.println("Tüketici geldi, veriyi elden alıyor...");
                String item = syncQueue.take();
                IO.println("Tüketici teslim aldı: " + item);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Üretici Thread
        IO.println("Üretici: Veri teslim edilmeye çalışılıyor (put)...");
        long start = System.currentTimeMillis();
        syncQueue.put("Ultra-Önemli VIP Mesaj"); // Tüketici gelene kadar 1 saniye burada BLOKE OLUR!
        long elapsed = System.currentTimeMillis() - start;

        IO.println("Üretici: Veri el sıkışarak teslim edildi! Geçen süre: " + elapsed + " ms");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Executors.newCachedThreadPool() hangi kuyruğu kullanır ve neden?' Yanıt: 'SynchronousQueue kullanır! Çünkü gelen görevi hiçbir ara tamponda bekletmeden hemen boşta bekleyen bir thread'e teslim etmek ister; boşta thread yoksa anında yeni bir thread açar!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Crucial Interview Question: 'Which queue powers `Executors.newCachedThreadPool()` and why?' Answer: 'It uses `SynchronousQueue`! It demands immediate direct handoff to an idle worker. If all workers are busy, the handoff fails immediately, forcing the pool to spawn a new native thread instantly.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- SynchronousQueue'nun kapasitesi sıfırdır; veriyi asla bellekte tutmaz.
- put() çağrısı başka bir thread take() çağırana kadar bloke bekler.
- Executors.newCachedThreadPool doğrudan handoff için SynchronousQueue kullanır.

🎯 Key Takeaways for the Interview

- SynchronousQueue has zero internal capacity and never buffers elements.
- put() blocks until a matching take() arrives, executing an atomic thread-to-thread handoff.
- newCachedThreadPool relies on SynchronousQueue to enforce immediate task assignment.

35

BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

Kilit-Siz Kuyruklar: ConcurrentLinkedQueue & Michael-Scott Algoritması

Lock-Free Queues: ConcurrentLinkedQueue & The Michael-Scott Algorithm

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              MICHAEL-SCOTT LOCK-FREE BAĞLI LİSTE KUYRUĞU │
└──────────────────────────────────────────────────────────┘

[ DUMMY HEAD NODE ] ──▶ [ NODE 1 ] ──▶ [ NODE 2 ] ──▶ null
        ▲                                   ▲
        |                                   |
   head pointer                        tail pointer

YENİ ELEMAN EKLEME (ENQUEUE - 2 ADIMLI CAS):
Adım 1: tail.next referansını CAS ile yeni düğüme bağla:
        [ NODE 2 ].next = [ YENİ NODE 3 ] ──▶ (CAS Başarılı!)

Adım 2: tail pointer'ını bir ileri kaydır:
        tail = [ YENİ NODE 3 ] ──▶ (CAS ile kaydır)

YARDIMLAŞMA PRENSİBİ (HELPING):
Eğer bir thread Adım 1'i yapıp Adım 2'yi yapamadan CPU'dan düşerse;
Gelen başka bir thread tail'in geride kaldığını fark eder ve tail'i ONUN YERİNE
ileri kaydırır (Helping mechanism)! Sistem ASLA kilitlenmez!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bayrak yarışı düşünün. Koşucular bayrağı birbirine devrederken sırayı durdurup hakemden izin almazlar (Kilit yok). Eğer bir koşucu bayrağı bir sonraki arkadaşına uzatıp tam çizgiye basamadan ayağı takılıp düşerse, arkadan gelen takım arkadaşı onun düşmesine aldırmaz; hemen eğilip onun yapamadığı adımı onun yerine tamamlar (Helping)! Kimse durmaz, bayrak yarışı ışık hızında devam eder!

🇬🇧 ENGLISH (ELI5)

Imagine a relay race with no referee stopping the clock for handoffs (Lock-Free). If Runner A clips the baton to Runner B's hand but trips before advancing the lane marker, Runner C arriving behind them doesn't panic. Runner C notices the unadvanced marker, nudges it forward on Runner A's behalf (The Helping Mechanism), and proceeds! Nobody waits for fallen runners; active threads complete stalled work cooperatively!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Michael-Scott Kuyruk Algoritması Derinliği: 1) Non-Blocking Garanti: Maged M. Michael ve Michael L. Scott tarafından 1996'da tasarlanan bu algoritma, `ConcurrentLinkedQueue`'nun kalbidir. Sistemin bütününde en az bir thread daima ilerleme (Progress guarantee) kaydeder. 2) Dummy Node (Yalancı Baş Düğüm): Kuyruk hiçbir zaman tamamen boş kalmaz; daima verisiz bir dummy node ile başlar. Bu sayede `head == tail` durumu özel bir kontrol gerektirmez. 3) Hop Count Optimizasyonu: HotSpot uygulamasında tail her `offer()` çağrısında değil, iki adımda bir güncellenir (`hops >= 2`). Bu sayede her eklemede iki kez CAS yapma maliyeti yarıya indirilir! 4) Bloklanmaz (Non-Blocking): `take()` gibi bloklayan bir metodu yoktur; boşsa `poll()` anında `null` döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Michael-Scott Lock-Free Queue Mechanics: 1) Theoretical Significance: Formally proven non-blocking queue published in 1996, foundational to `ConcurrentLinkedQueue`. It guarantees lock-free system-wide progress. 2) The Sentinel Dummy Node: The queue is initialized with a dummy sentinel node where `head == tail`. Enqueue operations never encounter an empty reference, eliminating null checks during concurrent pointer updates. 3) The Helping Hand Protocol: Enqueue is a two-step operation: swing `tail.next` to the new node, then swing `tail` to the new node. If Thread A stalls after step 1, Thread B detects `tail.next != null` and autonomously completes step 2 for Thread A before proceeding with its own enqueue! 4) Hop Batching: HotSpot batches tail updates every two hops, halving the number of expensive CAS cache-invalidation cycles.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: ConcurrentLinkedQueue ile Kilit-Siz Yüksek Hızlı Kuyruk
import java.util.concurrent.ConcurrentLinkedQueue;

public class LockFreeQueueBenchmark {
    void main() throws InterruptedException {
        var queue = new ConcurrentLinkedQueue<Integer>();

        int producers = 4;
        int itemsPerProducer = 100_000;
        Thread[] threads = new Thread[producers];

        long start = System.currentTimeMillis();

        for (int i = 0; i < producers; i++) {
            threads[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < itemsPerProducer; j++) {
                    queue.offer(j); // SIFIR KİLİT! Tamamen CAS ile ekleme
                }
            });
        }

        for (Thread t : threads) t.join();
        long elapsed = System.currentTimeMillis() - start;

        IO.println("400.000 eleman kilit-siz eklendi. Süre: " + elapsed + " ms");
        IO.println("Kuyruk boyutu: " + queue.size());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Performans Uyarısı: `ConcurrentLinkedQueue.size()` metodunu döngü içinde KESİNLİKLE çağırmayın! Çünkü kilit-siz kuyrukta sabit bir `size` değişkeni tutulamaz (tüm thread'lerin o değişkende kilitlenmesi gerekirdi). Bu yüzden `size()` metodu her çağrıldığında tüm kuyruğu baştan sona tek tek sayar (O(N) karmaşıklık)! Kuyruğun boş olup olmadığını anlamak için `isEmpty()` kullanılmalıdır (O(1)).

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Severe Performance Trap: Never invoke `ConcurrentLinkedQueue.size()` in a tight loop! Because a lock-free queue cannot maintain an atomic size counter without crippling CAS scalability, `size()` iterates the entire linked list from head to tail ($O(N)$ runtime). Always invoke `isEmpty()` which checks `head.next == null` in $O(1)$ time.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ConcurrentLinkedQueue, Michael-Scott kilit-siz kuyruk algoritmasını kullanır.
- Helping mekanizması sayesinde geri kalan pointer'lar diğer thread'ler tarafından tamamlanır.
- size() metodu O(N) karmaşıklıktadır; boşluk kontrolü için daima isEmpty() kullanılmalıdır.

🎯 Key Takeaways for the Interview

- ConcurrentLinkedQueue is built upon the Michael-Scott non-blocking queue algorithm.
- Cooperative helping allows active threads to swing stalled tail pointers forward.
- size() runs in O(N) linear time; always use isEmpty() for O(1) empty checks.

36

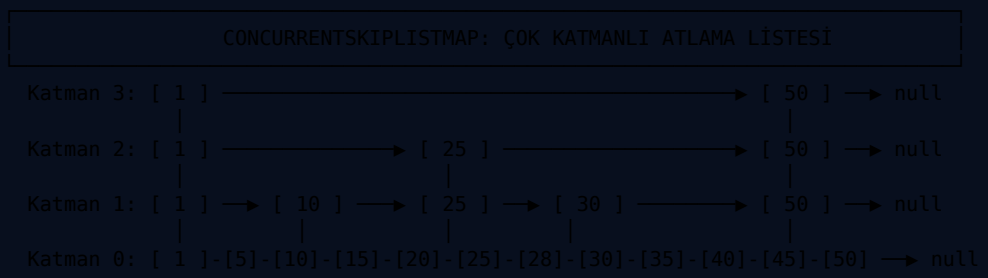
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

ConcurrentSkipListMap: Thread-Safe Sıralı Haritalar & Logaritmik Arama

ConcurrentSkipListMap: Thread-Safe Sorted Maps & Logarithmic Skip-Lists

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tren hattı düşünün. En alttaki tren her istasyonda durur (Yerel banliyö treni: 1, 2, 3, 4…). Yolculuk saatler sürer. Ama üstte bir ekspres tren vardır; sadece büyük şehirlerde durur (1, 25, 50…). Siz önce ekspres trene binip hedefinize en yakın büyük şehre gidersiniz, oradan inip yerel trene binersiniz! SkipList tam olarak budur: Sıralı veriyi kilit kullanmadan O(log N) hızında arayan çok katmanlı ekspres tren hattıdır!

🇬🇧 ENGLISH (ELI5)

Think of an express subway system. The local train on the bottom track stops at every single station (1, 2, 3… 50). Riding it takes forever. But directly above it runs an Express line stopping only at major hubs (1, 25, 50). To find Station 30, you ride the top express line to Station 25, hop down one flight of stairs to the local track, and arrive at 30 in seconds! SkipLists provide lock-free logarithmic search (O(log N)) without rebalancing locks!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ConcurrentSkipListMap vs TreeMap Mimarisi: 1) TreeMap Neden Thread-Safe Yapılamaz?: Kırmızı-Siyah Ağaçlar (Red-Black Trees) ekleme veya silme yapıldığında ağacı dengelemek (Rotation) zorundadır. Ağaç döndürme işlemi kök düğüm dahil birden fazla düğümü kilitler; kilit-siz (lock-free) bir AVL veya Red-Black Tree yazmak pratikte imkansızdır. 2) William Pugh Skip-List Mimarisi: Düğümler olasılıksal olarak yazıtura atılarak (Coin Flipping) rastgele katmanlara terfi ettirilir. Ekleme ve silme sadece yerel komşu düğümler üzerinde CAS ile yapılır; global ağaç döndürme gerekmez! 3) Thread-Safe Sıralı Gezinme: `subMap()`, `headMap()`, `tailMap()` gibi aralık (Range-Query) sorguları çoklu thread ortamında yüksek başarımla çalışır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

SkipList Topology vs Red-Black Trees: 1) The Impossibility of Lock-Free Trees: Balancing binary search trees (AVL, Red-Black) requires multi-node structural rotations. Synchronizing rotations across concurrent mutations creates massive contention bottlenecks, making lock-free balanced trees practically intractable. 2) Probabilistic Level Promotion: William Pugh's Skip-List utilizes random coin flips (1/2 probability) to build index levels. Mutations require CAS modifications strictly on local adjacent pointers without structural rebalancing! 3) Range Query Superiority: `ConcurrentSkipListMap` delivers thread-safe NavigableMap semantics, enabling concurrent subMap and range scans with bounded O(log N) search latency under heavy concurrent writes.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ConcurrentSkipListMap ile Sıralı Liderlik Tablosu (Leaderboard)
import java.util.concurrent.ConcurrentSkipListMap;

public class HighFrequencyLeaderboard {
    // Skorlara göre doğal sıralı thread-safe harita
    private final ConcurrentSkipListMap<Integer, String> scores = new ConcurrentSkipListMap<>();

    public void addScore(int score, String player) {
        scores.put(score, player); // O(log N) kilit-siz ekleme
    }

    public void printTopScores(int minScore) {
        // minScore üzerindeki oyuncuları sıralı getir (Range Query)
        var topTier = scores.tailMap(minScore);
        IO.println("Liderlik Tablosu (" + minScore + "+ Puan):");
        topTier.descendingMap().forEach((score, player) ->
            IO.println("• " + player + " : " + score + " Puan")
        );
    }

    void main() {
        var board = new HighFrequencyLeaderboard();
        board.addScore(9800, "Alice");
        board.addScore(4500, "Bob");
        board.addScore(12400, "Charlie");
        board.printTopScores(9000);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Sorusu: 'Collections.synchronizedSortedMap(new TreeMap()) varken neden ConcurrentSkipListMap kullanalım?' Yanıt: 'synchronizedSortedMap tek bir kaba kilit (Mutex) kullanır; aynı anda sadece tek bir thread işlem yapabilir. ConcurrentSkipListMap ise kilit-siz (lock-free CAS) çalışır; yüzlerce thread aynı anda haritayı okuyup yazabilir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

MAANG Interview Comparison: 'Why prefer ConcurrentSkipListMap over Collections.synchronizedSortedMap(new TreeMap())?' Senior Answer: '`synchronizedSortedMap` wraps a single coarse-grained lock around every method, serializing all threads. `ConcurrentSkipListMap` employs lock-free CAS pointer manipulation on local nodes, scaling concurrently across hundreds of CPU cores.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ConcurrentSkipListMap, TreeMap'in thread-safe ve kilit-siz alternatifidir.
- Skip-List yapısı rastgele katmanlarla O(log N) arama, ekleme ve silme sunar.
- Kırmızı-Siyah ağaç rotasyonlarına ihtiyaç duymadığı için lock-free ölçeklenir.

🎯 Key Takeaways for the Interview

- ConcurrentSkipListMap is the lock-free, thread-safe counterpart to TreeMap.
- Skip-Lists employ probabilistic index levels to achieve O(log N) operational bounds.
- Eliminating tree rotations allows skip-lists to scale concurrently under heavy writes.

BÖLÜM 7 / MODULE 7

Asenkron Boru Hatları & CompletableFuture

Asynchronous Pipelines & CompletableFuture

Geleneksel Future engellemelerinden (blocking get) kurtuluş, CompletableFuture ile reaktif veri akışları, thenCompose/thenCombine bileşimleri, allOf/anyOf orkestrasyonu, hata kurtarma zincirleri ve zaman aşımı yönetimi.

Escaping traditional blocking Future traps, CompletableFuture reactive processing pipelines, thenCompose/thenCombine functional chains, allOf/anyOf orchestration, resilient error recovery, and timeout handling.

37

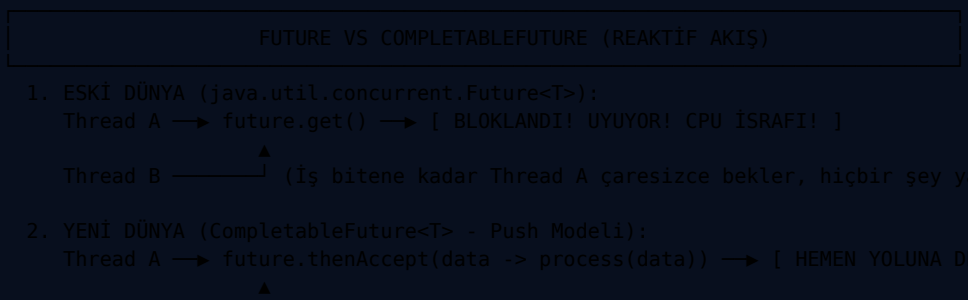
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Future<T> vs CompletableFuture<T>: Polling Kâbusundan Reaktif İtmeye Geçiş

Future<T> vs CompletableFuture<T>: Escaping the Polling Nightmare

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski Future, pizza siparişi verip kapının deliğine gözünü dayayarak 45 dakika boyunca hiçbir yere kımıldamadan kuryeyi beklemeye benzer (`future.get()` işi bitene kadar thread'i kilitler). CompletableFuture ise kapıya 'Pizza gelince zili çal, pizzayı fırına atıp kutuyu çöpe at' yazılı bir akıllı talimat yapıştırmaktır. Kurye geldiğinde talimatlar otomatik yürütülür, sen o sırada salonda televizyon izlemeye devam edersin!

🇬🇧 ENGLISH (ELI5)

The legacy `Future\<T>` is like ordering food and standing motionlessly with your eye glued to the door peephole for 45 minutes (`future.get()` blocks the thread dead). `CompletableFuture\<T>` is leaving an intelligent sticky note on the door: 'When the courier arrives, put pizza on the table, tip $5, and text me'. You continue watching television. When the data is ready, callbacks push reactions forward without blocking any CPU thread!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 5'te gelen `Future\<T>` arayüzünün 3 devasa zafiyeti vardı: 1) Bloklayan Çekme Modeli: Sonucu almak için `future.get()` çağrılır ve çağıran thread uyutulur (Context Switch maliyeti). 2) Zincirleme Yapılamaz: Bir Future'ın çıktısını diğer bir Future'ın girdisi yapacak deklaratif yapı yoktur. 3) Manuel Tamamlama İmkânsızdır: Dışarıdan bir etkinlik (web-socket, JMS mesajı) geldiğinde Future'ı el ile sonuçlandıramazsınız. Java 8 ile gelen `CompletableFuture\<T>`, `CompletionStage` ve `Future` arayüzlerini birleştirir. ForkJoinPool.commonPool() üzerinde veya özel thread havuzlarında asenkron görevleri non-blocking (olay güdümlü) zincirler halinde yürütür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Shortcomings of legacy `Future\<T>`: 1) Blocking Polling Trap: Callers are trapped in synchronous blocking via `future.get()`, squandering expensive kernel threads. 2) Absence of Declarative Chaining: Cannot natively express 'execute task B when task A completes'. 3) Immutable Completion: Callers cannot externally resolve or fail a Future instance. `CompletableFuture\<T>` implements both `Future\<T>` and `CompletionStage\<T>`. It adopts the Promise pattern: tasks push transformations down a directed acyclic graph (DAG) executed on `ForkJoinPool.commonPool()` or custom `Executor` contexts without idling caller threads.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Non-blocking CompletableFuture ile Sipariş İşleme
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.Executors;

public class NonBlockingOrderPipeline {
    record Order(String id, double amount, String status) {}

    public static void main(String[] args) {
        try (var customPool = Executors.newFixedThreadPool(4)) {
            // Asenkron veri çekme başlat (Non-blocking!)
            CompletableFuture<Order> orderFuture = CompletableFuture.supplyAsync(() -> {
                System.out.println("Sipariş çekiliyor: " + Thread.currentThread());
                return new Order("ORD-9821", 250.0, "PENDING");
            }, customPool);

            // Reaktif zincir: Veri geldiğinde otomatik işlet (Bloklama yok!)
            orderFuture
                .thenApply(order -> new Order(order.id(), order.amount() * 1.20, "TAX_INCLUDED"))
                .thenAccept(processed -> {
                    System.out.println("İşlem Tamamlandı: " + processed + " Thread: " +
Thread.currentThread());
                });

            System.out.println("Ana thread asla bloklanmadı, diğer işlere devam ediyor...");
            Thread.sleep(100); // Demo amaçlı arka plan işinin bitmesini bekle
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Argümansız supplyAsync(Supplier) varsayılan olarak ForkJoinPool.commonPool() kullanır! Ağ/veritabanı gibi bloklayan (I/O-bound) işleri commonPool'a yüklerseniz havuzdaki tüm CPU çekirdeklerini kilitler ve sistemdeki paralel stream'leri felç edersiniz. I/O işleri için daima özel bir Executor (veya Java 21+ Virtual Thread Executor) geçin!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Executing `supplyAsync()` without an explicit executor defaults to `ForkJoinPool.commonPool()`. Flooding the common pool with blocking I/O (database, REST calls) exhausts CPU-worker threads, starving parallel streams and application background tasks. Always supply a dedicated I/O thread pool or a Virtual Thread executor!

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Future.get() çağrısı thread'i kilitler. CompletableFuture ise reaktif itme (push) modelidir; iş bittiğinde sonraki aşama otomatik tetiklenir.

🎯 Key Takeaways for the Interview

• `Future.get()` blocks threads synchronously. `CompletableFuture` employs non-blocking event-driven completion stages via push semantics.

38

BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Dönüşüm Boru Hatları: thenApply vs thenAccept vs thenRun & Thread Havuzu Geçişleri

Transformation Pipelines: thenApply vs thenAccept vs thenRun & Async Variants

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                DÖNÜŞÜM OPERATÖRLERİ KARŞILAŞTIRMASI          │
└──────────────────────────────────────────────────────────────┘

Operatör        | Girdi Alır mı?   | Çıktı Üretir mi? | Fonksiyonel Arayüz
----------------|------------------|------------------|-------------------
thenApply(f)    | EVET (T)         | EVET (U)         | Function<T, U>
thenAccept(c)   | EVET (T)         | HAYIR (void)     | Consumer<T>
thenRun(r)      | HAYIR (void)     | HAYIR (void)     | Runnable

*Async Farkı:
stage.thenApply(fn)      ──► Önceki görevi bitiren thread AYNI thread'de devam eder.
stage.thenApplyAsync(fn) ──► Görevi Executor kuyruğuna atar, BAŞKA bir thread çalıştırır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1) thenApply: Aşçıya eti verirsin, pişirip biftek olarak geri verir (Girdi var, dönüşüm var). 2) thenAccept: Garsona bifteği verirsin, afiyetle yer veya müşteriye sunar, sana bir şey geri vermez (Girdi var, çıktı yok). 3) thenRun: İki işlem de bitince aşçı mutfaktaki gongu çalar; ne etle ilgilenir ne tabakla, sadece 'İş bitti!' der (Girdi yok, çıktı yok). Sonuna 'Async' eklersen bu işi başka bir personele devredersin!

🇬🇧 ENGLISH (ELI5)

1) `thenApply`: Handing raw flour to a baker and receiving bread back (consumes `T`, returns `U`). 2) `thenAccept`: Handing bread to a customer who consumes it without returning a value (consumes `T`, returns `void`). 3) `thenRun`: The kitchen clock ringing at closing time; independent of data (consumes `void`, returns `void`). Appending `*Async` (e.g., `thenApplyAsync`) dispatches execution to an Executor queue rather than running inline on the completing thread.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Senkron ve Asenkron varyantlar arasındaki kritik ayrım: `stage.thenApply()` çağrıldığında, eğer önceki aşama zaten tamamlanmışsa fonksiyon çağıran thread üzerinde çalışır. Önceki aşama henüz bitmemişse, o aşamayı tamamlayan iş parçacığı (worker thread) üzerinde inline olarak çalıştırılır; böylece gereksiz context-switch önlenir. `stage.thenApplyAsync(fn, executor)` ise işi kesinlikle belirtilen executor'ın iş kuyruğuna teslim eder. Uzun süren CPU hesaplamaları veya bağımsız thread context'i (örneğin MDC/SecurityContext) gerektiren adımlar için `*Async` varyantları tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Inline Execution vs Async Dispatch: In `stage.thenApply(fn)`, if the preceding stage has already completed, `fn` executes synchronously on the calling thread. If pending, it executes inline on whichever worker thread completes the upstream stage, eliminating queue dispatch overhead. Conversely, `thenApplyAsync(fn, pool)` guarantees task submission into the designated executor's run-queue. Use `*Async` methods when the transform logic is CPU-intensive or requires clean thread isolation.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Pipeline Aşamaları ve Özel Executor Geçişi
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.Executors;

public class PipelineTransformationDemo {
    record UserProfile(long id, String rawName, String email) {}
    record EnrichedProfile(long id, String formattedName, boolean isVerified) {}

    void main() {
        try (var cpuPool = Executors.newFixedThreadPool(2);
             var ioPool = Executors.newVirtualThreadPerTaskExecutor()) {

            CompletableFuture.supplyAsync(() -> {
                // I/O işlemi: Veritabanından profili çek
                System.out.println("1. DB Fetch: " + Thread.currentThread());
                return new UserProfile(42L, "  ada lovelace  ", "ada@lovelace.org");
            }, ioPool)
            // Dönüşüm (Function): String temizleme & zenginleştirme (CPU Pool'a devret)
            .thenApplyAsync(user -> {
                System.out.println("2. CPU Transform: " + Thread.currentThread());
                return new EnrichedProfile(user.id(), user.rawName().trim().toUpperCase(), true);
            }, cpuPool)
            // Tüketim (Consumer): Bildirim gönder veya logla
            .thenAccept(enriched -> {
                System.out.println("3. Notification Sent: " + enriched);
            })
            // Tetikleme (Runnable): Metrik sayacını artır
            .thenRun(() -> {
                System.out.println("4. Pipeline tamamlandı, metrik güncellendi.");
            }).join(); // Test için ana thread'i beklet
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

thenApply içine asla bloklayan I/O yazmayın! Eğer bir önceki işlem hızlıca bitmişse, thenApply çağıran ana thread (örneğin Tomcat HTTP worker thread) üzerinde çalışıp ana isteği kilitler. Bloklayan her adım için thenApplyAsync ve dedicated I/O pool kullanın.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Never execute blocking I/O inside `thenApply`! If the upstream stage finished prior to registration, `thenApply` executes directly on the caller thread (e.g., HTTP request thread), causing catastrophic thread stalling. Reserve blocking operations for `thenApplyAsync`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• thenApply veri dönüştürür (T->U), thenAccept veriyi tüketir (T->void), thenRun sadece sinyal verir (void->void). *Async varyantları işi havuza devreder.

🎯 Key Takeaways for the Interview

• `thenApply` transforms data (T->U), `thenAccept` consumes (T->void), `thenRun` signals completion (void->void). `*Async` submits to an executor.

39

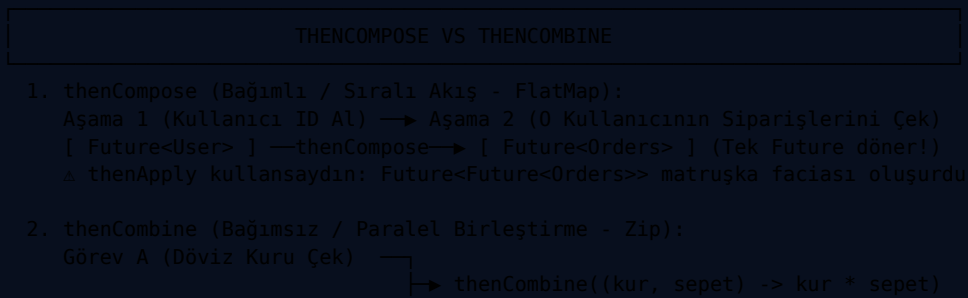
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Bileşim ve Birleştirme: thenCompose (FlatMap) vs thenCombine (Zip) Farkı

Composition & Merging: thenCompose (FlatMap) vs thenCombine (Zip)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

thenCompose: Bir dedektif gibidir. Önce anahtarın nerede olduğunu bulur (1. aşama), sonra o anahtarla kasayı açar (2. aşama). Biri bitmeden diğeri başlayamaz; iki adımı tek bir temiz sonuca bağlar. thenCombine ise düğün pastası hazırlamak gibidir: Biri pastayı fırınlarken diğeri süsleme kremasını çırpar (iki bağımsız iş aynı anda çalışır). İkisi de bitince pastayla kremayı birleştirirler!

🇬🇧 ENGLISH (ELI5)

`thenCompose` is sequential dependency (monadic flatMap): Step 1 retrieves the encryption key; Step 2 uses that key to decrypt the vault payload. Step 2 cannot begin until Step 1 yields its output. `thenCombine` is parallel aggregation (applicative zip): Worker A calculates flight pricing while Worker B fetches hotel availability concurrently. When both arrive, their results are combined into a travel package.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Monadik Yapı ve Tip İmzaları: 1) `thenCompose(Function\<T, CompletionStage\<U>>)`: Fonksiyonel programlamadaki `flatMap` operatörünün birebir karşılığıdır. Eğer `thenApply` kullansaydınız `CompletableFuture\<CompletableFuture\<U>>` şeklinde içiçe geçmiş (nested) bir tür elde ederdiniz. `thenCompose` bu katmanı düzleştirir (flattening). 2) `thenCombine(CompletionStage\<U>, BiFunction\<T, U, V>)`: İki bağımsız `CompletableFuture`'ı paralel olarak koşturur. Her ikisi de başarıyla tamamlandığında `BiFunction`'ı tetikleyerek iki sonucu tek bir `V` değerinde sentezler. Biri çökerse birleşik future da çöker.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Monadic Anatomy: 1) `thenCompose`: Represents functional `flatMap`. If you map an asynchronous function `T -> CompletableFuture\<U>` via `thenApply`, you produce a nested `CompletableFuture\<CompletableFuture\<U>>`. `thenCompose` flattens the hierarchy, binding stages sequentially. 2) `thenCombine`: Functional `zipWith`. Executes both stages asynchronously in parallel. Upon dual completion, it supplies both outputs `(T, U)` to a `BiFunction\<T, U, V>`. If either stage aborts exceptionally, the aggregated stage terminates with `ExecutionException`.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: thenCompose (Sıralı) ve thenCombine (Paralel) Kullanımı
import java.util.concurrent.CompletableFuture;

public class CompositionAndCombineDemo {
    record User(long id, String creditTier) {}
    record CreditLimit(long userId, double limitUsd) {}
    record Cart(long userId, double totalUsd) {}
    record CheckoutDecision(boolean approved, String reason) {}

    static CompletableFuture<User> fetchUser(long id) {
        return CompletableFuture.supplyAsync(() -> new User(id, "PREMIUM"));
    }

    static CompletableFuture<CreditLimit> fetchLimit(User user) {
        return CompletableFuture.supplyAsync(() ->
            new CreditLimit(user.id(), user.creditTier().equals("PREMIUM") ? 10000.0 : 1000.0)
        );
    }

    static CompletableFuture<Cart> fetchCart(long userId) {
        return CompletableFuture.supplyAsync(() -> new Cart(userId, 3200.0));
    }

    void main() {
        long userId = 101L;

        // 1. thenCompose: User -> CreditLimit (Bağımlı Sıralı Akış)
        CompletableFuture<CreditLimit> limitFuture = fetchUser(userId)
            .thenCompose(CompositionAndCombineDemo::fetchLimit);

        // 2. Sepeti bağımsız paralel çek
        CompletableFuture<Cart> cartFuture = fetchCart(userId);

        // 3. thenCombine: Limit ile Sepeti birleştirip karar ver (Paralel Zip!)
        CompletableFuture<CheckoutDecision> decisionFuture = limitFuture.thenCombine(cartFuture,
            (limit, cart) -> {
                boolean ok = cart.totalUsd() <= limit.limitUsd();
                return new CheckoutDecision(ok, ok ? "Onaylandı" : "Kredi limiti yetersiz!");
            });

        CheckoutDecision decision = decisionFuture.join();
        System.out.println("Ödeme Kararı: " + decision);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

thenCombine kullanırken iki görevin birbirini beklemediğinden emin olun. Eğer Görev A'nın girdisi Görev B'ye lazımsa thenCombine ÇALIŞMAZ; o zaman mecburen thenCompose kullanmalısınız. Yanlış operatör seçimi deadlock veya derleme hatalarına yol açar.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`thenCombine` mandates that both futures are strictly independent. If stage B requires an argument generated by stage A, using `thenCombine` is invalid; you must use `thenCompose`. Misusing them creates cascading compile-time type mismatches.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• thenCompose bağımlı asenkron adımları birbirine zincirler (flatMap). thenCombine bağımsız iki asenkron görevi paralel çalıştırıp birleştirir (zip).

🎯 Key Takeaways for the Interview

• `thenCompose` sequences dependent async stages (flatMap). `thenCombine` executes two independent stages in parallel and fuses their outputs (zip).

40

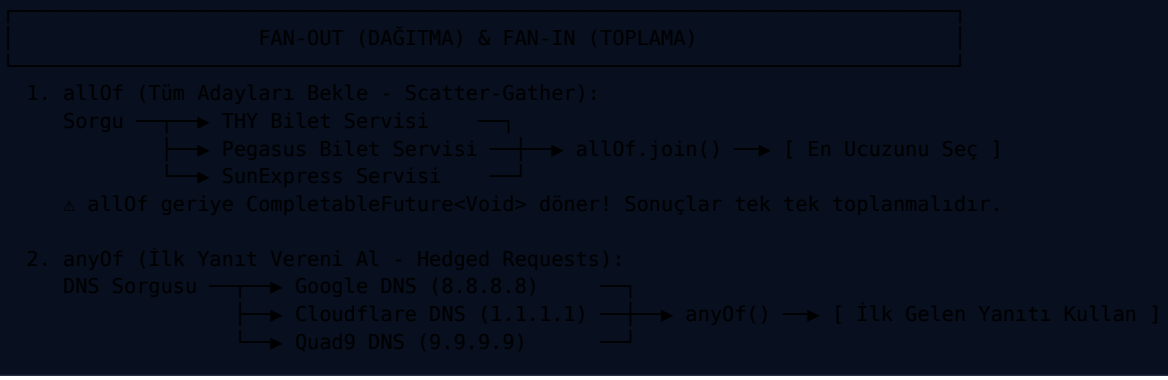
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Fan-Out & Fan-In Orkestrasyonu: CompletableFuture.allOf vs anyOf ile Dağıtık Koordinasyon

Fan-Out & Fan-In Coordination: CompletableFuture.allOf vs anyOf

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

allOf: Bir otobüs şoförünün tüm yolcuların (uçak bileti arayan 5 farklı sitenin) binmesini beklemesi gibidir. Herkes binmeden otobüs kalkmaz; sonra yolcuların biletlerini sayarsın. anyOf ise bir taksi durağında sıraya girip ilk gelen boş taksiye binmektir. 3 taksiyi birden çağırırsın, kapıya ilk yanaşan taksiye biner gidersin; diğerleri boşa çıkar!

🇬🇧 ENGLISH (ELI5)

`CompletableFuture.allOf` is scatter-gather: A travel aggregator querying 5 airline APIs concurrently. The master request waits until every single airline responds before filtering the lowest price. `CompletableFuture.anyOf` is hedged requests: Querying 3 redundant DNS mirrors (Google, Cloudflare, Quad9) simultaneously and immediately returning whichever responds first, discarding trailing packets.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot allOf ve anyOf İç Mantığı: 1) `allOf(CompletableFuture\<?>... cfs)`: Dahili olarak bir `BiRelay` ağacı kurar. Her bir future tamamlandığında bir sayaç eksiltilir. Sayaç 0'a ulaştığında bileşik future tetiklenir. DÖNÜŞ TİPİ `CompletableFuture\<Void>`'dir! Sonuçları almak için orijinal future dizisi stream edilip `CompletableFuture::join` ile toplanır (bu noktada tümü bittiği için join ASLA bloklamaz). 2) `anyOf(CompletableFuture\<?>... cfs)`: İlk tamamlanan (veya ilk istisna fırlatan) future bileşik nesneyi çözer. DÖNÜŞ TİPİ `CompletableFuture\<Object>`'tir ve tip dönüşümü (cast) gerektirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Implementation Details: 1) `allOf`: Constructs a binary tree of completion relays. Each completing leaf decrements an atomic countdown latch. Returns `CompletableFuture\<Void>` by design to accommodate heterogeneous return types. To extract payloads, stream over the input list mapping via `CompletableFuture::join`—which is guaranteed non-blocking because all tasks are already resolved. 2) `anyOf`: Binds an identical completion handler to all upstream stages; the first node to transition invokes `completeRelay()`. Returns `CompletableFuture\<Object>`, requiring explicit downcasting.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: allOf ile Scatter-Gather Fiyat Toplama
import java.util.List;
import java.util.concurrent.CompletableFuture;

public class ScatterGatherAggregator {
    record PriceQuote(String provider, double price) {}

    static CompletableFuture<PriceQuote> fetchQuote(String provider, double price, long delayMs) {
        return CompletableFuture.supplyAsync(() -> {
            try { Thread.sleep(delayMs); } catch (InterruptedException _) {}
            return new PriceQuote(provider, price);
        });
    }

    void main() {
        var futures = List.of(
            fetchQuote("Airline-A", 180.0, 50),
            fetchQuote("Airline-B", 150.0, 80),
            fetchQuote("Airline-C", 210.0, 30)
        );

        // allOf ile tüm servislerin bitmesini bekle
        CompletableFuture<Void> allDone = CompletableFuture.allOf(
            futures.toArray(new CompletableFuture[0])
        );

        // Non-blocking toplama (join garantili olarak anında döner!)
        CompletableFuture<List<PriceQuote>> quotesFuture = allDone.thenApply(_ ->
            futures.stream().map(CompletableFuture::join).toList()
        );

        List<PriceQuote> quotes = quotesFuture.join();
        PriceQuote bestQuote = quotes.stream()
            .min((q1, q2) -> Double.compare(q1.price(), q2.price()))
            .orElseThrow();

        System.out.println("En İyi Teklif: " + bestQuote);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

allOf içinde tek bir future patlarsa (exception), allOf da anında patlar! Ancak diğer çalışan future'lar arka planda çalışmaya ve kaynak tüketmeye devam eder. Dağıtık sistemlerde allOf kullanırken her alt task'ın içine 'exceptionally' koyarak default fallback üretmesini sağlayın.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

If any child future fails inside `allOf`, the composite future fails immediately. However, surviving peer futures continue running in the background, squandering resources. Always attach `.exceptionally()` or fallback handlers to individual stages before passing them into `allOf`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• allOf tüm görevlerin bitmesini bekler (scatter-gather, Void döner). anyOf ilk biteni alır (hedged request, Object döner).

🎯 Key Takeaways for the Interview

• `allOf` waits for all stages to conclude (scatter-gather, returns `Void`). `anyOf` yields the first finished stage (hedged calls, returns `Object`).

41

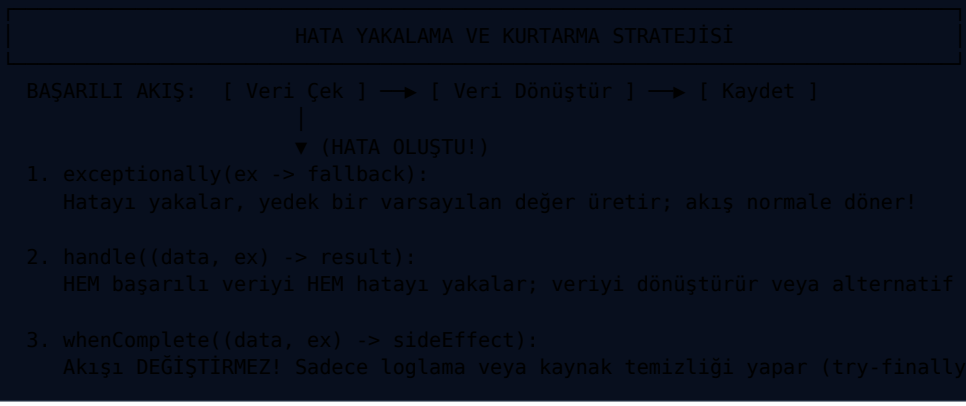
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Hata Kurtarma Zincirleri: exceptionally vs handle vs whenComplete ile Dayanıklılık

Resilience & Exception Handling: exceptionally vs handle vs whenComplete

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kargo teslimatı düşünün: 1) exceptionally: Müşteri evde yoksa kargoyu kapıcıya bırakmaktır (hata olursa yedek plan devreye girer, paket teslim edilmiş sayılır). 2) handle: Kuryenin kapıyı çaldıktan sonra 'Paketi teslim ettim' veya 'Evde yoktu, şubeye yönlendirdim' diye merkez sisteme rapor hazırlamasıdır (hem başarıyı hem hatayı tek bir yeni sonuca çevirir). 3) whenComplete: Paket teslim edilse de kaybolsa da kuryenin araba günlüğüne 'Bu adrese uğrandı' diye imza atmasıdır (akışı değiştirmez, sadece log tutar).

🇬🇧 ENGLISH (ELI5)

1) `exceptionally`: Like a fallback generator. If the primary power grid cuts out, the backup diesel generator fires up seamlessly, supplying replacement power downstream. 2) `handle`: An inspection checkpoint. It receives both `(result, throwable)`. Whether the product is pristine or shattered, it packs it into an inspected report object for downstream consumers. 3) `whenComplete`: Pure side-effect auditing (like `finally`). It logs whether the transaction succeeded or threw an error, but re-emits the exact upstream state downstream without alteration.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Metot İmzaları ve İstisna Kapsülleme (CompletionException): CompletableFuture içindeki kontrol edilmeyen hatalar HotSpot tarafından otomatik olarak `CompletionException` ile sarılır. 1) `exceptionally(Function\<Throwable, T>)`: Yalnızca istisna durumunda tetiklenir. İstisnayı yutar ve geriye geçerli bir `T` değeri döndürerek pipeline'ı kurtarır. 2) `handle(BiFunction\<T, Throwable, U>)`: İstisna olsun veya olmasın her durumda tetiklenir. Eğer `ex != null` ise hata işlenir, aksi halde `res` işlenir. Yeni bir `U` tipi döndürür. 3) `whenComplete(BiConsumer\<T, Throwable>)`: Gözlemcidir (side-effect). Geriye değer döndürmez (`void`). Orijinal stage'in sonucunu veya istisnasını aynen sonraki aşamalara geçirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Exception Handling Mechanics: Unchecked exceptions in asynchronous pipelines are wrapped in `CompletionException`. 1) `exceptionally`: Invoked exclusively on stage failure. Catches the `Throwable`, recovers with an alternative value `T`, and transitions the downstream stage to normal completion. 2) `handle`: Invoked unconditionally on dual outcomes (`T result, Throwable ex`). Transforms either state into `U`. 3) `whenComplete`: Passive terminal/audit stage taking `BiConsumer\<T, Throwable>`. It executes side-effects (metrics, auditing) without intercepting or altering the original value/exception payload.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: CompletableFuture Dayanıklı Hata Kurtarma
import java.util.concurrent.CompletableFuture;

public class ResilientPipelineDemo {
    record Price(String symbol, double amount, boolean isFallback) {}

    static CompletableFuture<Double> fetchLivePriceFromExchange(String symbol) {
        return CompletableFuture.supplyAsync(() -> {
            if ("BTC".equals(symbol)) {
                throw new IllegalStateException("Kripto Borsası Çöktü (503 Service Unavailable)!");
            }
            return 65000.0;
        });
    }

    void main() {
        String symbol = "BTC";

        CompletableFuture<Price> priceFuture = fetchLivePriceFromExchange(symbol)
            // 1. exceptionally ile hatayı yakala ve önbellekten fallback dön!
            .exceptionally(ex -> {
                System.err.println("⚠ Hata Yakalandı: " + ex.getMessage() + ". Önbelleğe
dönülüyor...");
                return 62000.0; // Cached fallback price
            })
            // 2. handle ile veriyi modelle
            .handle((val, ex) -> {
                boolean isFallback = (val == 62000.0);
                return new Price(symbol, val, isFallback);
            })
            // 3. whenComplete ile metrik logla (akışı bozmaz!)
            .whenComplete((price, ex) -> {
                System.out.println("📊 Audit Log: Fiyat başarıyla belirlendi -> " + price);
            });

        Price finalPrice = priceFuture.join();
        System.out.println("Sonuç: " + finalPrice);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

exceptionallyAsync kullanmazsanız, hata anında fallback üretme kodu hatanın patladığı worker thread üzerinde çalışır. Fallback kodu veritabanına veya diske gidiyorsa thread'i kilitler; bu tür I/O kurtarmalarında exceptionallyCompose veya exceptionallyAsync tercih edilmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Unless `exceptionallyAsync` is used, the fallback logic executes synchronously on whichever worker thread encountered the upstream exception. If your fallback invokes blocking I/O (e.g. disk or cache reads), use `exceptionallyComposeAsync` to avoid carrier thread exhaustion.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• exceptionally hatayı varsayılan değerle kurtarır. handle hem hatayı hem başarıyı dönüştürür. whenComplete ise yan etkisiz loglama yapar.

🎯 Key Takeaways for the Interview

• `exceptionally` recovers from failure with a fallback. `handle` translates dual states into a new value. `whenComplete` performs side-effects without altering state.

42

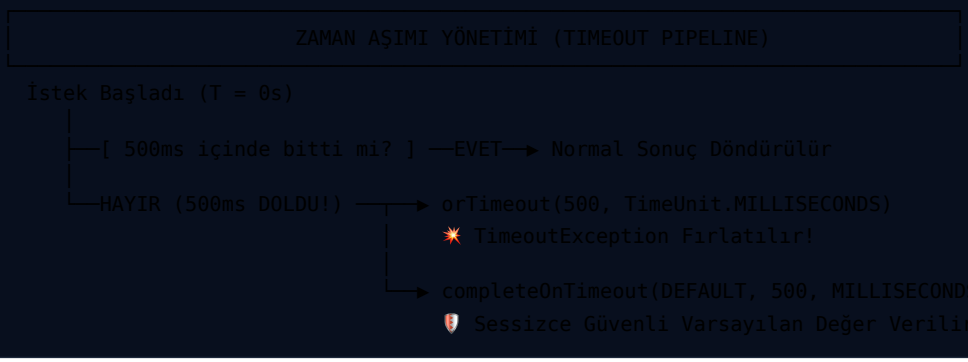
BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Zaman Aşımı ve Varsayılanlar: orTimeout() vs completeOnTimeout() ile Gecikme Yönetimi

Timeouts & Default Fallbacks: orTimeout() and completeOnTimeout()

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda sipariş verdiniz ve garsona dediniz ki: 1) orTimeout: '15 dakikada yemeğim gelmezse masayı terk eder, bağırarak çıkarım!' (Süre dolunca ortalığı ayağa kaldıran bir TimeoutException patlar). 2) completeOnTimeout: '15 dakikada biftek pişmezse bana hazır olan soğuk sandviçi getirin, karnımı doyurayım' demektir (Süre dolunca kimse üzülmez, önceden belirlenen yedek sandviç servis edilir).

🇬🇧 ENGLISH (ELI5)

`orTimeout()` is hard SLA enforcement: 'If data does not arrive in 200ms, abort the mission and throw a `TimeoutException` immediately!' `completeOnTimeout()` is graceful degradation: 'If the recommendation AI does not respond within 50ms, return a default list of trending items so the homepage never loads blank'.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 9+ ve Java 25 Zaman Aşımı Mimarisi: Java 8'de CompletableFuture için zaman aşımı desteği yoktu; harici bir `ScheduledExecutorService` ile geleceğe cancel/complete basmak gerekiyordu. Modern Java iki güçlü yöntem sunar: 1) `orTimeout(long timeout, TimeUnit unit)`: Süre aşıldığında future'ı `java.util.concurrent.TimeoutException` ile anormal şekilde tamamlar (`completeExceptionally`). 2) `completeOnTimeout(T value, long timeout, TimeUnit unit)`: Belirtilen süre dolduğunda eğer future henüz tamamlanmamışsa, verilen `value` ile normal olarak tamamlanır (`complete(value)`). Bu mekanizma arka planda HotSpot'un paylaşımlı `Delayer` zamanlayıcı thread'ini kullanır; her talep için yeni thread açmaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Java 9+ Timeout Architecture: Prior to Java 9, developers had to manually schedule cancellation tasks via `ScheduledExecutorService`. Modern CompletableFuture incorporates: 1) `orTimeout()`: If incomplete when duration elapses, it forces exceptional completion with `TimeoutException`. 2) `completeOnTimeout()`: If incomplete upon timer expiration, it transitions the future to normal completion using the designated fallback object. Both methods register non-blocking delays using an internal daemon `CompletableFuture.Delayer` wheel timer, avoiding per-timeout thread allocations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: completeOnTimeout ile MAANG Servislerinde Graceful Degradation
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;

public class TimeoutManagementDemo {
    record Recommendation(String productId, String category) {}

    static CompletableFuture<Recommendation> fetchAiRecommendations() {
        return CompletableFuture.supplyAsync(() -> {
            try {
                // Yapay zeka servisinin 800ms geciktiğini simüle et
                Thread.sleep(800);
            } catch (InterruptedException _) {}
            return new Recommendation("PROD-AI-99", "Personalized");
        });
    }

    void main() {
        Recommendation defaultFallback = new Recommendation("PROD-POPULAR-01", "Trending Now");

        // 200 ms SLA: 200 ms içinde gelmezse defaultFallback değerini bas!
        CompletableFuture<Recommendation> recoFuture = fetchAiRecommendations()
            .completeOnTimeout(defaultFallback, 200, TimeUnit.MILLISECONDS)
            .thenApply(reco -> {
                System.out.println("Kullanıcıya Gösterilen Ürün: " + reco);
                return reco;
            });

        Recommendation result = recoFuture.join();
        System.out.println("Ekrana Basılan Nihai Sonuç: " + result);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

completeOnTimeout çağrısı arka planda geciken asenkron işlemi İPTAL ETMEZ! Arka plandaki işçi thread geciken HTTP çağrısını yapmaya devam edebilir. Gerçekten soket veya thread iptali gerekiyorsa future.cancel(true) veya HTTP client zaman aşımları yapılandırılmalıdır.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`completeOnTimeout` does NOT cancel the underlying upstream task! The worker thread running the database or HTTP call continues until completion in the background. To abort remote sockets, configure client-level timeouts alongside CompletableFuture.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- orTimeout süre dolduğunda TimeoutException fırlatır. completeOnTimeout ise sessizce yedek bir varsayılan değerle aşamayı tamamlar.

🎯 Key Takeaways for the Interview

- `orTimeout` aborts exceptionally with `TimeoutException`. `completeOnTimeout` silently injects a safe fallback upon SLA breach.

BÖLÜM 8 / MODULE 8

Project Loom & Sanal Thread'ler (Virtual Threads)

Project Loom & Virtual Threads

Platform thread'leri ile sanal thread'lerin farkı (M:N mimarisi), Carrier thread'lere bağlanma/ayrılma (Mounting/Unmounting), Pinning tuzağı ve ReentrantLock çözümü, ThreadLocal bellek sızıntıları, Structured Concurrency ve Java 25 ScopedValue devrimi.

Platform threads vs Virtual Threads (M:N fibers), Carrier thread mounting and unmounting dynamics, the carrier pinning hazard and ReentrantLock remedy, ThreadLocal memory explosion, Structured Concurrency orchestration, and the Java 25 ScopedValue revolution.

43

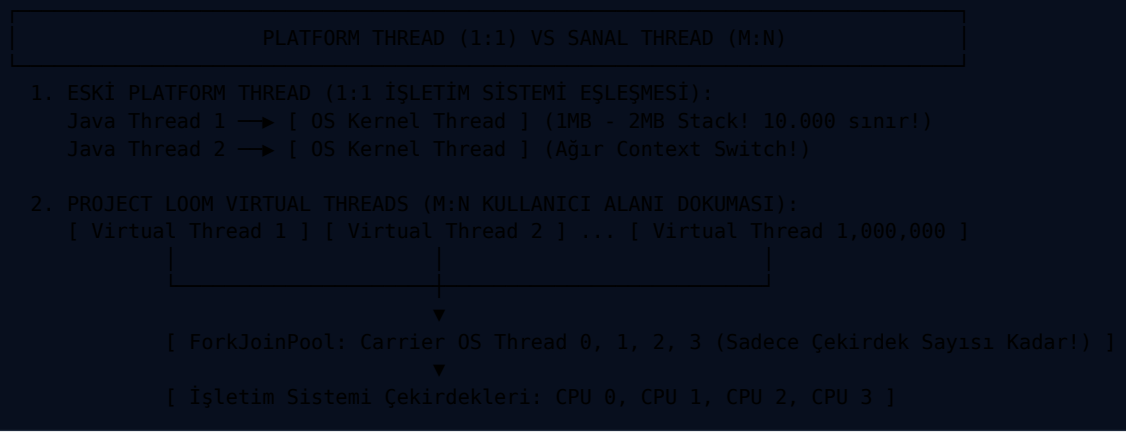
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Platform Thread vs Sanal Thread: M:N Mimarisi & Milyonlarca Eşzamanlı İş Parçacığı

Platform Threads vs Virtual Threads: M:N Fiber Architecture

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Platform thread, her posta paketi için şirkete 10 tonluk devasa bir TIR kiralamak gibidir. Garaja ancak 5000 TIR sığar; her TIR 2 MB yakıt yakar ve şoför değiştirmek dakikalar sürer. Virtual Thread (Sanal Thread) ise aynı TIR'ın içine konan milyonlarca hafif mektup zarfıdır! Mektup okunurken veya cevap beklenirken TIR durmaz; postacı hemen diğer zarfı açar. 1 milyon zarfı cebinde taşıyabilirsin, bilgisayarının nefesi bile kesilmez!

🇬🇧 ENGLISH (ELI5)

A Platform Thread is like chartering an entire 18-wheel cargo truck for every single envelope you mail. You run out of parking spots after a few thousand trucks, each consumes 1MB-2MB of stack space, and switching drivers is an expensive OS kernel interrupt. Virtual Threads are the individual letters inside a handful of cargo trucks. If a letter is waiting for a signature, the truck driver simply sets it aside and reads the next letter. You can spawn 1,000,000 virtual threads without breaking a sweat!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot M:N Fiber Tasarımı: 1) Bellek Verimliliği: Klasik platform thread'leri OS düzeyinde rezerve edilen 1 MB - 2 MB sabit yığın (stack) alanı tüketir. 10.000 thread 10-20 GB RAM demektir. Sanal thread'ler ise JVM heap alanında yaşar; ilk oluşturklarında yalnızca birkaç yüz baytlık bir Java nesnesidir (`java.lang.VirtualThread`). Yığınları çağrı derinliğine göre dinamik olarak büyür ve küçülür. 2) M:N Eşleme: Milyonlarca sanal thread (M), CPU çekirdek sayısı kadar platform thread'i (`Carrier Thread` - N) üzerinde ForkJoinPool zamanlayıcısıyla çalıştırılır. 3) Thread-per-Request Paradigması: Asenkron callback cehennemine (reactive spaghetti) girmeden, tamamen senkron ve okunabilir kod yazarak reaktif mimarilerin devasa verimine ulaşılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot M:N Fiber Architecture: 1) Memory Footprint: Platform threads bind 1:1 to OS kernel threads, reserving 1MB to 2MB of fixed contiguous off-heap stack memory. Spawning 10,000 platform threads demands 15 GB RAM and triggers OS thread limits (`OutOfMemoryError: unable to create native thread`). Virtual threads allocate dynamic stack frames directly on the JVM heap (`Continuation`), starting at under 1 KB. 2) M:N Scheduling: Millions of virtual threads (M) are multiplexed over a pool of N kernel carrier threads (typically matching CPU core count) managed by an internal FIFO `ForkJoinPool`. 3) Synchronous Simplicity with Reactive Scale: Engineers retain the familiar thread-per-request paradigm with synchronous stack traces and debuggers while achieving 100K+ concurrent I/O throughput.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: 100.000 Sanal Thread Başlatma ve Hız Karşılaştırması
import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

public class VirtualThreadScaleDemo {
    void main() {
        System.out.println("Java 25 Sanal Thread Testi Başlatılıyor...");
        Instant start = Instant.now();

        // 100.000 eşzamanlı I/O işini Sanal Thread'lerle yürüt
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            IntStream.range(0, 100_000).forEach(i -> {
                executor.submit(() -> {
                    // Bloklayan I/O simülasyonu (Ağ/DB gecikmesi)
                    Thread.sleep(Duration.ofMillis(100));
                    return i;
                });
            });
        } // Executor close() tüm 100.000 thread'in bitmesini bekler!

        Instant end = Instant.now();
        System.out.printf("100.000 sanal thread %.2f saniyede başarıyla tamamlandı!%n",
            Duration.between(start, end).toMillis() / 1000.0);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Sanal thread'leri ASLA havuzlamayın (pool)! newFixedThreadPool(100) gibi sanal thread havuzları açmak ölümcül bir anti-pattendir. Sanal thread'ler o kadar ucuzdur ki her iş için yenisi yaratılmalı ve iş bitince ölmelerine izin verilmelidir (Disposable Task Runners). Sınırlandırma gerekiyorsa havuz değil Semaphore kullanılmalıdır!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

NEVER pool Virtual Threads! Creating a fixed virtual thread pool is a catastrophic anti-pattern. Virtual threads are designed to be ephemeral and disposable: instantiate one per task, let it terminate, and allow GC collection. To constrain concurrency or access to scarce downstream resources (e.g. database connections), use a `Semaphore`, never a thread pool!

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Platform thread'leri 1:1 OS thread'idir ve 1 MB yer kaplar. Sanal thread'ler JVM heap'inde yaşar, yüzlerce bayttır ve milyonlarcası açılabilir.

🎯 Key Takeaways for the Interview

• Platform threads bind 1:1 to costly OS kernel threads. Virtual threads are lightweight heap-allocated fibers multiplexed over carrier threads.

44

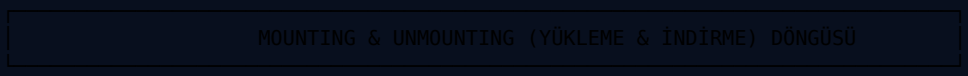
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Carrier Thread, Mounting & Unmounting: Loom Non-Blocking I/O ile Nasıl Park Eder?

Carrier Threads, Mounting & Unmounting: Epoll Non-Blocking I/O Integration

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic




```
Virtual Thread (VT-1) ──► Carrier Thread üzerinde çalışıyor (MOUNTED)
   │
   ▼ socket.read() çağrıldı! (Veri henüz ağdan gelmedi)
JVM Continuation.yield() Tetiklenir:
   1. VT-1'in stack frame'leri HEAP'e kopyalanır (UNMOUNTED).
   2. Soket Epoll / Kqueue olay dinleyicisine kaydedilir.
   3. Carrier Thread anında BOŞA ÇIKAR ve VT-2'yi çalıştırmaya başlar! (0 CPU Kaybı!)
   │
   ▼ Ağdan veri geldi! (Epoll Interrupt)
JVM Zamanlayıcı:
   1. VT-1'in stack frame'leri boşta olan herhangi bir Carrier Thread'e geri yüklenir.
   2. VT-1 kaldığı yerden sanki hiç durmamış gibi devam eder! (MOUNTED Tekrar!)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Satranç şampiyonunun aynı anda 50 kişiyle körleme maç yapmasını düşünün. Şampiyon (Carrier Thread) senin masana gelir, hamleni yapar (Mounting). Sen 'Düşüneyim…' dediğinde (I/O bloklaması), şampiyon masanın başında 10 dakika dikilip seni beklemez! Not defterine pozisyonu yazar ve hemen diğer masaya koşar (Unmounting). Sen hamleni yaptığında elini kaldırırsın, şampiyon boş kaldığı an masana geri dönüp devam eder!

🇬🇧 ENGLISH (ELI5)

Imagine a Grandmaster playing simultaneous chess against 50 opponents. The Grandmaster (Carrier Thread) steps up to your board and evaluates your move (Mounting). If you say 'Let me think for 2 minutes' (Blocking I/O), the Grandmaster does not stand idly waiting! He jots down your board coordinates in a pocket notebook and sprints to the next table (Unmounting). When you finally make your move, you raise your hand; the Grandmaster resumes your match on his next pass!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Continuation ve Non-Blocking Hook İç Mekaniği: 1) Continuation.yield(): Sanal thread bloklayan bir I/O işlemine (ör. `SocketChannel.read()`, `Thread.sleep()`) girdiğinde, HotSpot JDK soketin bloke olduğunu anlar. `VirtualThread.park()` çağrılır ve düşük seviyeli `Continuation.yield()` çalışır. 2) Yığın Ayrıştırma (Stack Freezing): O anki Java çağrı yığını Carrier thread'in C++ yığınından sökülüp JVM Heap'e taşınır. Carrier thread işletim sistemi açısından hiçbir zaman bloklanmaz; ForkJoinPool'daki diğer hazır sanal thread'leri çalıştırmaya devam eder. 3) Epoll/Kqueue Entegrasyonu: JVM'nin dahili Poller thread'i Linux `epoll` veya Mac `kqueue` üzerinden soketin hazır olduğunu duyunca sanal thread'i `RUNNABLE` yapar. 4) Yeniden Yükleme (Thawing): Boş bir carrier thread yığını heap'ten geri yükleyip (thawing) bytecode yürütmesine devam eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Continuation Mechanics and Epoll Integration: 1) `Continuation.yield()`: When a virtual thread encounters blocking I/O (e.g., `Socket.getInputStream().read()`, `LockSupport.park()`), the Java runtime intercepts the call. Instead of performing a kernel syscall that blocks the OS thread, it invokes `VirtualThread.park()`. 2) Stack Freezing: The virtual thread's call frames are copied from the carrier thread's native stack onto the JVM heap. The carrier thread is immediately disassociated (unmounted) and returns to its `ForkJoinPool` work-stealing loop. 3) Poller & Reactor Registration: The file descriptor is registered with HotSpot's global I/O Poller thread (using Linux `epoll` or macOS `kqueue`). 4) Thawing & Resumption: Upon I/O readiness, the Poller re-enqueues the virtual thread. An available carrier thread thaws the stack frames from the heap and resumes instruction execution transparently.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Carrier Thread Geçişini Gösteren Canlı Test
import java.util.concurrent.Executors;

public class CarrierThreadHandoffDemo {
    void main() throws InterruptedException {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            executor.submit(() -> {
                // I/O öncesi Carrier Thread kimliğini yazdır
                System.out.println("1. Bloklama Öncesi: " + Thread.currentThread());

                try {
                    // Bu çağrı unmount eder; Carrier Thread boşa çıkar!
                    Thread.sleep(200);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }

                // I/O sonrası: Farklı bir Carrier Thread tarafından mount edilmiş olabilir!
                System.out.println("2. Bloklama Sonrası: " + Thread.currentThread());
            });
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Sanal thread'ler CPU-bound (yoğun matematik, video işleme, büyük veri sıkıştırma) işler için HİÇBİR performans artışı sağlamaz! Aksine unmount/mount overhead'i yüzünden platform thread'lerinden daha yavaş kalırlar. Sanal thread'lerin tek süper gücü I/O-bound (Ağ, DB, Disk, REST API) bloklamalarını sıfır maliyete indirmektir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Virtual threads provide ZERO throughput gains for CPU-bound computational workloads (e.g. video rendering, cryptography, dense matrix math). Context freezing and thawing actually introduces measurable overhead. Virtual threads are designed exclusively to eliminate concurrency ceilings caused by blocking I/O (databases, HTTP microservices, message queues).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sanal thread bloklandığında (I/O) Continuation.yield() ile unmount edilir, Carrier thread boşa çıkar. Epoll uyanınca yeniden mount edilir.

🎯 Key Takeaways for the Interview

• When a virtual thread blocks on I/O, `Continuation.yield()` unmounts its stack to the heap, freeing the carrier thread. Epoll notifications re-mount it seamlessly.

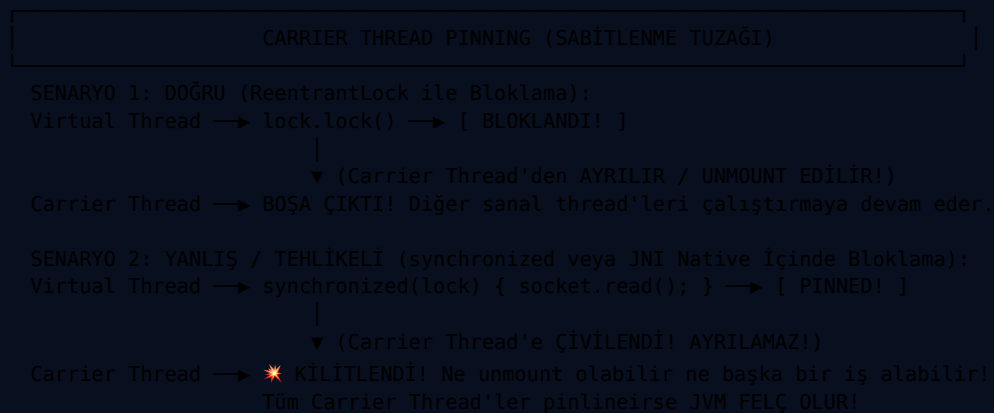
45

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

The Pinning Hazard (Sabitlenme Tehlikesi): synchronized vs ReentrantLock & Native Kod Tuzakları

The Pinning Hazard: synchronized vs ReentrantLock & Native Methods

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Sanal thread bir posta güvercini gibidir; mektubu bırakıp hemen uçar. Ancak güvercinin ayağına ağır bir demir gülle bağlarsan uçamaz. 'Pinning' tam olarak budur: Eğer sanal thread `synchronized` bloğu veya eski C kütüphanesi (JNI) içindeyken ağdan veri beklerse, altındaki devasa platform thread'e çivilenir! Platform thread kımıldayamaz hale gelir. 16 çekirdeğin varsa ve 16 sanal thread pinlineirse bütün sunucu kilitlenir!

🇬🇧 ENGLISH (ELI5)

A virtual thread is an agile gymnast capable of leaping off the balance beam at any moment. Carrier Pinning is like super-gluing the gymnast's feet to the beam. If a virtual thread enters a blocking I/O operation inside a legacy `synchronized` block or a native JNI C-library call, it cannot unmount! The underlying OS carrier thread is held hostage. If all carrier threads become pinned, the entire JVM throughput collapses to zero!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Pinning Mekanizması ve Çözümler: 1) Neden Sabitlenir (Pinning)? Bir thread native C frame'i (`JNI`) yürütüyorsa veya bir nesnenin monitörüne (`synchronized`) sahipken bloklayan bir işlem yaparsa, HotSpot C++ çalışma zamanı monitör sahipliğini native stack'ten ayıramaz. Buna `Pinned Carrier Thread` denir. 2) Sonuçları: Carrier thread unmount edilemez; diğer binlerce sanal thread kuyrukta açlıktan ölür (starvation). HotSpot telafi etmek için geçici olarak `ForkJoinPool`'a ek carrier thread ekler, ancak bu OS thread patlamasına yol açar. 3) Teşhis: JVM `-Djdk.tracePinnedThreads=full` veya `short` parametresiyle başlatılarak pinlineen yerler loglanır. 4) Çözüm: Bloklayan I/O içeren tüm `synchronized` blokları derhal `java.util.concurrent.locks.ReentrantLock` ile değiştirilmelidir. (Not: Java 24/25'te synchronized pinning'i büyük ölçüde hafifletilmiştir ancak native JNI ve kritik bölümler için ReentrantLock endüstri standardıdır).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Pinning Internals & Diagnostic Protocol: 1) Root Cause: A virtual thread cannot be unmounted when its call frame contains native frames (JNI) or when holding an object monitor inside a `synchronized` block/method that enters `park()`. The HotSpot native frame metadata is bound to the carrier's OS stack. 2) Catastrophic Degradation: With all carrier threads pinned, the `ForkJoinPool` cannot service pending virtual threads. HotSpot attempts compensatory thread injection, spiking OS kernel thread counts and memory consumption. 3) Diagnostics: Enable JVM flag `-Djdk.tracePinnedThreads=full` to print stack traces whenever pinning occurs. 4) Modern Remediation: Replace blocking `synchronized` sections with `ReentrantLock`, which relies on `LockSupport.park()` and fully unmounts virtual threads without pinning.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Pinning Tuzağı ve ReentrantLock ile Güvenli Dönüşüm
import java.util.concurrent.Executors;
import java.util.concurrent.locks.ReentrantLock;

public class PinningPreventionDemo {
    private final ReentrantLock modernLock = new ReentrantLock();
    private final Object legacyLock = new Object();

    // ❌ TEHLİKELİ ESKİ YÖNTEM: synchronized içinde I/O (Pinning Yaratır!)
    public void hazardousMethod() {
        synchronized (legacyLock) {
            try {
                // Pinning alarmı! Carrier thread unmount olamaz!
                Thread.sleep(100);
            } catch (InterruptedException _) {}
        }
    }

    // ✅ GÜVENLİ MODERN YÖNTEM: ReentrantLock (Carrier Thread Serbest Kalır!)
    public void resilientMethod() {
        modernLock.lock();
        try {
            // Virtual thread unmount olur, carrier thread serbestçe diğer işleri yapar!
            Thread.sleep(100);
        } catch (InterruptedException _) {
            Thread.currentThread().interrupt();
        } finally {
            modernLock.unlock();
        }
    }

    void main() {
        System.setProperty("jdk.tracePinnedThreads", "short");
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 5; i++) {
                executor.submit(this::resilientMethod);
            }
        }
        System.out.println("Tüm sanal thread'ler sıfır pinning ile başarıyla tamamlandı!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Eski JDBC sürücüleri ve üçüncü parti kütüphanelerin derinliklerinde socket okumaları synchronized blokları içine sarılmış olabilir! Projenizi Sanal Thread'lere geçirirken kesinlikle `-Djdk.tracePinnedThreads=full` bayrağını açıp harici kütüphanelerdeki pinning noktalarını denetleyin.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Legacy JDBC drivers, XML parsers, and HTTP clients often wrap socket reads in `synchronized` blocks. When migrating production services to Virtual Threads, always audit dependencies with `-Djdk.tracePinnedThreads=full` to eliminate stealth carrier thread starvation.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• synchronized veya JNI içinde I/O yapmak Carrier Thread'i kilitler (Pinning). Bloklayan I/O bölgelerinde daima ReentrantLock kullanın.

🎯 Key Takeaways for the Interview

• Blocking I/O inside `synchronized` or JNI pins the carrier thread. Replace `synchronized` blocks wrapping I/O with `ReentrantLock`.

46

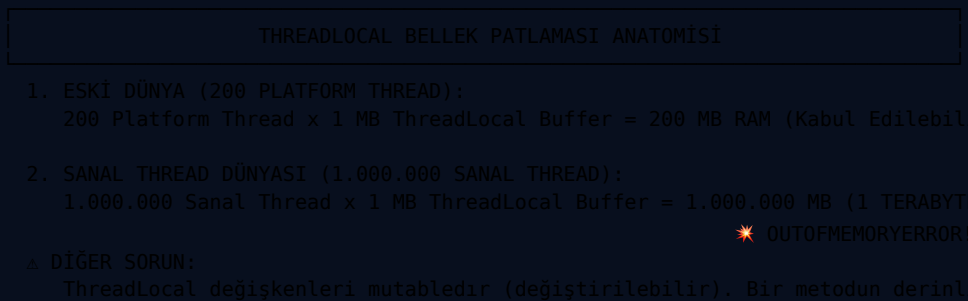
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

ThreadLocal Tehlikeleri: Sanal Thread Dünyasında Milyonluk Bellek Patlaması

ThreadLocal Perils: High-Scale Memory Explosions in Virtual Thread Worlds

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette 10 çalışan varken herkese 1'er adet koca bir takım çantası vermek sorun olmaz (10 çanta yer tutmaz). Ama yarın şirkete 1 milyon geçici stajyer girerse ve her stajyere 20 kiloluk özel bir takım çantası (`ThreadLocal`) verirsen bina çöker! Sanal thread'ler milyonlarcadır; her birine özel ağır yükler yüklersen sunucunun belleği saniyeler içinde patlar!

🇬🇧 ENGLISH (ELI5)

Equipping 50 long-tenured field engineers with a heavy 20kg diagnostic toolkit (`ThreadLocal`) is fine. But if you suddenly deploy 1,000,000 ephemeral seasonal interns and issue each one an identical 20kg toolkit, your warehouse immediately collapses under the weight! Virtual threads scale to millions; binding heavy stateful data to each thread triggers catastrophic OutOfMemoryErrors.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ThreadLocal Anti-Pattern'inin Sanal Thread Çöküşü: 1) Bellek Çarpımı: Geleneksel Spring/Jakarta EE mimarilerinde güvenlik bağlamı (`SecurityContext`), veritabanı bağlantısı veya transaction bilgisi `ThreadLocal` içinde saklanır. Platform thread havuzlarında bu nesneler 200-500 thread arasında tekrar tekrar kullanılır. Ancak sanal thread dünyasında her HTTP isteği yeni bir sanal thread açar. 1 milyon istek geldiğinde 1 milyon adet kopya oluşur ve heap çöker. 2) Temizlik Zorluğu: Sanal thread'ler hızla ölüp gitse de, yaşam süreleri boyunca tuttukları nesneler GC üzerinde devasa yük yaratır. 3) Korumasız Değiştirilebilirlik: `ThreadLocal.set()` çağrısı herhangi bir alt fonksiyon tarafından çağrılabilir; bu da yan etkilerin takip edilmesini zorlaştırır. Çözüm: Java 25'te `ScopedValue` mimarisine geçmektir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Virtual Thread / ThreadLocal Collision: 1) Multiplicative Memory Bloat: Traditional enterprise frameworks cache connection contexts, cryptographic buffers, and request metadata in `ThreadLocal`. In platform thread pools (200 threads), memory is bounded. In virtual thread architectures handling 500,000 concurrent requests, allocating per-thread buffers multiplies memory consumption by $500,000$, resulting in immediate `OutOfMemoryError`. 2) GC Pressure: Ephemeral virtual threads creating and releasing heavy ThreadLocal entries saturate generational GC collectors. 3) Unconstrained Mutability: `ThreadLocal.set()` permits arbitrary downstream code to mutate context invisibly. Modern architectures mandate migrating to immutable `ScopedValue`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ThreadLocal Bellek Tuzağı vs Güvenli Sanal Thread Kullanımı
import java.util.concurrent.Executors;

public class ThreadLocalHazardDemo {
    // ⚠ TEHLİKE: Her sanal thread için 1 MB'lık buffer açan ThreadLocal!
    private static final ThreadLocal<byte[]> heavyBuffer =
        ThreadLocal.withInitial(() -> new byte[1024 * 1024]); // 1MB

    void main() {
        System.out.println("ThreadLocal Tehlikesi Simülasyonu...");

        // Sanal thread oluştururken ThreadLocal'ı devre dışı bırakma seçeneği!
        Thread.Builder.OfVirtual builder = Thread.ofVirtual()
            .name("safe-vt-", 0)
            .inheritInheritableThreadLocals(false); // Miras almayı kapat

        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 1000; i++) {
                executor.submit(() -> {
                    // Kötü niyetli veya dikkatsiz kod bu buffer'ı çağırırsa:
                    // byte[] buffer = heavyBuffer.get(); -> 1000 MB RAM harcar!
                    // Güvenli yaklaşım: İş bitince buffer.remove() veya ScopedValue!
                });
            }
        }
        System.out.println("Bellek güvenliği sağlandı.");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Eski kütüphanelerde `SimpleDateFormat` veya `Jackson ObjectMapper` gibi pahalı nesnelerin ThreadLocal ile paylaşıldığı görülür. Sanal thread kullanırken bu kütüphanelerin güncel thread-safe sürümlerine geçtiğinizden veya nesneleri static final/ScopedValue olarak tanımladığınızdan emin olun.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Legacy codebases frequently use `ThreadLocal` to cache non-thread-safe instances like `SimpleDateFormat` or JSON mappers. In virtual thread environments, this causes memory exhaustion. Upgrade to modern thread-safe abstractions or immutable `ScopedValue` containers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Platform thread'lerinde 200 adet ThreadLocal zararsızdır; 1 milyon sanal thread'de aynı kod Terabaytlarca RAM tüketip OOM patlatır.

🎯 Key Takeaways for the Interview

- ThreadLocal is benign across 200 platform threads, but fatal across 1,000,000 virtual threads, multiplying memory to multi-gigabyte exhaustion.

47

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Structured Concurrency: StructuredTaskScope ile Kaotik Thread'leri Hizaya Sokma

Structured Concurrency: StructuredTaskScope (ShutdownOnFailure & Success)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                UNSTRUCTURED VS STRUCTURED CONCURRENCY        │
└──────────────────────────────────────────────────────────────┘

1. KAOTİK (UNSTRUCTURED):
   Ana Görev ──┬──► Alt Görev 1 (Uçak Bileti) ──► [ HATA PATLADI! 💥 ]
               └──► Alt Görev 2 (Otel Rezerv.) ──► [ YETİM KALDI! (Sonsuza dek çalışır!) ]
   ⚠ Ana metod dönse bile Alt Görev 2 arka planda para ve CPU harcamaya devam eder!

2. YAPISAL (STRUCTURED - StructuredTaskScope):
   try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
       scope.fork(altGorev1); // Sanal thread'de çalışır
       scope.fork(altGorev2); // Sanal thread'de çalışır
       scope.join();          // Birlikte başlarlar, birlikte biterler!
       scope.throwIfFailed(); // Biri hata verirse DİĞERLERİ OTOMATİK İPTAL EDİLİR!
   } // Scope kapanırken yetim thread kalması İMKÂNSIZDIR!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski asenkron kod, 3 çocuğunu lunaparkın ortasında serbest bırakıp eve dönen bir ebeveyne benzer. Biri kaybolsa diğerleri akşama kadar dönme dolapta boşuna para harcar (yetim thread). Structured Concurrency ise çocukların ellerini tutarak parka girmektir. Hep birlikte girersiniz, hep birlikte çıkarsınız. Birinin ayağı takılıp düşerse diğer ikisini de hemen durdurup eve dönersiniz; kimse kaybolmaz, para israf olmaz!

🇬🇧 ENGLISH (ELI5)

Unstructured concurrency is dropping three kids off at an amusement park and driving home. If one breaks an arm, the other two keep wandering and riding rollercoasters indefinitely (orphaned threads). Structured Concurrency is holding their hands together in a single group. You enter together and leave together. If one gets injured, you immediately cancel the outing for everyone. No thread is ever abandoned in the wild!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

StructuredTaskScope ve Yaşam Döngüsü Yönetimi: 1) Yapısal Programlama Disiplini: Tıpkı `if-else` veya `for` döngülerinin tek bir giriş ve çıkış noktası olması gibi, eşzamanlı görevlerin de aynı blokta başlayıp aynı blokta sonlanması kuralıdır (Lexical Scoping). 2) Kısa Devre İptali (Short-Circuit Cancellation): `ShutdownOnFailure` politikasında alt görevlerden biri başarısız olursa, kapsamdaki diğer tüm kardeş görevler otomatik olarak `interrupt` edilir; boşuna ağ ve CPU tüketilmez. 3) İlk Kazanan Politikası (`ShutdownOnSuccess`): Dağıtık bir kütüphanede aynı veriyi 3 farklı yedekten ararken ilk geçerli yanıtı alan görev kazanır, diğer 2 sorgu anında iptal edilir. 4) Hata Ayıklama (Observability): Thread dökümlerinde (Thread Dump) hiyerarşik ebeveyn-çocuk ilişkisi net biçimde görünür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

StructuredTaskScope Mechanics: 1) Lexical Scoping for Concurrency: Paralleling structured control flows (`if/while`), concurrent tasks are bound to explicit lexical scopes. Subtasks spawned within a block are guaranteed to terminate before the block exits. 2) Eliminating Thread Leakage: In unstructured models (`CompletableFuture.supplyAsync`), failed tasks leave sibling tasks running as invisible orphans. `StructuredTaskScope.ShutdownOnFailure` automatically intercepts exceptions, issuing immediate cancellation interrupts to surviving sibling threads. 3) Hedged Optimization via `ShutdownOnSuccess`: In multi-replica queries, the first subtask to return resolves the scope, aborting remaining parallel requests. 4) Observability: Thread dumps reflect pristine hierarchical parent-child trees rather than flat unorganized lists.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: StructuredTaskScope.ShutdownOnFailure ile Güvenli Tatil Rezervasyonu
import java.util.concurrent.StructuredTaskScope;
import java.util.function.Supplier;

public class StructuredReservationService {
    record Flight(String flightNo, double price) {}
    record Hotel(String hotelName, double price) {}
    record VacationPackage(Flight flight, Hotel hotel) {}

    static Flight bookFlight() throws InterruptedException {
        Thread.sleep(100);
        return new Flight("TK-1923", 450.0);
    }

    static Hotel bookHotel() throws InterruptedException {
        Thread.sleep(80);
        return new Hotel("Grand Hyatt", 600.0);
    }

    public VacationPackage bookVacation() throws Exception {
        // Alt görevlerden biri patlarsa diğerini otomatik iptal et!
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            // İki bağımsız görevi sanal thread'ler üzerinde çatalla (fork)
            Supplier<Flight> flightSubtask = scope.fork(StructuredReservationService::bookFlight);
            Supplier<Hotel> hotelSubtask = scope.fork(StructuredReservationService::bookHotel);

            scope.join();           // İki görevin bitmesini bekle
            scope.throwIfFailed();  // Biri hata verdiyse burada fırlat!

            // Her ikisi de başarılı oldu; sonuçları birleştir
            return new VacationPackage(flightSubtask.get(), hotelSubtask.get());
        } // Scope çıkışında hiçbir thread açık kalamaz!
    }

    void main() throws Exception {
        var service = new StructuredReservationService();
        VacationPackage trip = service.bookVacation();
        System.out.println("Rezervasyon Başarılı: " + trip);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

StructuredTaskScope try-with-resources bloğunda join() çağırmayı unutursanız IllegalStateException fırlatılır! Java çalışma zamanı, tüm alt thread'lerin sonucunu beklemeden scope'tan çıkmanıza ASLA izin vermez.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Attempting to retrieve subtask results via `.get()` prior to calling `scope.join()` throws an `IllegalStateException`. StructuredTaskScope strictly enforces that scopes cannot exit until all child threads have settled.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

• StructuredTaskScope ile alt görevler tek bir blokta başlar ve biter. Biri patlarsa kardeş görevler otomatik iptal edilir, yetim thread kalmaz.

• StructuredTaskScope enforces lexical task lifecycles: tasks begin and end together. Failure in one sibling immediately cancels the rest.

48

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

ScopedValue Devrimi: Java 25 ile Değişmez, Miras Alınabilir & Sıfır Bellek Bağlamı

ScopedValue in Java 25: Immutable, Inheritable, Zero-Allocation Context

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────────────┐
│                    SCOPEDVALUE VS THREADLOCAL                        │
└──────────────────────────────────────────────────────────────────────┘

Özellik                 | ThreadLocal                    | ScopedValue (Java 25)
------------------------|--------------------------------|-----------------------------------
Değiştirilebilirlik     | Mutable (Herkes değiştirebilir)| IMMUTABLE (Kesinlikle Değişmez!)
Yaşam Döngüsü           | Belirsiz (remove() unutulur)   | Sınırlı (Scope kapanınca yok olur)
Bellek Maliyeti         | Her thread için ayrı kopya!    | SIFIR KOPYA (Paylaşımlı Referans)
Sanal Thread Uyumu      | Kötü (OOM Riski)               | MÜKEMMEL (Milyonlarca VT destekler)

Kullanım Modeli:
ScopedValue.where(TENANT_ID, "ACME_CORP")
           .run(() -> {
               // Sadece bu blok ve altındaki sanal thread'ler okuyabilir!
               service.handleRequest();
           }); // Blok bittiğinde bağlam otomatik temizlenir!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

ThreadLocal, şirkette her personelin cebine koyduğu ve canı isteyenin üzerine kurşun kalemle yazı yazabildiği bir kâğıttır. Biri kâğıdı silmeyi unutursa sonraki vardiyadaki kişi yanlış bilgiyle çalışır. ScopedValue ise toplantı odasının kapısına asılan mühürlü bir levhadır: 'Bu odadayken şirket ACME Corp'tur'. İçerideki herkes levhayı okuyabilir ama kimse değiştiremez. Odadan çıktığınız anda levha geçerliliğini yitirir; kimsenin cebinde çöp kalmaz!

🇬🇧 ENGLISH (ELI5)

`ThreadLocal` is handing every employee an erasable notepad that anyone can scribble on at any time. If someone forgets to erase it, the next shift inherits corrupted state. `ScopedValue` is an immutable badge worn strictly during a specific meeting. Everyone inside the room can inspect the badge, no one can tamper with it, and the moment you step outside the door, the badge dissolves into thin air. Zero leaks, zero copies!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ScopedValue Mimarisi ve JVM Optimizasyonları: 1) Değişmezlik (Immutability): `ScopedValue\<T>` değerleri bağlandıktan sonra (`where`) asla değiştirilemez (`set` metodu yoktur). Yalnızca içiçe yeni bir kapsam (`rebinding`) açılabilir. 2) Dinamik Kapsam (Dynamic Scoping): Değer sadece `run` veya `call` bloğu yürütülürken mevcuttur; blok bittiğinde GC otomatik olarak erişim bağlarını serbest bırakır; manuel `remove()` çağırma gerekliliği ve bellek sızıntısı riski ortadan kalkar. 3) Sıfır Tahsisli Miras (Zero-Allocation Inheritance): Alt sanal thread'ler (`StructuredTaskScope`) çatallandığında ana thread'in ScopedValue değerlerini kopyalamaz; aynı değişmez referansı doğrudan paylaşırlar. Bu da milyonlarca sanal thread açıldığında bellek tüketimini sıfıra indirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot ScopedValue Architecture & Mechanical Sympathy: 1) Strict Immutability: Unlike `ThreadLocal`, `ScopedValue\<T>` exposes no `.set()` method. Once bound via `ScopedValue.where()`, the value is read-only for the duration of the callable invocation. Nested scopes may dynamically shadow (rebind) values without mutating parent states. 2) Bounded Extent: The context lifetime strictly mirrors the execution lifetime of the enclosing lambda, guaranteeing deterministic cleanup without relying on defensive `.remove()` invocations. 3) Zero-Copy Fork Inheritance: When child virtual threads are spawned via `StructuredTaskScope`, they inherit access to enclosing scoped values without duplicating map entries or allocating heap buffers. HotSpot optimizes ScopedValue lookups using JIT-compiled intrinsic cache lines.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ScopedValue ile Güvenli ve Değişmez Güvenlik Bağlamı Taşıma
import java.util.concurrent.StructuredTaskScope;

public class ScopedValueModernDemo {
    // Statik ve değişmez ScopedValue tanımları
    public static final ScopedValue<String> TENANT_ID = ScopedValue.newInstance();
    public static final ScopedValue<String> USER_ROLE = ScopedValue.newInstance();

    static void executeDatabaseQuery() {
        // İhtiyaç duyulan yerde bağlamı anında ve güvenle oku!
        System.out.printf("[DB] Tenant: %s | Role: %s | Thread: %s%n",
            TENANT_ID.get(), USER_ROLE.get(), Thread.currentThread());
    }

    void main() throws Exception {
        // Bağlamı sınırlı bir kapsamda (scope) bağla ve çalıştır
        ScopedValue.where(TENANT_ID, "FINTECH_CORP_01")
                   .where(USER_ROLE, "ADMIN")
                   .run(() -> {
                       System.out.println("Kapsam içine girildi...");
                       executeDatabaseQuery();

                       // Alt sanal thread'ler de bu bağlamı sıfır maliyetle paylaşır!
                       try (var scope = new StructuredTaskScope<Void>()) {
                           scope.fork(() -> {
                               executeDatabaseQuery();
                               return null;
                           });
                           scope.join();
                       } catch (InterruptedException _) {}
                   });

        // Kapsam dışına çıkıldığında değerler otomatik olarak erişilemez olur!
        System.out.println("Kapsam bitti. TENANT_ID bağlı mı? " + TENANT_ID.isBound());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ScopedValue.get() metodunu where() ile bağlanmamış bir kapsamda çağırırsanız NoSuchElementException fırlatılır! Güvenli kontrol için isBound() veya orElse() metotlarını kullanabilirsiniz.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Invoking `ScopedValue.get()` outside of an active `.where()` binding scope throws an immediate `NoSuchElementException`. Always guard optional scopes using `.isBound()` or provide a default via `.orElse(defaultValue)`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• ScopedValue değişmezdir, kapsamı belirlidir ve miras alındığında sıfır bellek tüketir. ThreadLocal'ın modern ve güvenli alternatifidir.

🎯 Key Takeaways for the Interview

• ScopedValue is immutable, lexically bounded, and zero-allocation upon thread inheritance, replacing hazardous ThreadLocal in modern Java.

BÖLÜM 9 / MODULE 9

Eşzamanlılık Hataları, Teşhis & Düşük Seviyeli Primitifler

Concurrency Bugs, Anti-Patterns & Low-Level Primitives

Klasik yarış durumları (Check-then-Act, Read-Modify-Write), Coffman ölümcül kilitlenme (Deadlock) koşulları ve tespiti, Livelock ve Starvation, ABA problemi ve AtomicStampedReference, jstack/jcmd thread dump adli analizi, ve modern VarHandle bellek çitleri.

Classic race conditions (Check-then-Act, Read-Modify-Write), Coffman deadlock prerequisites and programmatic detection, Livelock and thread starvation, ABA memory hazard and AtomicStampedReference, production thread dump forensics with jstack/jcmd, and modern VarHandle memory fences.

49

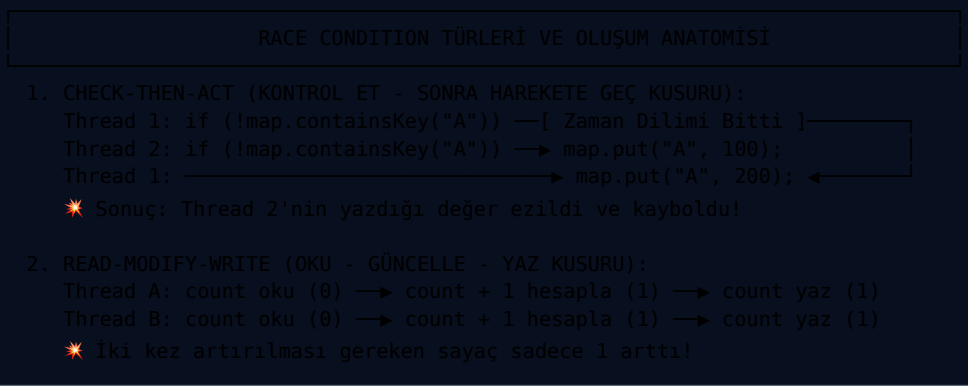
BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Klasik Yarış Durumları (Race Conditions): Check-Then-Act ve Read-Modify-Write Kusurları

Classic Race Conditions: Check-Then-Act & Read-Modify-Write Flaws

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Check-then-act, banyoya girmeden önce kapıyı tıklatıp içeride kimse olmadığını görmek, tam içeri adım atacakken başka birinin yan kapıdan hızla banyoya girmesi gibidir. Sen 'Banyo boştu!' diyerek içeri dalarsın ve felaket yaşanır! Kontrol anı ile eylem anı arasında geçen mikrosaniyelik boşlukta evren değişmiştir. Read-modify-write ise aynı anda iki kişinin buzdolabında kalan son yumurtayı görüp ikisinin de tavaya kırmasıdır; sonunda yumurta yok olur!

🇬🇧 ENGLISH (ELI5)

Check-then-Act is knocking on a dressing room door, confirming it is empty, and pausing for one second before stepping in. In that one split-second, someone else slips inside. You burst in thinking it is still vacant, causing chaos. The state observed during the 'check' was invalidated before the 'act'. Read-Modify-Write is two cooks reading an inventory sheet showing 1 egg left, both taking that egg to bake a cake simultaneously, resulting in missing inventory!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kritik Bölge Zafiyetleri ve Atomik Çözümler: 1) Check-Then-Act: En yaygın örneği `Map.containsKey()` ardından `Map.put()` yapılması veya tembel yüklemedeki (Lazy Init) `if (instance == null)` kontrolüdür. Kontrol ile yazma arasında atomik bir bağ yoktur. Eşzamanlı ortamda durum başka bir thread tarafından değiştirilebilir. Çözüm: `ConcurrentMap.putIfAbsent()` veya `computeIfAbsent()` gibi donanımsal CAS tabanlı atomik primitifler kullanmaktır. 2) Read-Modify-Write: `count++` işlemi aslında 3 bytecode komutudur (`getfield`, `iadd`, `putfield`). İki thread aynı anda `getfield` yaparsa biri diğerinin artırmasını ezer. Çözüm: `AtomicInteger.incrementAndGet()` veya `synchronized` kilididir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Critical Section Vulnerabilities: 1) Check-Then-Act: Arises when a state inspection informs a subsequent mutation, but the observed state becomes stale before the mutation executes. Common in compound collections (`if (map.containsKey(k)) map.put(k, v)`). Resolution: Leverage compound atomic methods like `ConcurrentMap.putIfAbsent()` or `computeIfAbsent()`. 2) Read-Modify-Write: Manifests in expressions like `count++`, which compile down to three discrete bytecode instructions (`getfield`, `iadd`, `putfield`). When interleaved, concurrent threads read identical initial values, discarding increments. Resolution: Atomic CAS wrappers (`AtomicInteger.incrementAndGet()`) or mutual exclusion locks.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Check-then-Act Hatası ve Modern Atomik Çözümü
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Executors;

public class RaceConditionDemo {
    private final ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();

    // ❌ HATALI: Check-then-act yarış durumu (Compound Operation)
    public void unsafePut(String key, String value) {
        if (!cache.containsKey(key)) { // Kontrol
            // ⚠ TAM BURADA başka bir thread araya girip key'i ekleyebilir!
            cache.put(key, value);     // Eylem
        }
    }

    // ✅ DOĞRU: Tek adımda atomik işlem (Atomic Compound)
    public void safePut(String key, String value) {
        cache.putIfAbsent(key, value); // JVM ve Donanım seviyesinde atomik!
    }

    void main() {
        var demo = new RaceConditionDemo();
        try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 1000; i++) {
                final int id = i;
                pool.submit(() -> demo.safePut("CONFIG_KEY", "VAL_" + id));
            }
        }
        System.out.println("Nihai güvenli değer: " + demo.cache.get("CONFIG_KEY"));
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ConcurrentHashMap kullanmak tek başına tüm yarış durumlarını ÇÖZMEZ! Eğer haritadan okuyup (get), bir hesaplama yapıp, sonra tekrar yazıyorsanız (put), iki metodun arası atomik değildir ve yarış durumu yaşanır. Her zaman compute(), computeIfAbsent() veya merge() kullanmalısınız!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Merely wrapping state in `ConcurrentHashMap` does NOT immunize against race conditions! If your code performs `map.get()`, calculates a new value, and issues `map.put()`, the interval between calls is unguarded. You must encapsulate compound state transitions within `compute()` or `merge()`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Birden fazla adımdan oluşan işlemlerde araya başka thread girebilir. Kontrol ve eylem mutlaka tek bir atomik operasyonda birleştirilmelidir.

🎯 Key Takeaways for the Interview

• Compound actions without isolation permit thread interleaving. Merge check-and-act steps into single atomic operations.

50

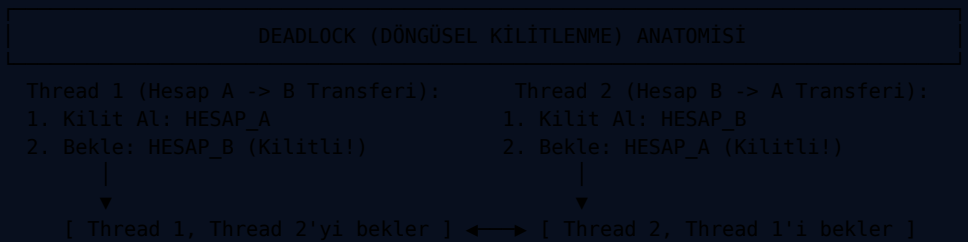
BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Deadlock (Ölümcül Kilitlenme) Forenziği: Coffman Koşulları, Kilit Sıralaması & ThreadMXBean

Deadlock Forensics: Coffman Conditions, Lock Ordering & JVM Detection

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki inatçı keçinin tek şeritli dar bir tahta köprüde kafa kafaya gelmesini düşünün. Biri 'Önce sen geri adım at', diğeri 'Hayır önce sen çekil' der. İkisi de yolu bırakmaz ve ikisi de köprüyü geçemez. Açlıktan ölene kadar orada dikilirler. Deadlock tam olarak budur: Thread 1, Thread 2'nin elindeki kilidi bekler; Thread 2 ise Thread 1'in elindekini bekler; sistem sonsuza dek donar!

🇬🇧 ENGLISH (ELI5)

Two stubborn goats meeting in the middle of a narrow single-lane bridge. Goat A demands Goat B back up; Goat B insists Goat A retreat first. Neither yields, neither moves forward, and both starve to death on the bridge. In Java Deadlock, Thread 1 holds Lock A and waits for Lock B; Thread 2 holds Lock B and waits for Lock A. Neither releases their lock, paralyzing both forever.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Coffman'ın 4 Ölümcül Koşulu ve Çözüm Yolları: Deadlock oluşması için 4 şartın AYNI ANDA gerçekleşmesi gerekir: 1) Mutual Exclusion (Karşılıklı Dışlama): Kaynak sadece tek bir thread tarafından tutulabilir. 2) Hold and Wait (Tut ve Bekle): Thread bir kaynağı tutarken diğerini bekler. 3) No Preemption (Zorla Alma Yok): Kaynak thread'in elinden zorla alınamaz. 4) Circular Wait (Döngüsel Bekleme): Thread'ler kapalı bir döngü zinciri oluşturur (T1->T2->T3->T1). Bu 4 şarttan HERHANGİ BİRİ kırılırsa deadlock imkânsız hale gelir! Endüstride en popüler çözüm: Kilit Sıralaması (Lock Ordering / Global Hierarchy) kuralıdır. Nesne kimliklerine (ör. Account ID veya System.identityHashCode) göre kilitler daima küçükten büyüğe sıralı alınır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Coffman Conditions & Systematic Remediation: Deadlock requires simultaneous presence of 4 prerequisites: 1) Mutual Exclusion: Resource held exclusively. 2) Hold and Wait: A thread holding a resource requests additional resources. 3) No Preemption: Resources cannot be forcibly expropriated. 4) Circular Wait: A closed chain of threads exists where each waits for a resource held by the next. Breaking ANY ONE condition prevents deadlock. Industry Standard Defense: Global Lock Ordering. By enforcing that locks are ALWAYS acquired in a strictly monotonic hierarchy (e.g. by comparing unique entity IDs or `System.identityHashCode`), circular wait becomes mathematically impossible.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Sıralı Kilitleme (Lock Ordering) ile Deadlock'tan Kaçınma
import java.util.concurrent.locks.ReentrantLock;

public class DeadlockFreeTransfer {
    record Account(long id, ReentrantLock lock) {}

    public static void transfer(Account from, Account to, double amount) {
        // Kilit sıralaması: Her zaman ID'si KÜÇÜK olanı önce kilitle!
        Account firstLock = from.id() < to.id() ? from : to;
        Account secondLock = from.id() < to.id() ? to : from;

        firstLock.lock().lock();
        try {
            secondLock.lock().lock();
            try {
                System.out.printf("Transfer yapıldı: %d -> %d tutar: %.2f%n",
                    from.id(), to.id(), amount);
            } finally {
                secondLock.lock().unlock();
            }
        } finally {
            firstLock.lock().unlock();
        }
    }

    void main() {
        var accA = new Account(1001, new ReentrantLock());
        var accB = new Account(2002, new ReentrantLock());

        // Ters yönlü transferler aynı anda gelse bile ASLA deadlock olmaz!
        transfer(accA, accB, 50.0);
        transfer(accB, accA, 30.0);
        System.out.println("Transferler güvenle tamamlandı.");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

tryLock(timeout) kullanarak deadlock riskini azaltabilirsiniz; ancak kilit alınamadığında daha önce alınan kilitleri bırakmayı (unlock) unutursanız livelock yaratırsınız. Kilit sıralaması (Lock Ordering) matematiksel olarak en sağlam yoldur.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mitigating deadlocks via `tryLock(timeout)` is fragile: if a thread fails to acquire the second lock but retains the first, it may induce Livelock. Monotonic lock ordering remains the definitive architectural defense.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Deadlock döngüsel kilit beklemesinden doğar. Kilitleri daima sabit bir sıralamayla (küçük ID'den büyük ID'ye) alarak döngüyü kırın.

🎯 Key Takeaways for the Interview

- Deadlock stems from circular waiting. Enforce strict lock ordering across all execution paths to break the cycle mathematically.

51

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Livelock, Thread Starvation & Priority Inversion: Sessiz Katiller

Livelock, Thread Starvation & Priority Inversion: The Silent Killers

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                    LIVELOCK VS STARVATION VS DEADLOCK

1. DEADLOCK (ÖLÜ):
   İki thread tamamen UYUR (BLOCKED). CPU kullanımı = %0. İlerleme = 0.

2. LIVELOCK (AŞIRI HAREKETLİ AMA SONUÇSUZ):
   İki thread sürekli birbirine yol verir. CPU kullanımı = %100! İlerleme = 0.
   Thread 1: "Önce sen geç!" ──► Geri çekilir.
   Thread 2: "Hayır sen geç!" ──► Geri çekilir. (Sonsuz kibarlık döngüsü!)

3. STARVATION (AÇLIK):
   Açgözlü (Greedy) yüksek öncelikli thread'ler CPU'yu bırakmaz.
   Düşük öncelikli işçi thread günlerce kuyrukta bekler, asla sıra gelmez!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Livelock, dar bir koridorda karşı karşıya gelen iki aşırı kibar insanın hikayesidir. Biri sağa kaçar, diğeri de aynı anda onun sağına kaçar. Sonra ikisi birden sola kaçar. Sürekli ter döker, adım atarlar (CPU %100 yanar) ama ikisi de koridordan geçemez! Starvation (Açlık) ise açık büfe kuyruğunda öndeki VIP müşterilerin sürekli araya tanıdıklarını alması yüzünden arkadaki garibanın akşama kadar tek bir tabak yemek alamamasıdır.

🇬🇧 ENGLISH (ELI5)

Livelock is two overly polite pedestrians trying to pass each other in a narrow corridor. Both step to the left simultaneously. Both apologize and step to the right simultaneously. They are sweating profusely and sprinting back and forth (CPU 100%), but making zero forward progress! Starvation is standing at the back of a cafeteria line where VIPs continuously cut in ahead; the poor worker at the rear starves forever without receiving a meal.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üç Sessiz Katilin Mekaniği: 1) Livelock: Genellikle deadlock'u önlemek için yazılan `tryLock` mantığının yanlış uygulanmasıyla ortaya çıkar. Thread A kilidi alamayınca kendi kilidini bırakır; Thread B de aynı anda bırakır; sonra aynı anda tekrar almaya çalışırlar. Çözüm: Çarpışma anında rastgele gecikme eklemektir (Jitter / Exponential Backoff). 2) Thread Starvation: Adil olmayan (non-fair) kilitler veya CPU'yu hiç bırakmayan sonsuz döngülü thread'ler yüzünden düşük öncelikli görevlerin kaynak bulamamasıdır. Çözüm: `ReentrantLock(true)` adil kilit modu veya adil iş kuyruklarıdır. 3) Priority Inversion (Öncelik Tersinmesi): Düşük öncelikli bir thread kilit tutarken, orta öncelikli bir thread CPU'yu kapar ve düşük önceliklinin kilidi bırakmasını engeller; böylece yüksek öncelikli thread dolaylı olarak orta öncelikli tarafından rehin alınır! Çözüm: Öncelik Devralma (Priority Inheritance) protokolleridir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Silent Killers Analyzed: 1) Livelock: Frequently induced by naive retry loops paired with `tryLock()`. Both threads detect contention, release held locks simultaneously, wait an identical fixed interval, and attempt re-acquisition, looping indefinitely at 100% CPU. Remedy: Introduce randomized truncated exponential backoff (jitter). 2) Starvation: High-priority threads or unfair synchronization mechanisms (`synchronized` or non-fair `ReentrantLock`) continuously bypass threads parked in wait queues. Remedy: Deploy fair synchronization (`new ReentrantLock(true)`) or work-stealing schedulers. 3) Priority Inversion: Occurs when a low-priority thread holds a shared lock, an un-synchronized medium-priority thread preempts it on the CPU, preventing the low-priority thread from releasing the lock needed by a stalled high-priority thread. Remedy: Priority Inheritance Protocols.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Rastgele Gecikme (Jitter) ile Livelock'u Engelleme
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.locks.ReentrantLock;

public class LivelockResolutionDemo {
    private final ReentrantLock lockA = new ReentrantLock();
    private final ReentrantLock lockB = new ReentrantLock();

    public boolean executeWithJitter(String workerName) throws InterruptedException {
        while (true) {
            if (lockA.tryLock()) {
                try {
                    if (lockB.tryLock()) {
                        try {
                            System.out.println(workerName + " iki kilidi de aldı, kritik iş
yapılıyor!");
                            return true;
                        } finally {
                            lockB.unlock();
                        }
                    }
                } finally {
                    lockA.unlock();
                }
            }

            // ⚠ LIVELOCK ÇÖZÜMÜ: Eşzamanlı çakışmada rastgele bekle (Jitter Backoff)
            int jitterMs = ThreadLocalRandom.current().nextInt(10, 50);
            System.out.printf("%s kilit alamadı, %d ms rastgele uyuyor...%n", workerName, jitterMs);
            Thread.sleep(jitterMs);
        }
    }

    void main() throws Exception {
        Thread.ofVirtual().start(() -> {
            try { executeWithJitter("Worker-1"); } catch (Exception _) {}
        });
        Thread.ofVirtual().start(() -> {
            try { executeWithJitter("Worker-2"); } catch (Exception _) {}
        });
        Thread.sleep(200);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ReentrantLock'ı her zaman adil modda (fair=true) başlatmak performansı dramatik şekilde düşürebilir (throughput 10x-20x azalabilir)! Açlık sorunu gerçekten kanıtlanmadığı sürece varsayılan haksız (non-fair) modu koruyun.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Enabling fair mode (`new ReentrantLock(true)`) imposes severe throughput degradation (often 10x-20x slower) due to context switches and mandatory FIFO queuing. Reserve fair locks strictly for workloads where starvation is an empirically proven threat.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Livelock %100 CPU harcayıp ilerleyememektir; rastgele jitter ile çözülür. Açlık (starvation) ise bazı thread'lere hiç sıra gelmemesidir.

🎯 Key Takeaways for the Interview

• Livelock spins at 100% CPU without progress; resolve via randomized jitter. Starvation occurs when unfair queues starve low-priority threads.

52

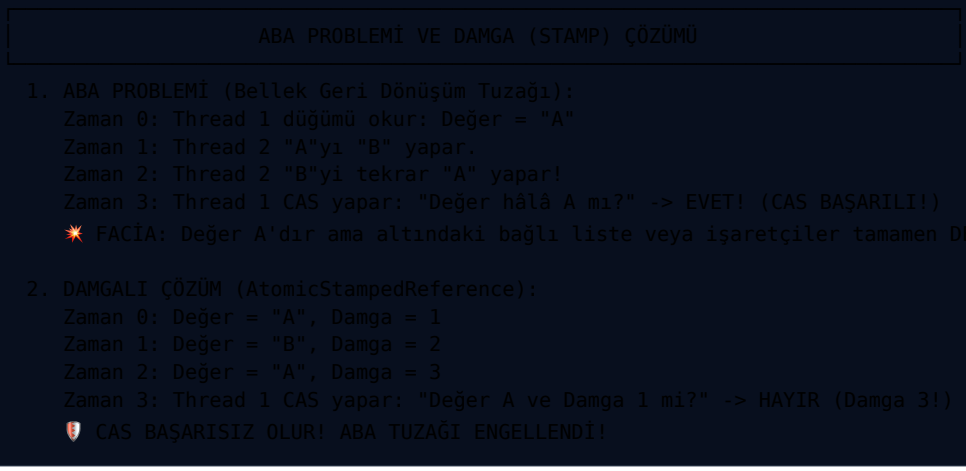
BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

ABA Problemi & Damgalı Referanslar: AtomicStampedReference vs AtomicMarkableReference

The ABA Problem & Stamped References: AtomicStampedReference vs Markable

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Masanın üstünde 100 liralık bir banknot bıraktınız. Siz kahve almaya gidince arkadaşınız o parayı alıp kumarda harcadı, sonra cebinden çıkarıp masaya yepyeni başka bir 100 liralık banknot koydu. Döndüğünüzde 'Aaa param burada duruyor' dersiniz ama o para aslında o para değildir! Arada koskoca bir kumar dönmüştür. AtomicStampedReference paranın üzerine görünmez bir versiyon damgası basar; para aynı kalsa bile damga değiştiği için kandırılmazsınız!

🇬🇧 ENGLISH (ELI5)

You leave a $100 bill on your desk. While you grab coffee, a thief steals it, gambles it away, wins, and places a completely different $100 bill back on your desk. You return and see '$100', falsely assuming nothing happened. In concurrent memory, the pointer value returned to 'A', but internal memory references were altered! `AtomicStampedReference` pairs the reference with an incrementing integer version stamp. Even if the value reverts to 'A', the mismatched stamp (version 3 vs version 1) halts the CAS instantly.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ABA Problemi ve Kilit-Siz (Lock-Free) Veri Yapıları: 1) Ortaya Çıkış: Lock-free yığınlarda (Treiber Stack) bir düğüm çıkarılırken (pop) üstteki düğüm A, onun altındaki B'dir. Thread 1, A'yı okur ve B'yi yeni baş yapmayı planlar. Araya giren Thread 2, A'yı ve B'yi çıkarır, sonra hafızadan geri dönüştürülen A nesnesini C'nin üstüne tekrar koyar. Thread 1 uyandığında yığının tepesinde hâlâ A'yı görür ve CAS yapar; yığın kafası B'ye geçer ancak B çoktan silinmiştir! Bellek bozulur. 2) AtomicStampedReference<V>: Referans ile int tipinde bir damgayı (`stamp`) atomik bir çift olarak tutar. `compareAndSet(expectedRef, newRef, expectedStamp, newStamp)` ile hem referansın hem damganın uyuştuğunu doğrular. 3) AtomicMarkableReference<V>: Damga yerine tek bitlik bir boolean (`mark`) tutar; nesnenin silinmek üzere işaretlendiğini (logical deletion) belirtmek için idealdir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ABA Hazard in Lock-Free Data Structures: 1) The Mechanics: In a lock-free Treiber Stack, Thread 1 observes top node `A` pointing to `B`. Before CAS completes, Thread 2 pops `A` and `B`, and then pushes a recycled instance `A` pointing to `C`. Thread 1 resumes and executes `CAS(top, A, B)`. The CAS succeeds because `top == A`, yet node `B` points to unallocated or corrupted memory. 2) `AtomicStampedReference\<V>`: Encapsulates both an object reference and an integer version counter. It invokes a dual-variable CAS (`compareAndSet(expectedRef, newRef, expectedStamp, newStamp)`). 3) `AtomicMarkableReference\<V>`: Pairs a reference with a single boolean mark, optimal for marking nodes as logically deleted before physical linked-list unlinking.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: AtomicStampedReference ile ABA Güvenli Hesap Güncellemesi
import java.util.concurrent.atomic.AtomicStampedReference;

public class AbaPreventionDemo {
    record AccountState(String status) {}

    void main() {
        var initial = new AccountState("ACTIVE");
        int initialStamp = 1;

        // Referans ve başlangıç versiyon damgası
        var accountRef = new AtomicStampedReference<>(initial, initialStamp);

        // Thread 1: O anki referansı ve damgayı okur
        int[] stampHolder = new int[1];
        AccountState observed = accountRef.get(stampHolder);
        int observedStamp = stampHolder[0];

        // Araya giren Thread 2: A -> B -> A döngüsü yapar (Damga artar!)
        var suspended = new AccountState("SUSPENDED");
        accountRef.compareAndSet(observed, suspended, observedStamp, observedStamp + 1);
        accountRef.compareAndSet(suspended, observed, observedStamp + 1, observedStamp + 2);

        System.out.println("Şu anki referans: " + accountRef.getReference().status());
        System.out.println("Şu anki güncel damga: " + accountRef.getStamp());

        // Thread 1 geri döner ve eski damgayla güncellemeye çalışır
        var closed = new AccountState("CLOSED");
        boolean success = accountRef.compareAndSet(observed, closed, observedStamp, observedStamp +
1);

        System.out.println("Thread 1 CAS başarılı oldu mu? " + success + " (ABA Engellendi!)");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

AtomicStampedReference dahili olarak her güncellemede yeni bir küçük nesne (Pair<V, Integer>) oluşturur! Çok yüksek yazma hacimli sistemlerde bu durum GC baskısı yaratabilir. Yüksek performanslı kütüphaneler (ör. Disruptor) damgayı long içinde bit maskeleme ile saklar.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`AtomicStampedReference` instantiates an internal `Pair\<V, Integer>` node upon state mutations, inducing GC allocation overhead under extreme write contention. Low-latency engines mitigate this by packing references and monotonic counters into a single 64-bit primitive `long` via bit shifting.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CAS sadece değerin eşitliğini kontrol eder, arada değişip geri gelmesini anlayamaz. Versiyon damgası (stamp) ekleyerek ABA hatasını önleyin.

🎯 Key Takeaways for the Interview

• CAS inspects current value equality, blind to intermediary transitions. Coupling references with monotonic version stamps prevents the ABA trap.

53

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

Production Thread Dump Analizi: jstack, jcmd, Thread Durumları & CPU Tavanlarını Teşhis

Production Thread Dump Analysis: jstack, jcmd & Diagnosing CPU Spikes

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                PRODUCTION THREAD DUMP ADLİ TIP REHBERİ

 CANLI SUNUCUDA CPU %100 VEYA KİLİTLENME VARSA:

 1. PID TESPİTİ:    top -H -p <PID> ──► En çok CPU yiyen Thread ID'yi bul (ör. 12948)
 2. HEX DÖNÜŞÜMÜ:   printf '%x\n' 12948 ──► "0x3294" (nid formatı)
 3. DUMP ALMA:      jcmd <PID> Thread.print > dump.tdump (veya jstack -l <PID>)
 4. NID ARAMA:      grep -A 20 "0x3294" dump.tdump ──► Suçlu metod ve satır karşında!

 DUMP'TA NE ARANIR?
 - "waiting to lock <0x00...>" ──► Kilidi kim tutuyor?
 - "locked <0x00...>"          ──► Kilidi tutan thread ne yapıyor?
 - "Found 1 deadlock."         ──► HotSpot deadlock'u otomatik listeler!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Thread dump, bir binadaki yangın anında bütün çalışanların dondurulup röntgenlerinin çekilmesidir. Kim elinde yangın tüpüyle bekliyor, kim kapıyı arkadan kilitlemiş, kim koridorda boşuna koşup terliyor tek bir fotoğrafta görünür. `top` komutuyla en çok terleyen personeli bulursun, thread dump ile de o personelin tam olarak hangi odada hangi kâğıdı imzalamaya çalıştığını saniyesi saniyesine yakalarsın!

🇬🇧 ENGLISH (ELI5)

A Thread Dump is an instantaneous radiographic snapshot of every single worker in your distributed factory. It captures who is sleeping, who is fighting over a wrench, and who is frantically running on a treadmill. Using `top -H` you pinpoint the exact OS thread ID consuming 100% CPU, convert it to hexadecimal (`nid`), and search the thread dump to reveal the exact Java file and line number executing the rogue loop.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Thread Dump Anatomisi ve Teşhis Teknikleri: 1) Üretim Komutları: Modern JDK'larda `jstack` yerine `jcmd \<PID> Thread.print -l` tercih edilir. Sanal thread'leri dökmek için `jcmd \<PID> Thread.dump_to_file -format=json dump.json` kullanılır. 2) Dump Başlığı Çözümleme: `"http-nio-8080-exec-12" #45 daemon prio=5 os_prio=0 tid=0x7f nid=0x3294 waiting on condition [0x7000]`. Burada `nid` (Native Thread ID) işletim sistemi thread ID'sinin hex karşılığıdır. 3) Kilit Zincirleri: `java.lang.Thread.State: BLOCKED (on object monitor)` görüyorsanız `waiting to lock \<0x00007f...>` adresini dump içinde aratıp o adresi `locked \<0x00007f...>` olarak elinde tutan thread bulunur. 4) Deadlock Tespiti: HotSpot JVM dökümün en altında `Found one Java-level deadlock` başlığıyla kilit döngüsünü otomatik raporlar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Thread Dump Anatomy & Diagnostic Protocol: 1) Modern Extraction Tooling: Prefer `jcmd \<PID> Thread.print -l` over legacy `jstack`. To analyze virtual threads, invoke `jcmd \<PID> Thread.dump_to_file -format=json threads.json`. 2) Header Dissection: In `"pool-1-thread-1" #12 prio=5 os_prio=0 cpu=142.12ms nid=0x1a4b waiting on condition`, `nid` is the native OS thread ID in hexadecimal. By mapping CPU-heavy PIDs from `top -H -p \<PID>` to hexadecimal, you identify the exact hotspot method trace. 3) Monitor Chain Resolution: Search for `waiting to lock \<0xADDR>` and grep for `locked \<0xADDR>` to construct lock dependency graphs. 4) JVM Deadlock Engine: HotSpot automatically scans for cyclic object monitor dependencies and appends a formal deadlock report to the dump tail.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Programatik Olarak Deadlock Tespiti ve Thread Dökümü Alma
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;

public class ThreadDumpDiagnosticDemo {
    public static void scanForDeadlocks() {
        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        long[] deadlockedThreadIds = bean.findDeadlockedThreads();

        if (deadlockedThreadIds != null && deadlockedThreadIds.length > 0) {
            System.err.println("🚨 KRİTİK ALARM: " + deadlockedThreadIds.length + " Thread Deadlock'a
girdi!");
            ThreadInfo[] infos = bean.getThreadInfo(deadlockedThreadIds, true, true);
            for (ThreadInfo info : infos) {
                System.err.printf("Deadlock Thread: %s (ID: %d) beklediği kilit: %s, kilidi tutan:
%s%n",
                    info.getThreadName(), info.getThreadId(),
                    info.getLockInfo(), info.getLockOwnerName());
            }
        } else {
            System.out.println("✅ Deadlock tespit edilmedi. Sistem sağlıklı.");
        }
    }

    void main() {
        scanForDeadlocks();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Production ortamında asla tek bir thread dump ile karar vermeyin! 10 saniye arayla en az 3 adet thread dump alın (ör. dump1, dump2, dump3). Eğer bir thread 3 dökümde de aynı satırda RUNNABLE durumundaysa sonsuz döngüdedir; eğer 3 dökümde de BLOCKED ise kilit açlığını gösterir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Never diagnose a production outage from a single thread dump! Always capture at least 3 consecutive snapshots spaced 5-10 seconds apart. A thread observed as `RUNNABLE` at the identical instruction line across all snapshots is trapped in an infinite loop; persistent `BLOCKED` states confirm lock starvation.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• top -H ile yüksek CPU kullanan thread ID'yi hex'e çevirin, jcmd Thread.print çıktısında 'nid' olarak aratıp sorunu nokta atışı tespit edin.

🎯 Key Takeaways for the Interview

• Convert high-CPU OS thread IDs to hex via `printf '%x'`, grep for `nid` in `jcmd Thread.print`, and trace the exact offending call stack.

54

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

VarHandle & Düşük Seviyeli Bellek Çitleri: Modern Java'da Unsafe'in Yerine Geçiş

Low-Level VarHandle & Memory Fences: Replacing sun.misc.Unsafe

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                    VARHANDLE BELLEK ERİŞİM MODLARI

Mod Adı              | Bellek Semantiği                | CPU Bariyeri
---------------------|---------------------------------|-----------------------
get() / set()        | Plain (Düz Okuma/Yazma)         | SIFIR Bariyer (En Hızlı)
getOpaque()          | Opaque (Derleyici optimiz. yok) | Reordering yok, bariyer yok
getAcquire()         | Acquire (Okuma Bariyeri)        | LoadLoad + LoadStore
setRelease()         | Release (Yazma Bariyeri)        | StoreStore + LoadStore
getVolatile()        | Volatile (Tam Sıralama)         | StoreLoad (En Katı)

Atomic RMW:
handle.compareAndSet(this, expected, update) ──► Donanımsal CMPXCHG
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski `Unsafe` sınıfı, eline dinamit verilmiş bir inşaat işçisi gibidir. Çok hızlıdır ama yanlışlıkla duvar yerine ana taşıyıcı kolonu patlatıp tüm JVM'i çökertebilir. Java 9 ve 25'teki `VarHandle` ise koruyucu kaskı, lazer hizalayıcısı ve sigortası olan yüksek teknolojili bir cerrahi alettir! Belleğe doğrudan, kilit almadan, ışık hızında dokunmanı sağlar ama hafızayı bozamayacağın güvenlik bariyerleriyle donatılmıştır.

🇬🇧 ENGLISH (ELI5)

Legacy `sun.misc.Unsafe` was handing a construction worker live dynamite. It bypassed all safety checks, but a 1-byte off-by-one pointer error crashed the JVM instantly with a Segfault. `VarHandle` (Java 9 - Java 25) is a precision laser scalpel. It offers the exact same raw, zero-allocation hardware access and atomic memory fences as `Unsafe`, but is fully type-safe, bounds-checked, and natively optimized by the JIT compiler.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

VarHandle Bellek Modeli ve Erişim Semantikleri: 1) Unsafe'in Emekliliği: `sun.misc.Unsafe` doğrudan bellek adresleriyle çalıştığı için JVM güvenliğini (Memory Safety) ihlal ediyordu. `java.lang.invoke.VarHandle`, MethodHandles API'si üzerinden değişkenlere atomik ve düşük seviyeli erişim sunar. 2) 5 Farklı Erişim Modu: a) Plain: Standart alan erişimi, reordering serbesttir. b) Opaque: Bytecode döngülerinde değişkenin önbelleğe alınmasını (hoisting) engeller, atomiktir ancak bellek bariyeri koymaz. c) Acquire/Release: Tek yönlü bariyerlerdir. `setRelease` öncesindeki yazmaların bu yazmadan sonra görünmesini garanti eder; tam `volatile` kadar pahalı değildir. d) Volatile: Çift yönlü tam bellek bariyeri (StoreLoad). 3) Düşük Seviyeli Bellek Çitleri: `VarHandle.fullFence()`, `acquireFence()`, `releaseFence()` gibi metotlarla CPU düzeyinde donanımsal bellek bariyerleri manuel olarak da tetiklenebilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

VarHandle Architecture & JVM Memory Modes: 1) Superseding `sun.misc.Unsafe`: `Unsafe` permitted raw memory dereferencing, provoking hard JVM segmentation faults. `VarHandle` integrates with `MethodHandles.Lookup`, enforcing variable type safety and reference validation while compiling down to native assembly intrinsics. 2) Granular Memory Access Modes: a) Plain: Normal reads/writes without fences. b) Opaque: Bitwise atomic with program-order coherence, preventing compiler hoisting while avoiding bus locks. c) Acquire/Release: One-way memory barriers. `setRelease` flushes preceding writes; `getAcquire` guarantees subsequent reads see updated state without incurring total StoreLoad fence penalties. d) Volatile: Enforces sequential consistency with global `StoreLoad` barriers. 3) Native CPU Fences: Exposes hardware primitives via `VarHandle.fullFence()` and `loadLoadFence()` for ultra-low-latency frameworks (e.g. Aeron, Disruptor).

⚡ Modern Java 25 LTS Kodu / ⚡ Modern Java 25 Implementation

```
// Java 25: VarHandle ile Kilit-Siz Atomik Sayaç ve Acquire/Release Semantiği
import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;

public class VarHandleCounter {
    private volatile int count = 0;

    // VarHandle tanımlayıcısı
    private static final VarHandle COUNT_HANDLE;

    static {
        try {
            // Field erişim kontrolü ve VarHandle oluşturma
            MethodHandles.Lookup lookup = MethodHandles.lookup();
            COUNT_HANDLE = lookup.findVarHandle(VarHandleCounter.class, "count", int.class);
        } catch (ReflectiveOperationException e) {
            throw new ExceptionInInitializerError(e);
        }
    }

    public void incrementRelaxed() {
        // Opaque veya Release modunda artırma (Volatile'dan daha ucuz CPU döngüsü!)
        int current;
        do {
            current = (int) COUNT_HANDLE.getOpaque(this);
        } while (!COUNT_HANDLE.compareAndSet(this, current, current + 1));
    }

    public int getAcquire() {
        return (int) COUNT_HANDLE.getAcquire(this);
    }

    void main() {
        var counter = new VarHandleCounter();
        counter.incrementRelaxed();
        System.out.println("VarHandle ile güncellenen sayaç: " + counter.getAcquire());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı / ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

VarHandle tanımlayıcıları (MethodHandles.Lookup) MUTLAKA `private static final` olarak tanımlanmalıdır! Eğer nesne düzeyinde (instance variable) veya metot içinde lookup yaparsanız JIT derleyicisi çağrıyı inline edemez ve performans 100 kat düşer.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`VarHandle` references MUST be declared `private static final`! If lookups are performed dynamically inside instance methods, the JIT C2 compiler cannot constant-fold the callsite into direct native assembly intrinsics, causing a catastrophic 100x performance drop.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar / 🎯 Key Takeaways for the Interview

- VarHandle, Unsafe'in güvenli ve modern halidir. Plain, Opaque, Acquire/Release ve Volatile modlarıyla CPU bariyerlerini milimetrik kontrol etmenizi sağlar.

- VarHandle replaces `sun.misc.Unsafe` with type-safe, JIT-intrinsic memory operations across Plain, Opaque, Acquire/Release, and Volatile barrier modes.

BÖLÜM 10 / MODULE 10

MAANG Eşzamanlılık Mülakatı & Sistem Tasarımı Ustalığı

MAANG Concurrency Whiteboard & System Design Mastery

MAANG mülakatlarında sorulan 4 efsanevi sistem tasarımı vakası: Thread-Safe O(1) LRU Cache, LMAX Disruptor RingBuffer kilit-siz mesajlaşma, Token-Bucket & Leaky-Bucket hız sınırlayıcı, HikariCP FastPath havuz mekaniği; 45 dakikalık mülakat stratejisi ve Staff+ mimar değerlendirme kriterleri.

Four legendary MAANG concurrency whiteboard challenges: Thread-Safe O(1) LRU Cache, LMAX Disruptor lock-free RingBuffer, Token-Bucket & Leaky-Bucket rate limiting, HikariCP ConcurrentBag connection pool architecture; plus the 45-minute interview blueprint and Staff+ architect evaluation criteria.

55

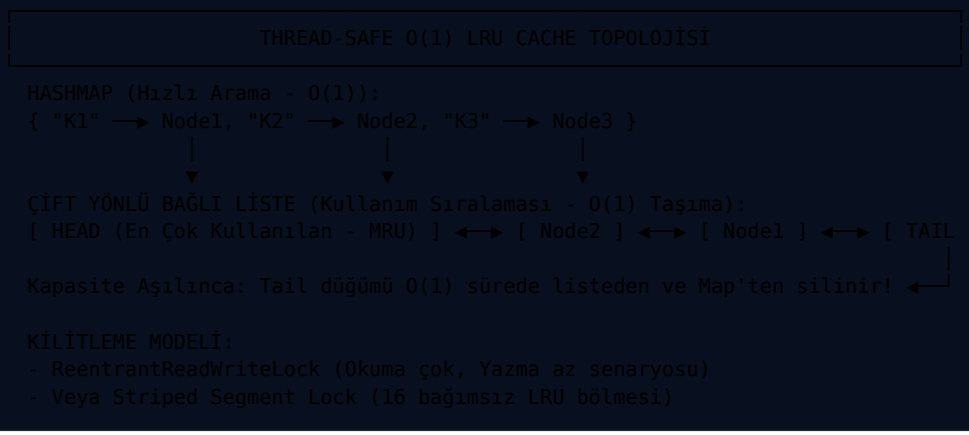
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 1: Thread-Safe O(1) LRU Cache Mimarisi (ReentrantReadWriteLock + Çift Bağlı Liste)

MAANG Case 1: Thread-Safe O(1) LRU Cache (Lock Striping & Linked Nodes)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphane masası düşünün. Masada en fazla 5 kitap durabiliyor. Birisi masadaki bir kitabı okuduğunda, o kitabı alıp masanın en sağına ('en son okunanlar' tarafına) koyarsınız. Eğer masaya yeni 6. bir kitap gelirse, masanın en solunda günlerdir kimsenin kapağını açmadığı en eski tozlu kitabı (`LRU - Least Recently Used`) çöpe atarsınız. Kitabı saniyede bulmak için bir fihrist (HashMap), sıralamayı anında güncellemek için ise iki yönlü bir tren vagonu zinciri (Doubly Linked List) kullanırız!

🇬🇧 ENGLISH (ELI5)

Imagine a coffee table that holds exactly 5 books. Whenever someone reads a book, you immediately slide it to the far right ('Most Recently Used'). If someone brings a 6th book and the table is full, you discard the book sitting at the far left that hasn't been touched in weeks ('Least Recently Used'). To find books in $O(1)$ time, we use an index card catalog (`HashMap`). To move books around in $O(1)$ time, we use a doubly linked train of cards (`DoublyLinkedList`). Both read and write operations maintain sub-millisecond thread safety!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesinde LRU Cache Tasarımı: 1) Veri Yapısı Seçimi: Yalnızca `HashMap` kullanılsaydı en eski elemanı bulmak $O(N)$ sürerdi. Yalnızca `LinkedList` kullanılsaydı arama $O(N)$ olurdu. `HashMap\<K, Node>` ile `DoublyLinkedList` birleştirildiğinde hem arama hem güncelleme $O(1)$ olur. 2) Eşzamanlılık Zafiyeti: LRU Cache'te `get(K)` işlemi MASUM BİR OKUMA DEĞİLDİR! Okunan elemanın listenin başına (`MRU`) taşınması gerektiği için liste üzerinde yapısal bir YAZMA (mutation) gerçekleşir. Bu yüzden saf `ReadLock` kullanılamaz; taşıma anında `WriteLock` gerekir. 3) Ölçeklenebilirlik Çözümü: Caffeine Cache gibi endüstri lideri kütüphaneler bu sorunu RingBuffer ve aktarım kuyrukları (Lossy / Event-Driven buffers) ile kilit almadan çözer. Mülakatta `ReentrantReadWriteLock` ile başlayıp Lock-Free buffer optimizasyonunu açıklamak Staff+ puanı kazandırır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Architectural Breakdown: 1) Structural Duality: Combining `HashMap\<K, Node\<K,V>>` with an internal Doubly-Linked List guarantees $O(1)$ key lookup and $O(1)$ eviction/re-linking. 2) The Read Mutation Trap: In LRU semantics, `get(key)` is NOT a read-only operation! Accessing an entry requires relocating its node to the Head (MRU), inducing structural mutations to adjacent pointer references. Consequently, standard read-locks fail; writers must acquire exclusive control. 3) Production Optimization: Explain to the interviewer that state-of-the-art caches (like Ben Manes' Caffeine) decouple reads from evictions using lock-free thread-local read buffers and a bounded MPSC ring-buffer to batch eviction updates asynchronously.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Thread-Safe LRU Cache Referans Uygulaması
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class ThreadSafeLruCache<K, V> {
    private static class Node<K, V> {
        K key;
        V value;
        Node<K, V> prev, next;
        Node(K k, V v) { this.key = k; this.value = v; }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null); // Dummy head
    private final Node<K, V> tail = new Node<>(null, null); // Dummy tail
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();

    public ThreadSafeLruCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public V get(K key) {
        rwLock.writeLock().lock(); // Erişim sıralaması değiştiği için write lock şart!
        try {
            Node<K, V> node = map.get(key);
            if (node == null) return null;
            moveToHead(node);
            return node.value;
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    public void put(K key, V value) {
        rwLock.writeLock().lock();
        try {
            Node<K, V> node = map.get(key);
            if (node != null) {
                node.value = value;
                moveToHead(node);
            } else {
                if (map.size() >= capacity) {
                    Node<K, V> lru = removeTail();
                    map.remove(lru.key);
                }
                Node<K, V> newNode = new Node<>(key, value);
                map.put(key, newNode);
                addToHead(newNode);
            }
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    private void addToHead(Node<K, V> node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void removeNode(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
    private void moveToHead(Node<K, V> node) {
        removeNode(node);
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatta LinkedHashMap(capacity, loadFactor, true) miras alarak removeEldestEntry metodunu ezmeyi önerebilirsiniz; ancak bu sınıf synchronized DEĞİLDİR! Collections.synchronizedMap() ile sarsanız bile kilit haritanın tamamını kilitler ve ölçeklenemez.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Subclassing `LinkedHashMap` and overriding `removeEldestEntry` is valid for rapid prototyping, but `LinkedHashMap` is not thread-safe. Wrapping it with `Collections.synchronizedMap()` imposes a coarse-grained bottleneck across the entire map, failing MAANG scalability reviews.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• LRU Cache'te okuma işlemi elemanın yerini değiştirdiği için yapısal bir yazmadır. HashMap + DoublyLinkedList birleşimi O(1) sağlar.

🎯 Key Takeaways for the Interview

• In LRU caches, reads mutate node order. Coupling a HashMap with a Doubly-Linked List guarantees \$O(1)\$ lookups and relocations under lock synchronization.

56

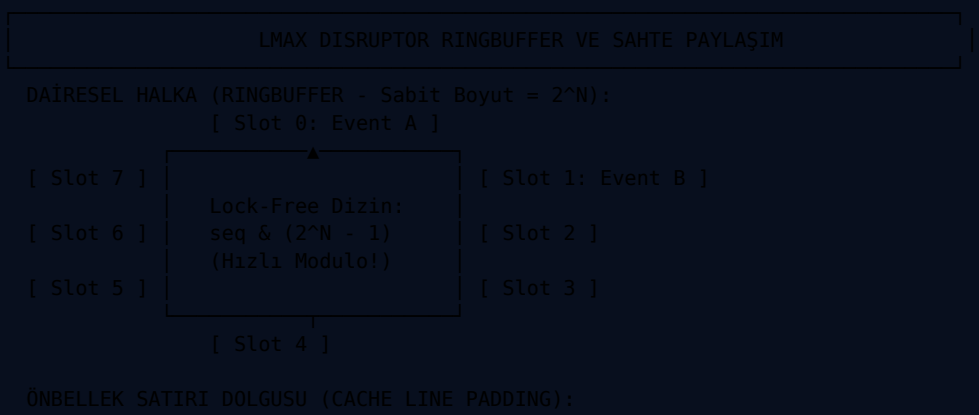
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 2: LMAX Disruptor Deseni & Lock-Free RingBuffer ile Ultra-Düşük Gecikme

MAANG Case 2: LMAX Disruptor & Lock-Free RingBuffer with Cache Padding

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Disruptor, Londra borsasındaki saniyede 6 milyon emri sıfır gecikmeyle işlemek için icat edilmiştir. Klasik kuyruklar iki kişinin birbirinin eline mektup tutuşturması gibidir (biri verirken diğeri bekler). RingBuffer ise döner bir dönme dolaptır. Sepetler önceden hazırdır ve hiç durmadan döner. Üretici sıradaki boş sepete kâğıdı fırlatır, tüketici arkasından gelip alır; ne kilit vardır ne bekleme! Bellek önbelleklerini (CPU Cache) korumak için de değişkenlerin sağına soluna boş dolgu konur, böylece çekirdekler birbirini hiç rahatsız etmez.

🇬🇧 ENGLISH (ELI5)

The LMAX Disruptor was engineered to process 6,000,000 orders/sec with nanosecond latency in financial exchanges. Standard queues use locks and dynamically allocate nodes, hammering the garbage collector. The Disruptor pre-allocates an immutable circular carousel (`RingBuffer`). Producers claim slots using monotonic atomic sequences, overwrite pre-allocated event slots in-place, and consumers read behind them with zero locks. Cache-line padding ensures CPU L1/L2 caches never suffer false sharing.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LMAX Disruptor'ın 4 Temel Mimari Sırrı: 1) Sıfır Çöp Üretimi (Zero-Allocation): RingBuffer boyutu 2'nin kuvvetidir (2^N). Tüm olay nesneleri uygulama başlarken heap'te önceden tahsis edilir (`pre-allocation`). Çalışma anında asla `new` çağrılmaz; aynı bellek yuvaları ezilerek yeniden kullanılır, GC duraklamaları (Stop-the-World) tamamen yok edilir. 2) İkili Maskeleme (Bitwise Indexing): Dizi indeksi hesaplanırken yavaş olan modulo (`%`) yerine `sequence & (bufferSize - 1)` bit düzeyinde VE işlemi kullanılır; tek bir CPU çevriminde slot bulunur. 3) Kilit-Siz Sıralama (Single-Writer Principle): Üretici sırayı tekil `volatile long cursor` ile takip eder; okuyucular CAS döngüsü olmadan `VarHandle.getAcquire` veya `Volatile` ile takip eder. 4) Sahte Paylaşım Önleme: Sıra değişkenleri arasına 56 baytlık dolgu (`padding`) yerleştirilerek değişkenin tek başına 64 baytlık L1 CPU Cache Line'ı kaplaması sağlanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Mechanical Sympathy of the LMAX Disruptor: 1) Zero Garbage Allocation: Array capacity is constrained to powers of two (2^N). Event slots are pre-allocated during initialization. Mutating state involves writing in-place to pre-existing slots, eliminating memory churn and GC pauses. 2) Bitwise Power-of-Two Indexing: Modulo arithmetic is replaced by bitwise masking (`sequence & (bufferSize - 1)`), executing in a single CPU clock cycle. 3) Lock-Free Mechanical Sympathy: Discards blocking queues and monitor locks in favor of monotonic sequence barriers backed by memory-fenced primitives. 4) Cache-Line Isolation: Sequence counters are isolated on separate 64-byte L1 cache lines using padding fields or `\@jdk.internal.vm.annotation.Contended`, preventing MESI cache invalidation storms.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Basitleştirilmiş Lock-Free RingBuffer Prensibi
import java.util.concurrent.atomic.AtomicLong;

public class MinimalDisruptorRingBuffer<T> {
    private final Object[] ringBuffer;
    private final int mask;
    // Cache Line Padding simülasyonu
    private final AtomicLong cursor = new AtomicLong(-1);

    public MinimalDisruptorRingBuffer(int powerOfTwoSize) {
        if (Integer.bitCount(powerOfTwoSize) != 1) {
            throw new IllegalArgumentException("Boyut 2'nin kuvveti olmalıdır!");
        }
        this.ringBuffer = new Object[powerOfTwoSize];
        this.mask = powerOfTwoSize - 1;
    }

    public long nextWriteSequence() {
        return cursor.incrementAndGet();
    }

    public void publish(long seq, T event) {
        int index = (int) (seq & mask); // Bitwise modulo (Işık hızında!)
        ringBuffer[index] = event;
    }

    @SuppressWarnings("unchecked")
    public T read(long seq) {
        int index = (int) (seq & mask);
        return (T) ringBuffer[index];
    }

    void main() {
        var buffer = new MinimalDisruptorRingBuffer<String>(1024);
        long seq = buffer.nextWriteSequence();
        buffer.publish(seq, "BIST100_LIMIT_ORDER_BUY_1000");

        String order = buffer.read(seq);
        System.out.println("RingBuffer'dan Okunan Emir: " + order);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

RingBuffer boyutu kesinlikle 2'nin kuvveti (1024, 4096, 65536) olmalıdır! Aksi halde `seq & (size - 1)` formülü çalışmaz ve dizi sınırları dışına taşma (ArrayIndexOutOfBoundsException) oluşur.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

RingBuffer capacity MUST strictly be a power of two (2^N). If breached, the bitwise indexing equation `sequence & (capacity - 1)` produces illegal indices, causing runtime exceptions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

• Disruptor sıfır çöp tahsisi, 2'nin kuvveti bit maskelemesi ve cache-line padding ile kilit almadan mikrosaniye altı gecikme üretir.

• The Disruptor achieves sub-microsecond latency through zero GC allocation, bitwise power-of-two ring indexing, and false-sharing cache padding.

57

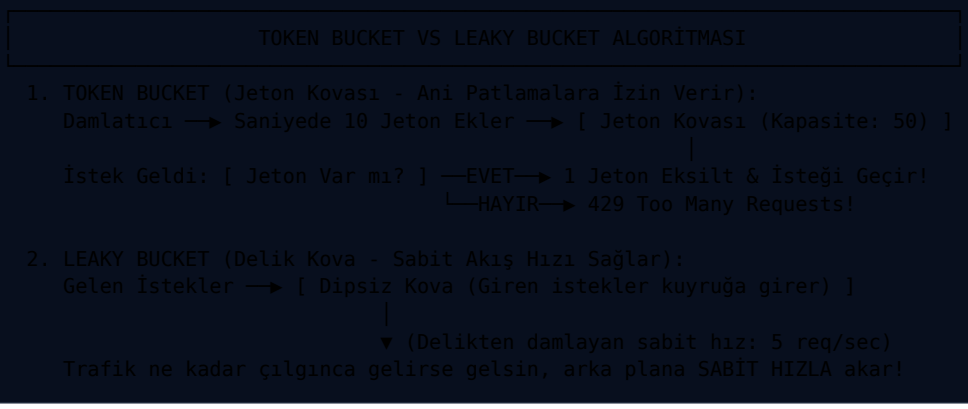
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 3: Dağıtık Token-Bucket & Leaky-Bucket Hız Sınırlayıcı (Rate Limiter)

MAANG Case 3: Distributed Token-Bucket & Leaky-Bucket Rate Limiter

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Token Bucket bir lunapark biletçisi gibidir. Kovaya her saniye 1 bilet atılır (en fazla 10 bilet birikir). Bir anda 5 arkadaş gelirseniz kovadaki 5 bileti alıp hemen dönme dolaba binebilirsiniz (patlamalı trafiğe izin verir). Bilet biterse kapıda kalırsınız. Leaky Bucket ise altı delik bir su kovasıdır. Kovaya yukarıdan bir kova suyu birden döksen de altındaki minik delikten su damla damla, sabit ve sakin bir hızla akar; arka plandaki narin veritabanını asla boğmaz!

🇬🇧 ENGLISH (ELI5)

Token Bucket is an arcade coin dispenser. It drops 1 token every second into a cup that holds up to 10 tokens. A burst of 5 gamers can grab 5 tokens simultaneously and play immediately (burst-tolerant). Once empty, newcomers are rejected. Leaky Bucket is a funnel with a tiny hole at the base. Even if a sudden flood of 100 gallons dumps into the funnel, water drips out of the bottom at a strictly constant rate of 1 drop/second, shielding fragile downstream services from traffic spikes.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Mülakatında Algoritma Karşılaştırması ve Kilit-Siz Uygulama: 1) Token Bucket Mantığı: İki ana değişken vardır: `capacity` ve `refillRate`. Her saniye bir zamanlayıcı thread çalıştırmak yerine 'Tembel Doldurma' (Lazy Refill) formülü uygulanır: `yeniJeton = min(kapasite, mevcutJeton + (simdikiZaman - sonYenilemeZamani) × refillRate)`. Bu sayede kova kontrolü yapay zeka gibi sıfır arka plan maliyetiyle yürütülür. 2) CAS ile Eşzamanlılık: `AtomicLong` veya `AtomicReference` kullanılarak jeton harcama işlemi kilit almadan tek çevrimde gerçekleştirilir. 3) Leaky Bucket Mantığı: Genellikle `ArrayBlockingQueue` veya Redis kuyruğu ile modellenir; gelen istekler kuyruğa alınır, bir tüketici thread sabit hızda (`rate`) kuyruktan iş çeker.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Algorithmic Breakdown: 1) Token Bucket Optimization: Avoid dedicated timer background threads. Implement 'Lazy Refill': compute accumulated tokens mathematically on-demand: `newTokens = min(capacity, tokens + (now - lastRefill) × rate)`. 2) Lock-Free State Mutation: Encapsulate token counts and timestamps in an immutable Java Record updated atomically via `AtomicReference.compareAndSet()`. 3) Leaky Bucket Topology: Models traffic shaping. Incoming requests enter a bounded FIFO buffer; an internal rate-governor drains requests at an uncompromising cadence, smoothing bursty traffic profiles before downstream ingress.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Lock-Free ve Tembel Doldurmalı Token Bucket Rate Limiter
import java.time.Instant;
import java.util.concurrent.atomic.AtomicReference;

public class AtomicTokenBucketLimiter {
    private final long capacity;
    private final double refillTokensPerSecond;

    // Durumu değişmez bir record içinde atomik tut
    private record State(long tokens, long lastRefillNanos) {}
    private final AtomicReference<State> state;

    public AtomicTokenBucketLimiter(long capacity, double refillTokensPerSecond) {
        this.capacity = capacity;
        this.refillTokensPerSecond = refillTokensPerSecond;
        this.state = new AtomicReference<>(new State(capacity, System.nanoTime()));
    }

    public boolean tryAcquire(long tokensRequested) {
        while (true) {
            State current = state.get();
            long now = System.nanoTime();
            long elapsedNanos = Math.max(0, now - current.lastRefillNanos());
            double newTokens = (elapsedNanos / 1_000_000_000.0) * refillTokensPerSecond;

            long updatedTokens = Math.min(capacity, current.tokens() + (long) newTokens);

            if (updatedTokens < tokensRequested) {
                return false; // Jeton yetersiz: İstek reddedildi (429)!
            }

            State next = new State(updatedTokens - tokensRequested, now);
            if (state.compareAndSet(current, next)) {
                return true; // Atomik olarak jeton düşüldü: İstek geçti!
            }
        }
    }

    void main() {
        var limiter = new AtomicTokenBucketLimiter(5, 2.0); // 5 kapasite, sn'de 2 jeton
        for (int i = 1; i <= 7; i++) {
            boolean allowed = limiter.tryAcquire(1);
            System.out.printf("İstek #%d Kabul Edildi mi? %s%n", i, allowed ? "✅ EVET" : "❌ 429
RED");
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Dağıtık sistemlerde (ör. Kubernetes üzerinde 50 pod) lokal RAM'deki Token Bucket işe yaramaz. Durumu Redis üzerinde Lua Script ile atomik yürütmeli veya NGINX / Envoy API Gateway katmanında merkezileştirmelisiniz.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Local in-memory rate limiters fail across horizontal deployments (e.g. 50 microservice pods). Synchronize global rate limiting states using atomic Redis Lua scripts or offload enforcement to edge gateways like Envoy or Cloudflare.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Token Bucket ani trafik patlamalarına izin verir (lazy refill ile kilit-siz hesaplanır). Leaky Bucket ise trafiği sabit hızda damlatarak pürüzsüzleştirir.

🎯 Key Takeaways for the Interview

• Token Bucket tolerates bursty ingress computed via lock-free lazy refills. Leaky Bucket smooths spikes into a constant-rate outflow.

58

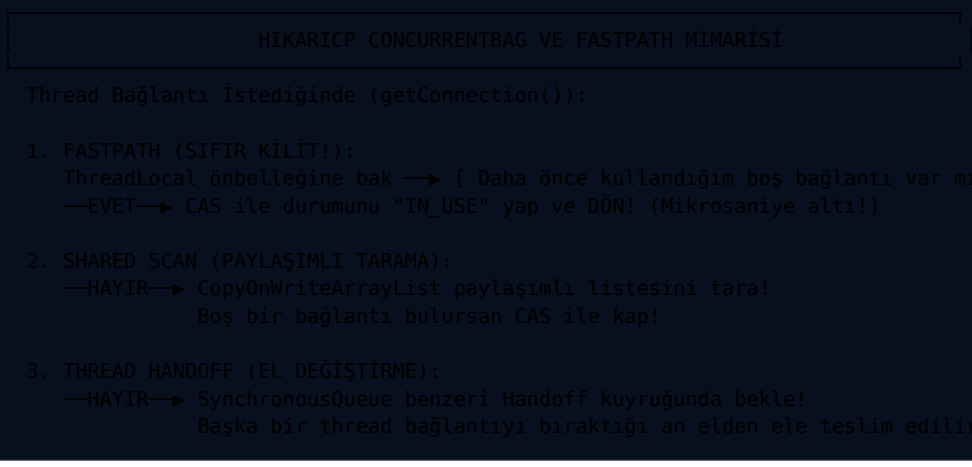
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 4: HikariCP FastPath Bağlantı Havuzu Mekaniği (ConcurrentBag & ThreadLocal Handoff)

MAANG Case 4: HikariCP FastPath Connection Pool (ConcurrentBag Architecture)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

HikariCP dünyanın en hızlı veritabanı bağlantı havuzudur. Eski havuzlar tek bir kapıcı koyar; 100 kişi o kapıcının önünde sıraya girerdi (`synchronized`). HikariCP ise şöyle der: 'Dün kullandığın anahtarı kendi cebine (`ThreadLocal`) koy. Yarın tekrar gelirsen kapıcıya hiç sormadan cebindeki anahtarı çıkarıp doğrudan içeri gir!' Cebinde anahtar yoksa masaya bakar, masada da yoksa anahtarı getirecek ilk kişiyi el sıkışma noktasında bekler!

🇬🇧 ENGLISH (ELI5)

HikariCP is universally recognized as the fastest JDBC connection pool on earth. Legacy pools (e.g. DBCP, c3p0) guarded connections behind a single synchronized gate. HikariCP implemented `ConcurrentBag`: each thread caches its most recently used connection in a `ThreadLocal`. When requesting a connection, it inspects its private thread cache first—if idle, it claims it with zero lock contention! If absent, it scans the shared bag; if empty, it registers on a zero-capacity handoff queue.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HikariCP ConcurrentBag'in 3 Katmanlı Çalma Hiyerarşisi: 1) Katman 1 (ThreadLocal FastPath): `threadLocalBagEntry.get()` çağrılır. Eğer referans varsa ve durumu `STATE_NOT_IN_USE` ise, tek bir atomik `CAS(STATE_NOT_IN_USE -> STATE_IN_USE)` ile kilit almadan bağlantı tahsis edilir. CPU context switch ve kilit çekişmesi sıfırdır. 2) Katman 2 (Shared List Scanning): ThreadLocal boşsa veya başka bir thread tarafından çalınmışsa, havuz `CopyOnWriteArrayList` üzerinde dolaşarak uygun boş bağlantıları CAS ile arar. 3) Katman 3 (Synchronous Handoff): Havuz tamamen meşgulse çağıran thread `SynchronousQueue` benzeri adil bir teslim kuyruğunda bekletilir. Başka bir thread `close()` çağırdığında bağlantıyı havuza iade etmek yerine doğrudan bekleyen thread'e teslim eder (Thread-to-Thread Handoff).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HikariCP ConcurrentBag Triple-Tiered Resolution: 1) Tier 1 (ThreadLocal FastPath): Queries thread-local cache. If a previously utilized entry is present in state `STATE_NOT_IN_USE`, an atomic CAS transitions state to `STATE_IN_USE`. Total time: nanoseconds, zero lock contention. 2) Tier 2 (Shared Bag Sweep): If the ThreadLocal is vacant or stolen, the thread iterates over a lock-free `CopyOnWriteArrayList` of pool entries, attempting CAS acquisitions. 3) Tier 3 (Synchronous Handoff Queuing): If all connections are occupied, the thread parks on a specialized `SynchronousQueue` handoff. When a loaner thread invokes `close()`, instead of recycling the entry to the shared list, it hands the connection off directly to the waiting worker thread.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: HikariCP ConcurrentBag FastPath Mantığının Minyatür Modeli
import java.util.concurrent.atomic.AtomicInteger;

public class MiniConcurrentBagPool {
    private static final int STATE_NOT_IN_USE = 0;
    private static final int STATE_IN_USE = 1;

    public static class BagEntry {
        final String connectionId;
        final AtomicInteger state = new AtomicInteger(STATE_NOT_IN_USE);
        public BagEntry(String id) { this.connectionId = id; }
    }

    // Her thread'in son kullandığı bağlantıyı tutan FastPath önbelleği
    private final ThreadLocal<BagEntry> fastPathCache = new ThreadLocal<>();
    private final BagEntry[] sharedBag;

    public MiniConcurrentBagPool(int size) {
        sharedBag = new BagEntry[size];
        for (int i = 0; i < size; i++) {
            sharedBag[i] = new BagEntry("CONN_JDBC_" + i);
        }
    }

    public BagEntry borrowConnection() {
        // 1. ADIM: FastPath (ThreadLocal'dan kilit-siz al!)
        BagEntry cached = fastPathCache.get();
        if (cached != null && cached.state.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
            System.out.println("⚡ FastPath ile anında alındı: " + cached.connectionId);
            return cached;
        }

        // 2. ADIM: Paylaşımlı havuzu tara
        for (BagEntry entry : sharedBag) {
            if (entry.state.compareAndSet(STATE_NOT_IN_USE, STATE_IN_USE)) {
                fastPathCache.set(entry); // Gelecek sefer için önbelleğe al!
                System.out.println("🔍 Shared Bag'den bulundu: " + entry.connectionId);
                return entry;
            }
        }
        throw new RuntimeException("Tüm bağlantılar dolu (Havuz Tükendi)!");
    }

    public void releaseConnection(BagEntry entry) {
        entry.state.set(STATE_NOT_IN_USE);
    }

    void main() {
        var pool = new MiniConcurrentBagPool(5);
        var c1 = pool.borrowConnection();
        pool.releaseConnection(c1);
        // İkinci istek FastPath üzerinden yıldırım hızında gelir!
        var c2 = pool.borrowConnection();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

HikariCP'de havuz boyutunu 1000 yapmak bağlantı hızını ARTIRMAZ, aksine veritabanı disk I/O ve CPU'sunu context switch ile boğar! HikariCP formülü: `pool_size = Tn × (CoreCount × 2 + EffectiveSpindleCount)` şeklindedir. Çoğu MAANG servisi için 10-20 bağlantı binlerce eşzamanlı isteği karşılamaya yeterlidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Configuring pool sizes to 500+ does NOT improve throughput; it cripples database CPU and disk subsystems with context-switching thrash. Follow the HikariCP sizing formula: `pool_size = CoreCount × 2 + SpindleCount`. For typical services, a modest pool of 10-20 connections out-scales bloated pools by 10x.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- HikariCP'nin sırrı ConcurrentBag'dir: ThreadLocal FastPath ile önce kendi önbelleğine bakar, yoksa paylaşımlı havuzdan CAS ile çalar.

🎯 Key Takeaways for the Interview

- HikariCP's supremacy lies in ConcurrentBag: ThreadLocal FastPath bypasses locks; shared sweeps and direct handoffs eliminate pool queuing.

59

BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

Eşzamanlılık Beyaz Tahta Planı: 45 Dakikalık MAANG Mülakat Yönetim Stratejisi

Concurrency Whiteboard Blueprint: 45-Minute MAANG Execution Strategy

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                45 DAKİKALIK MAANG CONCURRENCY MÜLAKAT ZAMAN ÇİZELGESİ

00-05 dk: GEREKSİNİMLERİ NETLEŞTİR (Clarify Requirements & Thread Safety Scope)
05-12 dk: VERİ YAPILARI & ÇEKİŞME ANALİZİ (Data Structures & Contention Points)
12-25 dk: ÇALIŞAN İLK KODU YAZ (Baseline Thread-Safe Code - Monolithic Lock)
25-35 dk: KİLİTSİZ / DETAYLI KİLİT OPTİMİZASYONU (Fine-Grained / CAS / Disruptor)
35-42 dk: SINIR DURUMLAR & TEŞHİS (Deadlock, Spurious Wakeups, Memory Leaks)
42-45 dk: ÖZET & STAFF+ MİMAR PERSPEKTİFİ (Metrics, Loom Virtual Threads, Tuning)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

MAANG mülakatı bir boks maçına benzer. Ringe çıkar çıkmaz gözün kapalı yumruk sallarsan (soruyu dinlemeden hemen kod yazmaya başlarsan) ilk rauntta nakavt olursun! Önce rakibini süz (soruyu sor: 'Kaç thread yazacak? Okuma mı çok yazma mı?'). Sonra gardını alıp sağlam bir vuruş yap (önce basit ama çalışan kilitli kodu yaz). Sonra özel hareketini göster (Lock-Free veya Virtual Thread optimizasyonunu açıkla). Hakemler şovuna hayran kalır!

🇬🇧 ENGLISH (ELI5)

A MAANG concurrency interview is like a surgical operation. If you grab the scalpel and start cutting before reviewing the patient's vitals (writing code without clarifying read/write contention ratios), the patient dies. First clarify invariants and boundaries. Second, provide a working baseline using clean abstractions. Third, optimize hot paths using fine-grained locks or lock-free CAS. Finally, walk through edge cases like thread starvation, spurious wakeups, and observability.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Mülakatçının Not Defteri ve 5 Kritik Değerlendirme Ekseni: 1) Soru Sorma (Requirement Clarification): 'Sistemimiz read-heavy mi yoksa write-heavy mi? Gecikme toleransı (SLA) nedir? Thread sayısı sabit mi yoksa dinamik mi?' sorularını sormak adayı anında Senior havuzuna taşır. 2) İlerlemeli Tasarım (Progressive Enhancement): Doğrudan aşırı karmaşık lock-free kod yazmaya çalışmayın! Önce `synchronized` veya `ReentrantLock` ile %100 doğru çalışan bir temel (baseline) kurun. Ardından 'Burada çekişme olursa Striped Locking veya ConcurrentHashMap'e evrilebiliriz' diyerek optimizasyona geçin. 3) Bellek Modeli Hissiyatı (JMM Fluency): Kod yazarken `volatile`, happens-before ve görünürlük etkilerini sesli olarak mülakatçıya anlatın. 4) Köşe Vakalar (Edge Cases): Spurious wakeups, interrupt durumunda thread bayrağını koruma (`Thread.currentThread().interrupt()`), ve kilitlerin `finally` içinde bırakılması adayın reflekslerini gösterir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Interviewer Scoring Rubric: 1) Clarification Phase: Proactively inquire about read-to-write ratios, thread counts, latency budgets, and memory constraints. 2) Progressive Implementation: Never dive prematurely into unverified lock-free wizardry. Implement an unassailable baseline using idiomatic locks first, then systematically profile and optimize hotspots to demonstrate maturity. 3) Mechanical & JMM Fluency: Articulate happens-before edges, CPU memory fences, and cache-line effects aloud while drafting variables. 4) Defensive Rigor: Always restore interrupted statuses, guarantee lock releases in `finally` blocks, and guard condition checks within `while` loops against spurious wakeups.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: MAANG Mülakatında Sunulacak İdeal Şablon Kodu
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class MaangInterviewTemplate<T> {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition condition = lock.newCondition();
    private boolean ready = false;

    public void waitForSignal() throws InterruptedException {
        lock.lock();
        try {
            // ⚠ MÜLAKAT KURALI 1: if değil daima WHILE kullan (Spurious Wakeup Savunması!)
            while (!ready) {
                condition.await();
            }
            System.out.println("Sinyal alındı, kritik operasyon devam ediyor...");
        } finally {
            // ⚠ MÜLAKAT KURALI 2: Kilit DAİMA finally içinde açılır!
            lock.unlock();
        }
    }

    public void sendSignal() {
        lock.lock();
        try {
            ready = true;
            condition.signalAll(); // Tüm bekleyenleri haberdar et
        } finally {
            lock.unlock();
        }
    }

    void main() {
        System.out.println("MAANG şablonu hazır.");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatçının 'Neden synchronized yerine ReentrantLock kullandın?' sorusuna hazırlıklı olun. Yanıtınız: tryLock (timeout), adil kilit seçeneği, çoklu Condition kuyruğu ve kesilebilir (lockInterruptibly) kilitleme yetenekleri olmalıdır.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Be prepared for the inevitable question: 'Why prefer ReentrantLock over synchronized?' Your crisp answer: timed polling via `tryLock()`, interruptible lock acquisition, fairness flags, and multiple independent `Condition` wait-queues.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

🎯 Key Takeaways for the Interview

• Mülakata önce gereksinimleri netleştirerek başlayın. Basit ve doğru bir kilitli temel kurun, ardından lock-free optimizasyonları ve köşe vakaları tartışın.

• Begin by clarifying contention ratios. Implement a robust baseline first, then progressively optimize with lock-free primitives and edge-case defenses.

60

BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

Staff+ Concurrency Mimarı: Canlı Sistem Çöküş Otopsisi & Çok Çekirdekli Ayarlar

The Staff+ Concurrency Architect: Production Post-Mortems & Tuning

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



```
                STAFF+ MİMARININ CANLI SİSTEM MÜDAHALE HARİTASI

CANLI SİSTEM ÇÖKÜŞÜ (INCIDENT):
- Semptom: 99.9th Percentile Gecikme 50ms'den 12 Saniyeye Fırladı!
- CPU Kullanımı: %98 (128 Çekirdekli Sunucu)

ADLİ PROTOKOL (POST-MORTEM WORKFLOW):
1. METRİKLERİ İNCELE  ──► JVM GC Pauses? (ZGC / Shenandoah kontrolü)
2. THREAD DUMP AL     ──► jcmd <PID> Thread.print ──► Kilit çekişmesi var mı?
3. PROFİLER BAĞLA     ──► async-profiler ──► Cache Misses & Lock Contention
4. MİMARİ MÜDAHALE:
   - Kaba kilitleri Striped Lock veya Lock-Free'e çevir
   - Platform thread havuzlarını Project Loom Sanal Thread'lerine göç ettir
   - False Sharing olan kritik alanlara @Contended dolgusu uygula!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir çırak, araba bozulunca 'Motoru değiştirelim' der. Kıdemli bir usta ise kulağını motora dayar, 3. silindirin contasındaki 1 milimlik yırtığı saniyesinde bulur. Staff+ Concurrency Mimarı, binlerce sunucunun saniyede milyonlarca işlem yaptığı bir MAANG şirketinde sistem kilitlendiğinde paniklemez. 3 dakikada thread dump alır, hangi thread'in hangi kilit yüzünden tıkandığını teşhis eder ve sistemi sıfır veri kaybıyla ayağa kaldırır!

🇬🇧 ENGLISH (ELI5)

A novice engineer sees an outage and suggests restarting all servers. A Staff+ Concurrency Architect approaches a production failure like a master forensic detective. With a 128-core server melting down at 99% CPU and p99 latency spiking to 10 seconds, the Staff Architect captures continuous low-overhead async-profiler flame graphs, identifies a single contested cache line or lock convoy in under 3 minutes, and executes a targeted hot-patch.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Staff+ Seviyesinde Üretim Optimizasyonu ve Donanım Uyumu (Mechanical Sympathy): 1) async-profiler ile Saftepoint Bias Önleme: Standart Java profiler'ları thread'leri yalnızca Safepoint anlarında örnekler; bu da yanıltıcı sonuçlar verir. `async-profiler` Linux çekirdeğinin `perf_events` alt yapısını kullanarak safepoint bağımsız gerçek CPU döngülerini ve kilit bekleme alev grafiklerini (Flame Graph) çizer. 2) NUMA (Non-Uniform Memory Access) Duyarlılığı: Çok soketli sunucularda bir CPU çekirdeğinin diğer soketin RAM'ine erişmesi yüksek gecikmeye yol açar. JVM `-XX:+UseNUMA` bayrağı ile donatılmalıdır. 3) Modern Çöp Toplayıcılar (ZGC & Shenandoah): Yüksek eşzamanlı sistemlerde STW (Stop-The-World) duraklamalarını 1 milisaniyenin altına indirmek için Generational ZGC (`-XX:+UseZGC`) tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Staff+ Architectural Mastery & Mechanical Sympathy: 1) Safepoint-Bias-Free Profiling: Standard JVM profiler samples exclusively at safepoint intervals, introducing severe observational skew. Deploy `async-profiler` leveraging Linux kernel `perf_events` to construct accurate off-CPU and lock contention flamegraphs. 2) NUMA Node Awareness: On multi-socket NUMA hardware architectures, cross-socket memory bus hops inflate latency. Enable `-XX:+UseNUMA` to co-locate thread allocations within local memory banks. 3) Sub-Millisecond GC Engine: Deploy Generational ZGC (`-XX:+UseZGC`) to eliminate Stop-The-World latency jitter, capping max GC pauses at under 1 millisecond across multi-terabyte heaps under heavy concurrent throughput.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Staff+ Seviyesi - Metrik Toplama ve Sağlık Denetimi
import java.lang.management.ManagementFactory;
import java.lang.management.OperatingSystemMXBean;
import java.lang.management.ThreadMXBean;

public class StaffArchitectDiagnostics {
    public static void printSystemHealthReport() {
        OperatingSystemMXBean os = ManagementFactory.getOperatingSystemMXBean();
        ThreadMXBean threads = ManagementFactory.getThreadMXBean();

        System.out.println("================================================");
        System.out.println("🛡️ STAFF+ PRODUCTION HEALTH MONITORING REPORT");
        System.out.println("================================================");
        System.out.printf("Mevcut CPU Yükü: %.2f%%%n", os.getSystemLoadAverage() * 100);
        System.out.printf("Aktif Canlı Thread Sayısı: %d%n", threads.getThreadCount());
        System.out.printf("Zirve Thread Sayısı: %d%n", threads.getPeakThreadCount());
        System.out.printf("Toplam Başlatılan Thread: %d%n", threads.getTotalStartedThreadCount());

        long[] deadlocks = threads.findDeadlockedThreads();
        System.out.println("Deadlock Durumu: " + (deadlocks == null ? "SIFIR (Temiz)" : "🧯 " +
deadlocks.length + " Thread Kilitli!"));
        System.out.println("================================================");
    }

    void main() {
        printSystemHealthReport();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Canlı sistemde sorun çıkınca servisi hemen yeniden başlatmak (restart) kök nedeni sonsuza kadar yok eder! Restart atmadan önce MUTLAKA en az 3 adet thread dump ve heap dump (`jcmd \<PID> GC.heap_dump`) alın; delilleri koruyun.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Blindly restarting servers during a production outage destroys forensic evidence forever! Always automate taking 3 spaced thread dumps and a memory snapshot via `jcmd \<PID> GC.heap_dump` prior to restarting instances to preserve root-cause traces.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Staff+ mimar ezbere çözüm üretmez; metrikleri, thread dökümlerini ve donanım mimarisini (NUMA, Cache Line, ZGC) birleştirerek kök nedeni çözer.

🎯 Key Takeaways for the Interview

• A Staff+ architect marries software abstractions with hardware topology: profilers, lock contention traces, NUMA awareness, and sub-millisecond ZGC.

🎓 TEBRİKLER! / CONGRATULATIONS!

60 Derslik MAANG Java Concurrency & Multithreading Yolculuğunu Başarıyla Tamamladınız!

You have conquered all 60 lessons of the MAANG Java Concurrency & Multithreading Masterclass!

Java 25 LTS • Virtual Threads • JMM • Lock-Free Algorithms • LMAX Disruptor • HikariCP